



Ike Starter Set

Table of Contents

How to Read This Guide	1
The Identicon	1
Koncept Badges	1
The Component Kinds	2
Pattern-Shape Tables	4
The Glossary and the List of Patterns	5
1. What This Set Provides	5
2. Language Concepts	6
3. STAMP Concepts	9
3.1. Status	9
3.2. Author	9
3.3. Module	9
3.4. Path	10
3.5. Edit Coordinate	12
4. Logic Coordinates	13
5. Navigation Coordinates	14
6. EL++ Concepts	16
6.1. Roles and Role Operators	16
6.2. Inclusion, Necessary, and Sufficient Sets	16
6.3. Concrete Value Operators	17
6.4. Logical Definitions and Grouping	17
7. Semantic Field Model	19
7.1. Field Categories	19
7.2. Display Fields	20
7.3. Field Value Terminology	20
7.4. Constraining Fields	21
8. Field Constraints	23
8.1. The Taxonomy Field Constraint	24
8.2. The Value-set Field Constraint	26
8.3. The Member Match Relation	27
8.4. The Apparatus Constrains Itself	28
8.5. Constraints and Defaults	29
9. Default Values	29
9.1. The Default Value Semantic	29
9.2. The Defaults and Templates Module	30
9.3. Templates	30

9.4. Loud Defaults for Every Data Type.....	30
9.5. A Worked Default: Preferred Reviewer	34
10. Tinkar Base Model.....	34
10.1. Components and Chronicles.....	35
10.2. Base Model Primitives.....	42
11. Value Constraints and Reference Ranges.....	42
12. Authoring Actions and Legacy Constraints	44
13. Feature-Based and Intrinsic Roles	44
14. Annotations, Correlations and Editor Focus.....	45
15. Identifiers	47
16. Coordinate Properties: Language and Path.....	48
17. Tree Amalgams.....	49
18. Clinical Phenomena and Domain Content	49
19. Object Properties and Relationships.....	49
20. Provenance and Root.....	50
21. Konzept Glossary	50

How to Read This Guide

The Identicon

Every component in the knowledge base has a formal public identity. This guide makes that identity visible as an **identicon**: a small square image computed deterministically from the component's own public identifier — a visual hash of identity, and nothing else. Its purpose is recognition: the same component draws exactly the same identicon everywhere it appears — in this guide's HTML and PDF, in print, and onscreen in applications such as Komet, all of which compute it from the same identity data — so you can recognize a component at a glance, and see that two references name the same thing, before reading a word. Because it is derived from identity alone, an identicon carries no meaning about what the component is; the name and the sigil carry that.

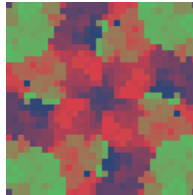


Figure 1. The identicon of English Language, computed from its public identifier — the same image wherever this component appears

Koncept Badges

Every formally identified term in this guide renders as a **Koncept Badge**: a kind sigil when the kind calls for one (next section), then the identicon, then the component's name in small caps. A sigil always immediately precedes an identicon — it marks what kind of thing the identicon identifies, and never stands alone in a badge. A badge is not styling; it is a reference to one exact component in the knowledge base, and it links to that koncept's entry in the Koncept Glossary at the end of this book, where its definition, axioms, and identifiers live. The prose is knowledge-base-native too: each chapter's narrative is itself stored in the knowledge base and rendered here, so the same content backs this guide and every other surface that reads the set.

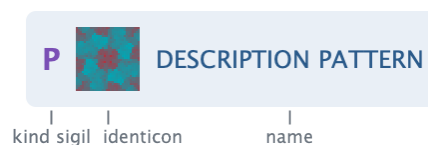


Figure 2. Anatomy of a Koncept Badge — Description Pattern's badge, dissected

A STAMP badge follows the same anatomy with provenance in place of a name — shown here for this set’s own inception STAMP, declared in the ledger source as Active · Inception · IKE Community · IkeFoundation module · Primordial path. Inception is the platform’s own named instant for time before any release — every version in this pre-release set carries it, and it renders as the word rather than a date on every surface.



Figure 3. Anatomy of a STAMP badge — the set’s inception stamp, dissected

The Component Kinds

Everything in the knowledge base is one of a small set of component kinds, and a badge is honest about which kind it names. The scheme is *Koncept bare, everything else marked*: a Koncept badge carries no extra mark, and every other kind announces itself with a sigil ahead of the identicon.

Table 1. The component kinds, their badge sigils, and a concrete example of each

Kind	Badge	What it is
Koncept	no sigil — bare e.g.  ENGLISH LANGUAGE	A Koncept — with a K — is the formal construct in the knowledge base, deliberately distinct from a concept in the mind. A concept is the thought a person holds; a Koncept is the identified, versioned construct that carries descriptions (its names) and a logical definition (its meaning) — formally, the referent of the semiotic triangle: the thing the badge’s name symbolizes and a reader’s concept refers to. Most badges in this guide are Koncepts.

Kind	Badge	What it is
Pattern	P violet P e.g. P  DESCRIPTION PATTERN	The schema for a family of semantics: what a semantic of this pattern means as a whole, and the ordered fields it carries — each with its own meaning, purpose, and data type. Every pattern in this set has its own shape table (see below).
Description	D amber D e.g. D UNINITIALIZED COMPONENT (SOLOR)	Human-readable text bound to a concept — itself a semantic, of P  DESCRIPTION PATTERN or a dialect pattern. Every name rendered in this guide, including the text inside each badge, is a description in the knowledge base.
Semantic	S green S e.g. S GRETEL	A structured assertion attached to any component, instantiating exactly one pattern: its fields hold the values, in the pattern's declared order. The <i>e.g.</i> rows in the shape tables are read from real semantics.

Kind	Badge	What it is
STAMP	 grey pentagon e.g. 	The provenance every version of every component carries: status, time, author, module, and path. A STAMP badge leads with the pentagon sigil — a point-up five-dimensional radar, one reading per STAMP dimension — then the STAMP's own identicon, which tells one STAMP from another at a glance, then the compact provenance text (status, time, author) in place of a name: a STAMP is identified like every component, but what it says is provenance.

The concept and pattern examples are live badges — click either to visit its glossary entry. The description, semantic, and STAMP examples are *specimen* badges drawn from the set's real content — its earliest-authored description, the author roster's first entry, and the set's inception provenance — shown unlinked and without identicons because those components are not named terms in this guide's glossary. On live surfaces that render individual semantics and versions (Komet itself, chat and mail integrations) the D, S, and pentagon badges appear in full, identicons included; the badge language is the same across all of them. A red question-mark sigil (?) marks a reference whose kind could not be determined.

Pattern-Shape Tables

Each pattern's chapter renders its shape as a numbered table — for example Description Pattern shape — organized as one three-row group per **Model Feature**: first the pattern's referenced component, then each declared field, 1-indexed, in declared order.

- An italic label row names the feature: *Referenced component* (the component — concept, semantic, pattern, or STAMP — this pattern's semantics attach to) or *Field N*. The referenced component is never numbered "Field 0" because it is not part of the fields array: it is a chronicle-level property of the semantic, structurally separate from the field values.
- A content row names the feature's **Meaning** — the concept saying what a value of this feature is — its **Purpose** — the concept saying why the pattern carries it — and its **Data type**, drawn from the closed set of data-type concepts the platform actually resolves. The referenced component has no data type, so that cell is an em dash.

- An indented *e.g.* row shows the feature’s value on one real semantic already in the knowledge base — deterministically the earliest-authored one, so the example is stable across regenerations — rendered as a badge when the value is itself an identified component, and as plain text otherwise. A pattern with no semantics yet simply has no *e.g.* rows.
- An indented *default* row shows the field’s value from the pattern’s default value semantic (see the Default Values chapter), rendered the same way. Only a pattern with an authored default carries these rows.

The Glossary and the List of Patterns

Every koncept referenced anywhere in this guide has an entry in the comprehensive, alphabetical Koncept Glossary that closes the book — definition, axioms, identifiers, and taxonomy links to parents and children. The List of Patterns closing the first chapter indexes every pattern’s glossary entry, the same book convention as a List of Tables or a List of Figures. Occasional indented concept trees show a live taxonomy subtree of the knowledge base, each node a badge.

1. What This Set Provides

The IkeFoundation knowledge set declares its content as an append-only ledger, rooted at  **IKEFOUNDATION ROOT** ; this guide and its comprehensive glossary are generated from the same materialized store as the released change set.

The chapters that follow describe how this set’s own concepts support the platform’s core mechanisms — language and dialect, STAMP versioning, logical and navigational coordinates, EL++, the field-level meta-schema, and the base component/chronicle model itself — each naming the actual koncepts involved. The Koncept Glossary at the end lists every koncept in the set, alphabetically, for lookup.

Every pattern in the set is defined by a referenced-component meaning and purpose, plus a declared, ordered list of fields, each with its own meaning, purpose, and data type — a pattern’s own semantics are exactly the set of components carrying that pattern’s public id. The list below indexes all of them — a List of Patterns, the same book convention as a List of Tables or a List of Figures, styled like the table of contents above — each entry linking to that pattern’s own full shape, rendered as a table in the Koncept Glossary.

List of Patterns

- 1. Comment pattern
- 2. Component Chronology Pattern
- 3. Component Version Pattern
- 4. Concept field pattern
- 5. Concept Version Pattern
- 6. Data Type Defaults Pattern
- 7. Description Pattern

- 8. EL++ Inferred Axioms Pattern
- 9. EL++ Stated Axioms Pattern
- 10. GB Dialect Pattern
- 11. Identifier Pattern
- 12. Inferred Navigation Pattern
- 13. Komet base model component pattern
- 14. Module origins pattern
- 15. OWL Axiom Syntax Pattern
- 16. Path origins pattern
- 17. Pattern Chronology Pattern
- 18. Pattern Version Pattern
- 19. Preferred Reviewer Pattern
- 20. Semantic Chronology Pattern
- 21. Semantic version field pattern
- 22. SOLOR concept assemblage
- 23. Solor Concepts Pattern
- 24. STAMP Chronology Pattern
- 25. STAMP pattern
- 26. STAMP version field pattern
- 27. Starter Set Author Roster Pattern
- 28. Stated Navigation Pattern
- 29. Taxonomy Field Constraint Pattern
- 30. Tinkar base model component pattern
- 31. US Dialect Pattern
- 32. Value Constraint Pattern
- 33. Value-set Field Constraint Pattern
- 34. Version control path pattern

2. Language Concepts

Every human-readable name in the knowledge base is a description: a  **DESCRIPTION PATTERN** semantic attached to a concept, carrying the description's own language, its text, its case significance, and which of three description types it is —

 **FULLY QUALIFIED NAME DESCRIPTION TYPE** (the one formal, unique name every concept must have),  **REGULAR NAME DESCRIPTION TYPE** (a synonym), or  **DEFINITION DESCRIPTION TYPE** (the natural-language definition text this guide renders for every concept).

A description's acceptability within a dialect is a separate semantic layered on top of it:

  **US DIALECT PATTERN** and   **GB DIALECT PATTERN** each record whether a description is  **PREFERRED** or merely  **ACCEPTABLE** for that dialect, so the same concept can carry a British spelling and an American one side by side, each preferred in its own dialect. Language itself is a concept too —  **ENGLISH LANGUAGE** and its siblings — not a string constant.

A `LanguageCoordinate` picks one description out of all of a concept's candidates by walking four preference lists, each itself modeled as a concept: `LANGUAGE NID FOR LANGUAGE COORDINATE` (which language to prefer), `DESCRIPTION TYPE PREFERENCE LIST FOR LANGUAGE COORDINATE` (fully qualified name before regular name, or the reverse), `DIALECT PATTERN PREFERENCE LIST FOR LANGUAGE COORDINATE` (US before GB, or the reverse), and `MODULE PREFERENCE LIST FOR LANGUAGE COORDINATE` (a final tie-break when two descriptions are otherwise equally preferred). The calculator matches language first, then walks description-type order, then prefers a dialect acceptability of `PREFERRED` matching the dialect preference order, then breaks any remaining tie by module preference.

As a pattern, `DESCRIPTION PATTERN` itself carries a referenced-component meaning of `DESCRIPTION SEMANTIC` — "purpose and meaning for the description pattern and dialect patterns," in its own words, though a later revision (IKE-Network/ike-issues#880) gives it a distinct purpose too: `DESCRIPTION ATTACHMENT`, naming why the pattern exists — to attach a human-readable name or definition to a concept. Four fields each carry their own meaning and purpose concept: `Language Concept Nid For Description` (meaning) for `LANGUAGE` (purpose), `TEXT FOR DESCRIPTION` (meaning) for `DESCRIPTION` (purpose), `DESCRIPTION CASE SIGNIFICANCE` (meaning) for `CASE SENSITIVITY RULE` (purpose — whether the text is case-sensitive), and `DESCRIPTION TYPE` (meaning) for `DESCRIPTION ROLE` (purpose — which of the three description roles this one plays).

The two dialect patterns named above share one shape, differing only in which national dialect each is attached to: `US DIALECT PATTERN` and `GB DIALECT PATTERN` each carry a referenced-component meaning of `DESCRIPTION ACCEPTABILITY` and a purpose of `DESCRIPTION SEMANTIC`, with one field — its meaning `UNITED STATES OF AMERICA ENGLISH DIALECT` for the US pattern, `GREAT BRITAIN ENGLISH DIALECT` for the GB one, its purpose `DESCRIPTION ACCEPTABILITY` in both — the field a description's `PREFERRED` / `ACCEPTABLE` acceptability value is actually recorded against.

Description Pattern shape shows the pattern's full shape: each Model Feature — the referenced component, then each field in declared order — with its meaning, purpose, and data type, and a live example value drawn from a real semantic already in the set.

Table 2. Description Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
<code>DESCRIPTION SEMANTIC</code>	<code>DESCRIPTION ATTACHMENT</code>	—
e.g. <code>UNINITIALIZED COMPONENT</code>		

Meaning	Purpose	Data type
<i>Field 1</i>		
 LANGUAGE FOR DESCRIPTION	 LANGUAGE	 COMPONENT DATA TYPE
<i>e.g.</i>  <i>ENGLISH LANGUAGE</i>		
<i>Field 2</i>		
 TEXT FOR DESCRIPTION	 DESCRIPTION	 STRING DATA TYPE
<i>e.g. Uninitialized Component (SOLOR)</i>		
<i>Field 3</i>		
 DESCRIPTION CASE SIGNIFICANCE	 CASE SENSITIVITY RULE	 COMPONENT DATA TYPE
<i>e.g.</i>  <i>DESCRIPTION NOT CASE SENSITIVE</i>		
<i>Field 4</i>		
 DESCRIPTION TYPE	 DESCRIPTION ROLE	 COMPONENT DATA TYPE
<i>e.g.</i>  <i>FULLY QUALIFIED NAME DESCRIPTION TYPE</i>		

The dialect patterns that layer acceptability on top of a description are shown in US Dialect Pattern shape and GB Dialect Pattern shape.

Table 3. US Dialect Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 DESCRIPTION ACCEPTABILITY	 DESCRIPTION SEMANTIC	—
<i>e.g. Uninitialized Component (SOLOR)</i>		
<i>Field 1</i>		
 UNITED STATES OF AMERICA ENGLISH DIALECT	 DESCRIPTION ACCEPTABILITY	 COMPONENT DATA TYPE
<i>e.g.</i>  <i>PREFERRED</i>		

Table 4. GB Dialect Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 DESCRIPTION ACCEPTABILITY	 DESCRIPTION SEMANTIC	—
Field 1		
 GREAT BRITAIN ENGLISH DIALECT	 DESCRIPTION ACCEPTABILITY	 COMPONENT DATA TYPE

3. STAMP Concepts

Every version of every component carries a STAMP: a (status, author, module, path, time) tuple recording who committed it, in what state, in which module, on which path, and when. Four of those five dimensions are themselves concepts in this set.

3.1. Status

 STATUS VALUE is the concept field holding a version's state — one of  ACTIVE STATE ,  INACTIVE STATE ,  PRIMORDIAL STATE ,  CANCELED STATE , or  WITHDRAWN STATE . A StampCoordinate doesn't accept just any state: its  ALLOWED STATES FOR STAMP COORDINATE narrows which of these count as "current" when the calculator resolves a component's latest version — ordinarily just  ACTIVE STATE , so a retired or withdrawn version stops being what callers see by default without ever being deleted.

3.2. Author

 AUTHOR identifies who committed a version, carried by the  AUTHOR FIELD concept field. A version's own author is  AUTHOR FOR VERSION ; a StampCoordinate doesn't filter by author (identity, unlike state, module, and path, isn't part of "what's current"), but an EditCoordinate does, via  AUTHOR FOR EDIT COORDINATE — the author a new edit is stamped with.

 AUTHOR FOR STAMP COORDINATE names the analogous idea should a future coordinate ever need to filter by author.

3.3. Module

 MODULE groups versions by the export or authoring boundary they belong to — the  MODULE FIELD concept field.  DEVELOPMENT MODULE and  PRIMORDIAL MODULE are the two this starter set's own content is stamped with. A StampCoordinate narrows which modules count via  MODULE PREFERENCE LIST FOR STAMP COORDINATE (preference order among several acceptable modules) and  MODULE EXCLUSION SET FOR STAMP COORDINATE (modules to ignore outright).

A module's own lineage is tracked the same way a path's is: **P**  **MODULE ORIGINS PATTERN** carries a referenced-component meaning of **ORIGINATED MODULE** — the module whose origins are declared — and a purpose of **MODULE LINEAGE** — why the record exists: to track which modules a given module originated from — with one field, **MODULE ORIGINS** (meaning) for **ORIGIN MODULE SET** (purpose), holding the set of modules this module originated from — the module-level counterpart to **P**  **PATH ORIGINS PATTERN** below.

Module origins pattern shape shows the module-origins pattern's shape.

Table 5. Module origins pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 ORIGINATED MODULE	 MODULE LINEAGE	—
<i>Field 1</i>		
 MODULE ORIGINS	 ORIGIN MODULE SET	 COMPONENT ID SET DATA TYPE

3.4. Path

PATH is the version-control lineage a version was committed on — the **PATH FIELD** concept field. This starter set declares four paths and their origin graph: **PRIMORDIAL PATH** is the root — no path precedes it, and released foundational knowledge lands on it, beginning with this set's own inception release (every version in this pre-release set carries the inception stamp pair, on Primordial); **DEVELOPMENT PATH** originates from Primordial at Latest, so a view positioned on Development inherits every release beneath it — after the 1.0 seal, editing moves to Development and never returns to Primordial at inception; **SANDBOX PATH** forks off Development at Latest, keeping experiments visible over mainline content without ever being inherited by it; and **MASTER PATH** carries released consumer-facing content. **PATH ORIGINS FOR STAMP PATH** records which path a path itself branched from, so a **StampCoordinate** positioned on one path can still resolve versions committed on an ancestor path before the branch point.

As a pattern, **P**  **PATH ORIGINS PATTERN** itself carries a referenced-component meaning of **ORIGINATED PATH** — the path whose branch point is declared — and a purpose of **PATH LINEAGE** — why the record exists: to track which path it branched from, and when — and two fields: **PATH CONCEPT** (meaning) for **BRANCH SOURCE** (purpose — which path this path branched from; the record's subject is the referenced component, never this field) and **PATH ORIGINS** (meaning) for **BRANCH POINT** (purpose — the moment this path branched, up to which the origin path's versions are visible from it). **P**  **VERSION CONTROL PATH PATTERN** is simpler still — a referenced-component meaning of **PATH** and purpose of **SET MEMBERSHIP** ,

with no fields of its own: a semantic of it merely marks a component as belonging to version-control path management, the same bare membership-pattern convention this guide's Tinkar Base Model chapter describes for  **TINKAR BASE MODEL COMPONENT PATTERN** and  **KOMET BASE MODEL COMPONENT PATTERN**.

Path origins pattern shape and Version control path pattern shape show the two path patterns' shapes.

Table 6. Path origins pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 ORIGINATED PATH	 PATH LINEAGE	—
<i>e.g.  MASTER PATH</i>		
<i>Field 1</i>		
 PATH CONCEPT	 BRANCH SOURCE	 COMPONENT DATA TYPE
<i>e.g.  DEVELOPMENT PATH</i>		
<i>Field 2</i>		
 PATH ORIGINS	 BRANCH POINT	 INSTANT LITERAL
<i>e.g. Latest</i>		

Table 7. Version control path pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PATH	 SET MEMBERSHIP	—
<i>e.g.  MASTER PATH</i>		

3.5. Edit Coordinate

Where a `StampCoordinate` only ever reads which version is current, an `EditCoordinate` names the STAMP an author's **next** edit will be stamped with — a distinct concern from any of the four dimensions above.  `AUTHOR FOR EDIT COORDINATE` names the author a new edit is attributed to (mentioned already in the Author subsection above);  `DEFAULT MODULE FOR EDIT COORDINATE` names which module a new edit lands in by default, with  `MODULE OPTIONS FOR EDIT COORDINATE` naming the full set an author may choose from instead, and  `PATH OPTIONS FOR EDIT COORDINATE` playing the analogous role for path.

 `PROMOTION PATH FOR EDIT COORDINATE` names a distinct idea again: not where a new edit is committed, but which path a component is **promoted** to once its authoring is complete — the sandbox-to-mainline movement  `SANDBOX PATH` /  `SANDBOX COMPONENT` content eventually undergoes.  `DESTINATION MODULE FOR EDIT COORDINATE` rounds this out for the analogous author-to-export-module handoff.

Put together, a `StampCoordinate` is exactly these four dimensions plus a time: the allowed states, the module preference and exclusion, and a position (path plus time). `StampCalculator` walks a component's version history and returns whichever version is both on an allowed path (at or before the coordinate's time) and in an allowed state — "latest state," computed the same way every time the same coordinate is used.

As a pattern,  `STAMP PATTERN` itself carries a referenced-component meaning of  `VERSION PROPERTIES` and a purpose of  `COMMIT PROVENANCE` — why the pattern exists: to record who committed a version, in what state, module, path, and when. Five fields each carry their own meaning and purpose concept:  `STATUS VALUE` (meaning) for  `STATUS FOR VERSION` (purpose),  `TIME` (meaning) for  `TIME FOR VERSION` (purpose), and  `AUTHOR` ,  `MODULE` , and  `PATH` (meaning) for  `AUTHOR FOR VERSION` ,  `MODULE FOR VERSION` , and  `PATH FOR VERSION` (purpose) respectively — the same four concepts named above, plus time, now restated here by their own exact field-meaning identity. That shape, feature by feature, is STAMP pattern shape.

Table 8. STAMP pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 <code>VERSION PROPERTIES</code>	 <code>COMMIT PROVENANCE</code>	—
<i>Field 1</i>		
 <code>STATUS VALUE</code>	 <code>STATUS FOR VERSION</code>	 <code>COMPONENT DATA TYPE</code>

Meaning	Purpose	Data type
<i>Field 2</i>		
 TIME	 TIME FOR VERSION	 LONG
<i>Field 3</i>		
 AUTHOR	 AUTHOR FOR VERSION	 COMPONENT DATA TYPE
<i>Field 4</i>		
 MODULE	 MODULE FOR VERSION	 COMPONENT DATA TYPE
<i>Field 5</i>		
 PATH	 PATH FOR VERSION	 COMPONENT DATA TYPE

4. Logic Coordinates

A concept's formal, machine-processable meaning lives in its stated and inferred logical axioms — separate from its human-readable descriptions. A `LogicCoordinate` names which patterns and which classifier resolve that meaning, and — like the preference lists on a `LanguageCoordinate` — each of its own fields is itself a concept:  `LOGIC COORDINATE PROPERTIES` groups them.

 `STATED PATTERN FOR LOGIC COORDINATE` names the pattern an author's own stated axioms are recorded under (`P`  `EL++ STATED AXIOMS PATTERN` in this starter set), and

 `INFERRED PATTERN FOR LOGIC COORDINATE` names the pattern the classifier's derived axioms are written to (`P`  `EL++ INFERRED AXIOMS PATTERN`) — stated is what was authored; inferred is what the classifier concluded follows from it, computed, never hand-written.

 `CLASSIFIER FOR LOGIC COORDINATE` names which classifier does that computation ( `SNOCKET CLASSIFIER` here), and  `DESCRIPTION LOGIC PROFILE FOR LOGIC COORDINATE` names the logic profile it classifies against ( `EL++ LOGIC PROFILE` — EL++, described below).

 `ROOT FOR LOGIC COORDINATE` anchors the whole taxonomy the coordinate reasons over.

`P`  `SOLOR CONCEPTS PATTERN` is the membership pattern that marks a concept as belonging to the SOLOR concept space a `LogicCoordinate` reasons over — a referenced-component meaning of  `CONCEPT PATTERN FOR LOGIC COORDINATE` (the same pattern named just above, for the concept dimension rather than the axiom dimension) and purpose of  `SET MEMBERSHIP`, with no fields of its own: a semantic of it is simply an element of its own set, nothing more. This is a fresh, IKE-native pattern, minted deliberately rather than resuming `P`  `SOLOR CONCEPT ASSEMBLAGE` — SOLOR's own dormant original for this exact idea, which carries zero live semantics anywhere in this starter set's ingested foundation, cited here as legacy prior art rather than resumed identity, the same

treatment  **CONCEPT CONSTRAINTS** gets in the Authoring Actions chapter.

 **SOLOR CONCEPTS PATTERN** is the same bare membership-pattern shape this guide's Tinkar Base Model chapter describes for  **TINKAR BASE MODEL COMPONENT PATTERN** and

 **KOMET BASE MODEL COMPONENT PATTERN**.

Solor Concepts Pattern shape and SOLOR concept assemblage shape show the membership patterns named here — the fresh Solor Concepts Pattern and the dormant legacy SOLOR concept assemblage it supersedes.

Table 9. Solor Concepts Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT PATTERN FOR LOGIC COORDINATE	 SET MEMBERSHIP	—

Table 10. SOLOR concept assemblage shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT PATTERN FOR LOGIC COORDINATE	 MEMBERSHIP SEMANTIC	—

5. Navigation Coordinates

Where a **LogicCoordinate** answers "what are this concept's axioms," a **NavigationCoordinate** answers "what are this concept's parents and children" — a faster question to ask repeatedly than re-deriving it from axioms every time.  **IS-A** is the relationship both questions are ultimately about — "designates the parent-child relationship," in its own words, the one every

 **NAVIGATION VERTEX** exists to make fast to walk.  **STATED NAVIGATION PATTERN** and

 **INFERRED NAVIGATION PATTERN** are the two caches navigation can be read from —

 **INFERRED NAVIGATION** the concrete pattern a coordinate actually points at by default — each

 **NAVIGATION VERTEX** recording a concept's direct parents and children, filtered to whichever

 **VERTEX STATE SET** states the coordinate considers current.

Both are **derived**, not authored: a classifier run populates them from the stated (or inferred) axioms, the same way it populates  **EL++ INFERRED AXIOMS PATTERN**. A freshly authored ledger that has never been classified — this starter set's own, today — has stated **axioms** ( **AXIOMS** below) but no stated **navigation** at all yet, which is exactly why this set's own documentation tooling reads taxonomy structure from stated axioms directly rather than from navigation.

P  **STATED NAVIGATION PATTERN** and **P**  **INFERRED NAVIGATION PATTERN** share one identical shape, differing only in which store each populates: both carry a referenced-component meaning of  **IS-A** and a purpose of  **TAXONOMY NAVIGATION CACHE** — a pre-computed parent/child structure so traversal doesn't require re-deriving from axioms every time — and two fields, each already correctly distinguishing meaning from purpose:  **RELATIONSHIP DESTINATION** (meaning) for  **IS-A** (purpose, the child-ward direction) and  **RELATIONSHIP ORIGIN** (meaning) for  **IS-A** again (the parent-ward direction) — the same relationship-direction terminology this guide's Object Properties chapter names as the generic idea these two fields specialize.

Stated Navigation Pattern shape and Inferred Navigation Pattern shape show the two navigation patterns' shapes.

Table 11. Stated Navigation Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 IS-A	 TAXONOMY NAVIGATION CACHE	—
<i>Field 1</i>		
 RELATIONSHIP DESTINATION	 IS-A	 COMPONENT ID SET DATA TYPE
<i>Field 2</i>		
 RELATIONSHIP ORIGIN	 IS-A	 COMPONENT ID SET DATA TYPE

Table 12. Inferred Navigation Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 IS-A	 TAXONOMY NAVIGATION CACHE	—
<i>Field 1</i>		
 RELATIONSHIP DESTINATION	 IS-A	 COMPONENT ID SET DATA TYPE
<i>Field 2</i>		
 RELATIONSHIP ORIGIN	 IS-A	 COMPONENT ID SET DATA TYPE

6. EL++ Concepts

 **AXIOMS** groups every concept in this set's meta-schema that describes how a concept's formal meaning is expressed:  **AXIOM SYNTAX** (the notation an axiom is written in) and  **AXIOM ORIGIN** (whether an axiom was authored or derived) sit alongside  **EL++ TERMINOLOGICAL AXIOMS** — this set's actual logic profile — which splits into  **EL++ STATED TERMINOLOGICAL AXIOMS** (what was authored) and  **EL++ INFERRED TERMINOLOGICAL AXIOMS** (what the classifier concluded). A separate family,  **INTERVAL SET AXIOMS**, covers interval- and range-valued axioms rather than terminological ones.

EL is the description-logic profile SNOMED CT and Tinkar both build on: expressive enough for real terminological definitions (necessary conditions, role restrictions, concrete-value constraints), restricted enough that classification stays tractable at hundreds of thousands of concepts. This starter set can describe its own logic because the identity-exact ingest (IKE-Network/ike-issues#872) brought in Tinkar's entire EL terminology — the classifier, the axiom patterns, the profile concept itself — as part of the same "fork and own" ingest that gave every other concept in this guide its identity.

6.1. Roles and Role Operators

A role is EL++'s mechanism for a relationship between a defined concept and another concept — "has active ingredient," "has finding site," and so on.  **ROLE GROUP** is the pattern's top-level grouping;  **ROLE** and  **ROLE OPERATOR** both sit beneath it as siblings, not parent and child — a role names **what** relationship holds ( **ROLE TYPE** identifies which relationship kind), while a role operator names **how** that relationship is quantified over its target:  **EXISTENTIAL RESTRICTION** ("at least one"),  **UNIVERSAL RESTRICTION** ("all of"), and two SOLOR-specific refinements —  **REFERENCED COMPONENT SUBTYPE RESTRICTION** and  **REFERENCED COMPONENT TYPE RESTRICTION** — that further narrow which component kinds a dynamic field's value may legally reference.

 **ROLE GROUP** itself groups several attribute-or-role-value pairs so they are considered together within one concept definition rather than combined freely across the whole definition — the same distinction post-coordinated expressions in SNOMED CT rely on to keep, for example, two different findings' own laterality from cross-combining.

6.2. Inclusion, Necessary, and Sufficient Sets

Beneath  **EL++ STATED TERMINOLOGICAL AXIOMS** and  **EL++ INFERRED TERMINOLOGICAL AXIOMS** sit three kinds of role-set, each answering a different question about a concept's definition.

 **NECESSARY SET** holds relationships that are always true of the concept — necessary conditions, in description-logic terms — while  **SUFFICIENT SET** holds relationships that, together, are enough to fully differentiate the concept and its subtypes from every other concept: a concept with at least one sufficient set is **defined**, not merely **primitive**.  **INCLUSION SET** is weaker than either — relationships that are part of the concept's definition without being individually necessary, or

together sufficient.  **NECESSARY BUT NOT SUFFICIENT CONCEPT DEFINITION** names the definition-status value a concept carries when its own role sets fall short of full sufficiency — the everyday case for most authored content, since fully-differentiating, sufficient definitions are the exception in any real terminology, not the rule.

6.3. Concrete Value Operators

Not every constraint in an EL++ definition names another concept — some constrain a **literal** value instead: a lab reference range's upper bound, a dose's numeric strength.

 **CONCRETE VALUE OPERATOR** is the parent of the comparison operators that express these concrete-domain constraints:  **EQUAL TO**,  **GREATER THAN**,  **GREATER THAN OR EQUAL TO**,  **LESS THAN**,  **LESS THAN OR EQUAL TO**, and the two range endpoints  **MINIMUM VALUE OPERATOR** and  **MAXIMUM VALUE OPERATOR**. These are exactly the operator concepts  **VALUE CONSTRAINT PATTERN** draws on for its own min/max reference-range fields (see the Semantic Field Model chapter) — concrete-domain reasoning is what lets EL++ classification stay tractable even when a definition depends on a numeric threshold, rather than requiring a distinct concept for every possible value.

6.4. Logical Definitions and Grouping

 **LOGICAL DEFINITION** names the semantic value a stated or inferred terminological axiom's own definition status carries —  **EL++ STATED CONCEPT DEFINITION** for what was authored,  **EL++ INFERRED CONCEPT DEFINITION** for what the classifier concluded, mirroring the same stated-versus-inferred split  **AXIOMS** itself organizes around.  **STATED DEFINITION** and  **INFERRED DEFINITION** name the underlying relationship sets those definition-status values describe — "relationships/axioms of a concept that have been explicitly stated" versus "inferred," the same distinction spelled out at the level of individual relationships rather than a definition's overall status.

Two smaller, related concept families round out this set:  **GROUPING** distinguishes an  **EXACT** match (source and target that are semantically or lexically identical) from a  **PARTIAL** one, and  **DIRECTED GRAPH** is the parent of  **K EL Digraph** — the directed graph that results from classifying a full set of EL++ axioms, the data shape  **SNOCKET CLASSIFIER** itself produces and  **INFERRED NAVIGATION** is ultimately derived from.

The stated/inferred split has its own pair of patterns beneath it:  **EL++ STATED AXIOMS PATTERN** carries a referenced-component meaning of  **STATED DEFINITION** and purpose of  **LOGICAL DEFINITION**, with one field,  **EL++ STATED TERMINOLOGICAL AXIOMS** (meaning) for  **LOGICAL DEFINITION** (purpose);  **EL++ INFERRED AXIOMS PATTERN** is its inferred-side twin — meaning  **INFERRED DEFINITION**, purpose  **LOGICAL DEFINITION** again, its field  **EL++ INFERRED TERMINOLOGICAL AXIOMS** for  **LOGICAL DEFINITION** likewise — the two concrete patterns  **STATED PATTERN FOR LOGIC COORDINATE** and  **INFERRED PATTERN FOR LOGIC COORDINATE** (named in the Logic Coordinates chapter) actually

point at. **P**  **OWL AXIOM SYNTAX PATTERN** is smaller still: a referenced-component meaning of  **AXIOMATIZED COMPONENT** — the component whose definitional axioms the semantic expresses — and a purpose of  **AXIOM EXPRESSION**, with one field,  **AXIOM SYNTAX** for  **EXPRESS AXIOM SYNTAX**, holding the OWL functional-syntax text an axiom can optionally also be expressed in.

EL++ Stated Axioms Pattern shape, EL++ Inferred Axioms Pattern shape, and OWL Axiom Syntax Pattern shape show the three axiom patterns' shapes.

Table 13. EL++ Stated Axioms Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STATED DEFINITION	 LOGICAL DEFINITION	—
<i>e.g.</i>  UNINITIALIZED COMPONENT		
<i>Field 1</i>		
 EL++ STATED TERMINOLOGICAL AXIOMS	 LOGICAL DEFINITION	 DiTREE DATA TYPE
<i>e.g.</i> 4-vertex tree		

Table 14. EL++ Inferred Axioms Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 INFERRED DEFINITION	 LOGICAL DEFINITION	—
<i>Field 1</i>		
 EL++ INFERRED TERMINOLOGICAL AXIOMS	 LOGICAL DEFINITION	 DiTREE DATA TYPE

Table 15. OWL Axiom Syntax Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		

Meaning	Purpose	Data type
 AXIOMATIZED COMPONENT	 AXIOM EXPRESSION	—
<i>Field 1</i>		
 AXIOM SYNTAX	 EXPRESS AXIOM SYNTAX	 STRING DATA TYPE

7. Semantic Field Model

This chapter describes the field-level meta-schema: what kind of value a pattern's field holds, how that value renders in Komet's UI, what its value can mean, and how a field's legal values can be constrained.

7.1. Field Categories

 **FIELD CATEGORIES** groups the field-level meta-schema concepts: the terminology describing what **kind** of value a pattern's field holds, distinct from that field's own particular meaning and purpose in any one pattern declaration.  **COMPONENT FIELD** is the parent of every field kind whose value is a reference to another component;  **CONCEPT FIELD** narrows that to concepts specifically, and is itself the parent of the four STAMP-dimension field kinds this guide's STAMP chapter already named —  **AUTHOR FIELD**,  **MODULE FIELD**,  **PATH FIELD**,  **STATUS FIELD** — plus  **FIELD DEFINITION MEANING FIELD** and  **FIELD DEFINITION PURPOSE FIELD**, the field kinds a pattern's own field definitions use to record each field's meaning and purpose concepts: the same terminology every pattern declaration in this starter set's own source populates when it names a field's meaning, purpose, and data type.  **PATTERN FIELD**,  **SEMANTIC FIELD**, and  **STAMP FIELD** are the parallel component-field kinds for referencing a pattern, a semantic, or a STAMP directly, each further specialized ( **SEMANTIC PATTERN FIELD** for the pattern a semantic is of;  **SEMANTIC REFERENCED COMPONENT FIELD** for what it is attached to).

Sibling to  **COMPONENT FIELD** under  **FIELD CATEGORIES** itself sit the **collection** field kinds —  **COMPONENT VERSIONS FIELD** /  **COMPONENT VERSIONS SET** and their four specializations for concept, pattern, semantic, and STAMP versions — the field kinds that hold every version of a component rather than a single reference, and  **STRING FIELD** (parent of  **PUBLIC ID FIELD**) for text-valued fields.  **FIELD DEFINITION FIELD** and  **FIELD DEFINITIONS SET** name one field definition and the whole set of a pattern's field definitions, respectively, while  **FIELD VALUE FIELD** names the field kind for an actual field **value** inside a semantic instance — the same definition-versus-instance distinction that separates a pattern's own field declarations from the values a semantic of it actually carries.

7.2. Display Fields

 **DISPLAY FIELDS** is a pattern's own field's declared data type — confirmed by identity against `ConceptToDataType` (`dev.ikm.tinkar.terms`, `tinkar-core`), which resolves exactly this family's members to their real `FieldDataType`, and exactly what a pattern's own `.field(meaning, purpose, dataType)` declaration draws its third argument from throughout this starter set's own source. Several members carried misleading "display field" wording, as if a separate "how does this render" concept sat behind some other, truer storage type — it does not, and every member confirmed in use through `ConceptToDataType` has been renamed to say so plainly:

 **CONCEPT DATA TYPE** holds a concept's identity,  **COMPONENT DATA TYPE** the analogous case for any component (concept, semantic, pattern, or STAMP, not restricted to concepts);  **COMPONENT ID LIST DATA TYPE** and  **COMPONENT ID SET DATA TYPE** hold an ordered or unordered collection of component references.  **STRING DATA TYPE** and  **FLOAT DATA TYPE** hold a sequence of characters and a fractional number, respectively;  **BOOLEAN DATA TYPE**,  **INTEGER DATA TYPE**,  **DECIMAL DATA TYPE**,  **BYTE ARRAY DATA TYPE**, and  **ARRAY DATA TYPE** round out the renamed primitive and collection kinds, and  **SEMANTIC DATA TYPE** holds a reference to a semantic. Only the members `ConceptToDataType` does not recognize keep their display-era names until each is checked the same way:  **DOUBLE DISPLAY FIELD**,  **IMAGE DISPLAY FIELD**,  **UUID DISPLAY FIELD**, and  **LOGICAL EXPRESSION DISPLAY FIELD** (a rendered axiom expression).

Two structural kinds hold graphs directly:  **DIGRAPH DATA TYPE** (an ordered-pair directed graph — the same shape `K EL Digraph` takes) and  **DITREE DATA TYPE** (a directed tree obtained from an undirected one, one designated root), both likewise renamed.

7.3. Field Value Terminology

 **MEANING** is the broadest of these three groups: the terminology a pattern or semantic's field draws its **interpretation** from, distinct from the field **category** ( **FIELD CATEGORIES**) or declared **data type** ( **DISPLAY FIELDS**) concept families above.  **DYNAMIC COLUMN DATA TYPES** —  **BOOLEAN**,  **DECIMAL**,  **FLOAT**,  **DOUBLE**,  **LONG**,  **SIGNED INTEGER**,  **ARRAY**,  **BYTE ARRAY**, and  **UUID DATA TYPE** — looks like it should be the same family  **DISPLAY FIELDS** draws from, but it is a separate, disjoint one: Komet's own pattern-authoring UI (`PatternFieldsController`, `kview`) draws its "choose a data type" list from this branch, while a pattern's real field declaration draws its `dataType` tag from  **DISPLAY FIELDS** instead (`IkeTerm.LONG`, `IkeTerm.STRING`, `IkeTerm.COMPONENT_FIELD`, and so on, throughout this starter set's own source) — two families that happen to describe the same idea twice, not one. Reparented under  **LEGACY MODEL CONCEPTS** here as a deprecation signal; the disconnect on the komet side is tracked as `ikmdev/komet#880`.

 **LITERAL VALUE** groups the value shapes a concrete field literal can take —  **BOOLEAN LITERAL** (`TRUE/FALSE/UNKNOWN`),  **FLOAT LITERAL**,  **INSTANT LITERAL** — while  **FIELD SUBSTITUTION** groups the analogous notion for a query template:  **BOOLEAN SUBSTITUTION**,

 CONCEPT SUBSTITUTION ,  FLOAT SUBSTITUTION ,  INSTANT SUBSTITUTION each name what kind of value fills a placeholder when a stored query actually runs.  CONNECTIVE OPERATOR ( AND ,  OR ,  PART OF ,  DISJOINT WITH) and  TAXONOMY OPERATOR ( LOGICALLY EQUIVALENT TO) round out the operator concepts a field's meaning can name, and  SEMANTIC TYPE ( CONCEPT SEMANTIC ,  COMPONENT SEMANTIC ,  MEMBERSHIP SEMANTIC ,  LOGICAL EXPRESSION SEMANTIC) names what kind of thing a whole semantic — not just one field — represents.

7.4. Constraining Fields

Some of the concept-typed field kinds above —  AUTHOR FIELD ,  MODULE FIELD ,  STATUS FIELD , and others — are conventionally restricted to a known, bounded set of legal values: only concepts kind-of  AUTHOR make sense as a STAMP's author field, only the immediate children of  STATUS VALUE as its status. Nothing in the field-definition model above expresses that restriction — a field's declared data type says it holds **a** concept, never **which** concepts, and the same holds for every other data type.

Two constraint patterns close that gap, one per parameter shape — the pattern axis is the parameter shape, never the datatype, and never a union carried by sentinel values. Both attach to the pattern being constrained, and both share a referenced-component meaning of  CONSTRAINED PATTERN — the attachment target's role: the pattern that has one of its own fields constrained, identical for both shapes — and a purpose of  FIELD VALUE RESTRICTION . A pattern's constraint semantics compose conjunctively: each semantic is one conjunct, and a value is legal only when every conjunct on its field holds.

P  TAXONOMY FIELD CONSTRAINT PATTERN is the intensional shape: the legal set is derived from the taxonomy under the checking view. Its three fields each carry their own meaning and purpose concept:  CONSTRAINED FIELD (meaning) for  CONSTRAINT SCOPE (purpose — which field this rule governs);  CONSTRAINT KIND for  CONSTRAINT RULE (which rule applies); and  CONSTRAINT ANCHOR CONCEPT for  TAXONOMY REFERENCE POINT (the concept the rule measures against).  CONSTRAINT KIND 's legal values are  TAXONOMY FIELD CONSTRAINT KIND 's four children —  KIND-OF FIELD CONSTRAINT (the anchor plus every descendant, self included),  DESCENDANT FIELD CONSTRAINT (descendants only),  LEAF DESCENDANT FIELD CONSTRAINT (leaf descendants only — excluding grouping concepts that exist solely to organize others), and  IMMEDIATE CHILD FIELD CONSTRAINT (direct children only) — an identical tuple for all four, which is exactly why they are kinds within one pattern rather than four patterns. Concretely, this starter set's own STAMP pattern carries four such semantics: constrained field =  AUTHOR , kind =  KIND-OF FIELD CONSTRAINT , anchor =  AUTHOR — the same concept identifies both the field and the anchor here, since a later revision (IKE-Network/ike-issues#880) gave the Author field itself  AUTHOR as its own meaning — likewise Module and Path, and a fourth whose tuple reads constrained field =  STATUS VALUE , kind =  IMMEDIATE CHILD FIELD CONSTRAINT , anchor =  STATUS VALUE again: a concept can serve as both a field's own meaning and a constraint's own anchor without conflict, since the two roles are read from different fields entirely.

P **VALUE-SET FIELD CONSTRAINT PATTERN** is the extensional shape, and it is datatype-universal: the members are enumerated as data, and the member type is whatever the source pattern's named Model Feature declares, so this one shape yields closed enumerations of any field datatype. Its four fields: **CONSTRAINED FIELD** for **CONSTRAINT SCOPE** again; **VALUE-SET PATTERN** (meaning) for **LEGAL VALUE SOURCE** (purpose — the pattern whose active semantics enumerate the legal values); **VALUE-SET FIELD** for **VALUE DISAMBIGUATION** — naming the Model Feature that holds the member value by its meaning concept, either a declared field's meaning or the source pattern's referenced-component meaning (membership patterns supported, no sentinel); and

MEMBER MATCH RELATION for **MATCH RULE** (purpose — how a value must match a member). The constraint is satisfied when some member active under the checking view matches the value under the named relation — membership itself carries the ordinary STAMP lifecycle, so adding a member is authoring and removing one is retiring. This starter set's own illustrative

P **STARTER SET AUTHOR ROSTER PATTERN** shows why the Model-Feature pointer is needed: each roster entry is a semantic with **two** fields, **ROSTER AUTHOR** and **ROSTER ORDER**, and only the first is the value a member is matched against. The worked constraint on

P **PREFERRED REVIEWER PATTERN** reads: constrained field = **PREFERRED REVIEWER**, value-set pattern = **P** **STARTER SET AUTHOR ROSTER PATTERN**, value-set field = **ROSTER AUTHOR**, relation = **EQUAL MATCH RELATION**.

How a value must match an enumerated member is itself a concept, never invented syntax:

MEMBER MATCH RELATION is a closed taxonomy whose relations are directed and named in full. Evaluation dispatch stays code-sovereign — each relation's evaluator declares its relation concept's identity in code, so code points at knowledge and knowledge never carries an operative pointer at code: imported content can never redirect what constraint checking executes. Admission is a bijection, enforced by test: relation concepts correspond one-to-one with the shipped evaluators, so minting a relation without shipping its evaluator fails, and shipping an evaluator without minting its relation fails. **EQUAL MATCH RELATION** is the sole admitted relation today: equality by the member datatype's natural equality — entity references by identity, scalars by value, byte arrays by content, Decimal by numeric equality, and Float by IEEE comparison, so NaN matches nothing, keeping the loud-defaults coherence (an unrevised NaN default fails its constraint until revised); graph types await the isomorphic relations.

The apparatus constrains itself, as ordinary content: a **P** **TAXONOMY FIELD CONSTRAINT PATTERN** semantic on each constraint pattern holds its own discriminator field to its closed taxonomy —

CONSTRAINT KIND immediate-child of **TAXONOMY FIELD CONSTRAINT KIND**, and the

MEMBER MATCH RELATION field immediate-child of **MEMBER MATCH RELATION**.

Why mint fresh identity, rather than adopting **CONCEPT CONSTRAINTS** — a real, if long-dormant, SOLOR concept whose own definition, "defined filters for a given concept," reads almost as if it were written for this exact mechanism? Because it carries no live semantics anywhere in this ecosystem to confirm what "defined filters" actually meant — no field shape, no worked example, no consuming code — and adopting an identity on the strength of a plausible-sounding one-line gloss

risks importing an unintended meaning later, if that dormant concept's real intent turns out to differ from what is built here.  **CONCEPT CONSTRAINTS** remains exactly where it was — cited here as legacy prior art rather than resumed identity, a distinction worth drawing plainly, since the rest of this starter set leans hard the other way (adopting existing identity, as with  **AUTHOR FOR VERSION** and every other SOLOR-sourced concept named in this guide) whenever a real, checkable definition backs the reuse rather than merely a plausible-sounding name.

No pattern-model change was needed to represent any of this: every taxonomy constraint kind composes directly from graph-walk operations a navigation calculator already provides —  **KIND-OF FIELD CONSTRAINT** from `kindOf`,  **DESCENDANT FIELD CONSTRAINT** from `descendentsOf`,  **IMMEDIATE CHILD FIELD CONSTRAINT** from `childrenOf`, and  **LEAF DESCENDANT FIELD CONSTRAINT** by filtering `descendentsOf` down to the nodes `childrenOf` finds empty. Reading and **enforcing** these constraints — in Komet's own field editors, for instance, which today have no such notion at all — is a deliberately separate, later concern from representing them; what the representation already makes possible today is exactly the candidate set a reader would want to see, computed fresh every render, with no separate authoring step — the live listing just below is every concept a  **KIND-OF FIELD CONSTRAINT** anchored at  **AUTHOR** would accept, walked from the same  **AUTHOR** broader relationship the constraint's own definition points at.

 **AUTHOR**

 **SNO Rocket CLASSIFIER**

 **IKE COMMUNITY**

 **TINKAR STARTER DATA AUTHOR**

 **GRETEL**

 **KOMET USER**

This section is the shape-level introduction. The Field Constraints chapter, next, describes how the starter set supports constraints in practice — authoring and retrieval, conjunctive composition, each shape's legal-set derivation, the member match relations, and coherence with default values — and renders the two constraint patterns' shapes as Taxonomy Field Constraint Pattern shape and Value-set Field Constraint Pattern shape.

8. Field Constraints

Where the Semantic Field Model chapter's Constraining Fields section introduces the constraint apparatus — two patterns, one per parameter shape, both attaching to the pattern whose field they constrain — this chapter describes how the starter set supports constraints in practice: what authoring one looks like, how several constraints on one field compose, how each shape derives its legal set, and how constraints and default values cohere.

A constraint is first-class content of the pattern it constrains. Unlike a default value — which needs the Defaults and templates module as its category boundary (see the Default Values chapter) — a constraint needs no machinery at all: no special module, no exclusion from any read path, no dedicated accessor. Each constraint semantic attaches to the constrained pattern as its referenced component, and reading a pattern's constraints is retrieval by enumeration: walk the semantics attached to the pattern that are of the two constraint patterns. Identity is declared descriptively, so the ledger reads as prose — the STAMP author constraint below is declared as "Taxonomy Field Constraint: STAMP pattern Author field kind-of Author", the rule's own name saying exactly what it asserts.

Constraint semantics compose conjunctively. Each semantic is one conjunct, and a value is legal only when every conjunct on its field holds — no override, no precedence, no disjunction to reason about. A field may carry constraints of both shapes at once, a taxonomy constraint and a value-set constraint intersecting: legal then means inside the taxonomy region **and** matching an enumerated member.

The pattern axis is the parameter shape: one constraint pattern per parameter tuple, never one per datatype, and never a union carried by sentinel values. A kind or relation field earns its place as a genuine parameter only when every field of the tuple stays meaningful for every one of its values — the four taxonomy kinds share one identical (field, kind, anchor) tuple, so one pattern carries all four; value-set membership carries a different tuple entirely, so it is a pattern of its own — a shape, never a fifth kind. Every field of every constraint semantic is meaningful: no  BLANK CONCEPT sentinel anywhere in this story.

8.1. The Taxonomy Field Constraint

  **TAXONOMY FIELD CONSTRAINT PATTERN** is the intensional constraint shape: a semantic of it stores a rule, never a list. The legal set for the  **CONSTRAINED FIELD** is derived from the knowledge graph under the checking view's navigation coordinate — walked from the  **CONSTRAINT ANCHOR CONCEPT** by whichever of  **TAXONOMY FIELD CONSTRAINT KIND**'s four children the  **CONSTRAINT KIND** field names. The four kinds are four editorial intents about a taxonomy region:  **KIND-OF FIELD CONSTRAINT** says the anchor is itself a legal value, alongside everything beneath it;  **DESCENDANT FIELD CONSTRAINT** says the anchor is a header — legal values sit strictly beneath it;  **LEAF DESCENDANT FIELD CONSTRAINT** says only the concrete leaves qualify, never the grouping concepts that exist to organize them; and  **IMMEDIATE CHILD FIELD CONSTRAINT** says the taxonomy is a closed one-level value list — exactly how the STAMP pattern's status field holds to the immediate children of  **STATUS VALUE**.

Because the legal set is derived rather than copied, governing it moves to whoever governs that taxonomy region: minting a new kind of  **AUTHOR** extends what every author-constrained field accepts, with no constraint edit anywhere. And because the derivation runs under a view, the legal set is view-relative — two checking views can answer differently — so checking is an authoring-time and QA-time calculator operation, never a property a stored value carries with it.

Taxonomy Field Constraint Pattern shape shows the pattern's shape.

Table 16. Taxonomy Field Constraint Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONSTRAINED PATTERN	 FIELD VALUE RESTRICTION	—
e.g.   DESCRIPTION PATTERN		
Field 1		
 CONSTRAINED FIELD	 CONSTRAINT SCOPE	 COMPONENT DATA TYPE
e.g. <←2147479135>		
Field 2		
 CONSTRAINT KIND	 CONSTRAINT RULE	 COMPONENT DATA TYPE
e.g.  KIND-OF FIELD CONSTRAINT		
Field 3		
 CONSTRAINT ANCHOR CONCEPT	 TAXONOMY REFERENCE POINT	 COMPONENT DATA TYPE
e.g.  LANGUAGE		

The starter set carries five taxonomy constraints on real, long-standing patterns — the four STAMP dimensions and the Description Pattern's language field (STAMP pattern shape and Description Pattern shape show the constrained patterns' own shapes):

-   STAMP PATTERN 's author, module, and path fields are each kind-of constrained to their own dimension's taxonomy — anchors  AUTHOR ,  MODULE , and  PATH — the anchor itself a legal value, and every new author, module, or path minted beneath it a legal value from the moment it exists.
-   STAMP PATTERN 's status field is immediate-child constrained to  STATUS VALUE : STAMP status is a closed, one-level value list — exactly  ACTIVE STATE ,  INACTIVE STATE ,  PRIMORDIAL STATE ,  CANCELED STATE , and  WITHDRAWN STATE — and the constraint says so as data.

- **P**  **DESCRIPTION PATTERN** 's language field (meaning **K**  **Language Concept Nid For Description**) is kind-of constrained to **L**  **LANGUAGE** : any language concept, at any depth, is a legal description language.

8.2. The Value-set Field Constraint

P  **VALUE-SET FIELD CONSTRAINT PATTERN** is the extensional constraint shape: the legal values are enumerated as data — the active semantics of the named **V**  **VALUE-SET PATTERN** — and the constraint is satisfied when some member active under the checking view matches the constrained field's value under the named **M**  **MEMBER MATCH RELATION**. The shape is datatype-universal: the member type rides the source pattern's own declared Model Feature, the one the **L**  **VALUE-SET FIELD** pointer names by its meaning concept, so this one four-field shape yields closed enumerations of any field datatype — allowed Strings, allowed Integers, allowed concepts — never a pattern per datatype.

Membership carries the ordinary STAMP lifecycle, member by member: adding a member is authoring a semantic of the value-set pattern, removing one is retiring it, and every membership change carries its own provenance — who, when, in which module, on which path. What the value set contains is answered through the stamp calculus — active-under-the-checking-view membership — so the legal set evolves version by version, with no edit to the constraint semantic itself.

Value-set Field Constraint Pattern shape shows the pattern's shape.

Table 17. Value-set Field Constraint Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONSTRAINED PATTERN	 FIELD VALUE RESTRICTION	—
<i>e.g. P  PREFERRED REVIEWER PATTERN</i>		
<i>Field 1</i>		
 CONSTRAINED FIELD	 CONSTRAINT SCOPE	 COMPONENT DATA TYPE
<i>e.g. V  PREFERRED REVIEWER</i>		
<i>Field 2</i>		
 VALUE-SET PATTERN	 LEGAL VALUE SOURCE	 COMPONENT DATA TYPE
<i>e.g. P  STARTER SET AUTHOR ROSTER PATTERN</i>		

Meaning	Purpose	Data type
<i>Field 3</i>		
 VALUE-SET FIELD	 VALUE DISAMBIGUATION	 COMPONENT DATA TYPE
<i>e.g.</i>  ROSTER AUTHOR		
<i>Field 4</i>		
 MEMBER MATCH RELATION	 MATCH RULE	 COMPONENT DATA TYPE
<i>e.g.</i>  EQUAL MATCH RELATION		

The set's worked value-set constraint is the one the Default Values chapter's worked default already satisfies:   **PREFERRED REVIEWER PATTERN** 's single field,  **PREFERRED REVIEWER**, is constrained to membership in the starter set author roster. The constraint's four fields read: constrained field =  **PREFERRED REVIEWER**, value-set pattern =  **STARTER SET AUTHOR ROSTER PATTERN**, value-set field =  **ROSTER AUTHOR**, member match relation =  **EQUAL MATCH RELATION** (Preferred Reviewer Pattern shape and Starter Set Author Roster Pattern shape show both patterns' shapes).

The roster is why the  **VALUE-SET FIELD** pointer exists. Each roster entry is a semantic with two fields —  **ROSTER AUTHOR**, the member, and  **ROSTER ORDER**, a display order — and only the first is what a candidate value is matched against:  **ROSTER ORDER** is a characteristic of the entry, not the member. The pointer names the member-bearing Model Feature by its meaning concept — a declared field's meaning, or, for a membership pattern whose members are the referenced components themselves, the source pattern's referenced-component meaning — so any pattern whose active semantics enumerate things can serve as a value set, with no value-set-specific machinery on the source side.

8.3. The Member Match Relation

 **MEMBER MATCH RELATION** is the closed taxonomy of relations a   **VALUE-SET FIELD CONSTRAINT PATTERN** semantic can require between a constrained field's value and an enumerated member — and it is also that pattern's fourth field's own meaning: the field's value is a relation from this taxonomy. How a value must match a member is a concept, never invented syntax, and relations are directed and named in full — a value containing a member fragment and a value contained in a member would be different relations, each with its own name.

Admission is evaluator-backed. Dispatch stays code-sovereign through a typed

`MemberMatchEvaluator` service seam: each evaluator declares in code which relation concept it evaluates — code points at knowledge, never knowledge at operative code pointers, so imported content can never redirect what constraint checking executes — and each evaluator states its own operand type discipline, since what "the types must agree" means is itself relation-relative. The

admission gate is a bijection, enforced by test: the relations — this concept's children under the checking view — correspond one-to-one with the service-loaded evaluators, so minting a relation without shipping its evaluator fails, and shipping an evaluator without minting its relation fails.

One relation is admitted today:

 **EQUAL MATCH RELATION** — Equal, the sole relation admitted today — matches a value to a member when they are equal by the member datatype's natural equality, with the rulings made deliberately, datatype by datatype: entity references match by identity, whatever facade form carries them; String, Boolean, Integer, Long, and Instant match by value; byte arrays by content; arrays elementwise; id lists by order-sensitive element identity and id sets by membership. Decimal matches by numeric equality — `compareTo == 0`, so 2.5 admits 2.50 — an identical-representation rule would be its own relation, never an ambiguity inside this one. Float compares by IEEE semantics, so NaN matches nothing, itself included — keeping the loud-defaults coherence: an unrevised NaN default fails its constraint until someone chooses a real value.

Graph types sit outside Equal's operand conformance: graph enumerations await the isomorphic relations, which is where the canonical-form equality question belongs — deferred, never silently decided here.

8.4. The Apparatus Constrains Itself

The apparatus's own discriminator fields are exactly the closed one-level value lists the immediate-child kind exists for, and the set constrains them accordingly — as ordinary content, through the same pattern any other constraint uses:

- "Taxonomy Field Constraint: Taxonomy Field Constraint Pattern Constraint kind field immediate-child of Taxonomy field constraint kind" — constrained field =  **CONSTRAINT KIND**, kind =  **IMMEDIATE CHILD FIELD CONSTRAINT**, anchor =  **TAXONOMY FIELD CONSTRAINT KIND**: the kind field of every taxonomy constraint holds one of the four kinds, and nothing else.
- "Taxonomy Field Constraint: Value-set Field Constraint Pattern Member match relation field immediate-child of Member match relation" — constrained field =  **MEMBER MATCH RELATION**, kind =  **IMMEDIATE CHILD FIELD CONSTRAINT**, anchor =  **MEMBER MATCH RELATION** again: the relation field of every value-set constraint holds an admitted relation, and nothing else — the same concept serving as the field's meaning and the constraint's anchor without conflict, the two roles read from different fields entirely.

Nothing marks these two semantics as special: they are taxonomy constraints like the STAMP pattern's four, retrieved the same way, composed the same way. Growing either closed list is authoring: mint the child concept, and the existing constraint admits it without being touched — for a relation, the bijection gate additionally requires its evaluator to ship in the same change.

8.5. Constraints and Defaults

Constraints and defaults meet at one coherence rule: a default value must satisfy its field's constraints — a fresh semantic pre-populated from defaults should be legal where a curator has finished choosing, and loudly illegal where they have not.

P **PREFERRED REVIEWER PATTERN** shows the finished side: its authored default, **G** **GRETEL**, is the roster's own first entry, so the default satisfies the value-set constraint on the very field it fills — a real, constraint-satisfying value, visible as the *default* row of Preferred Reviewer Pattern shape (the Default Values chapter walks the full round trip).

The loud defaults show the unfinished side, deliberately. **U** **UNINITIALIZED COMPONENT** belongs to no domain taxonomy region and no curated value set, and **NaN** matches nothing under **E** **EQUAL MATCH RELATION**, itself included. So wherever a constrained field still carries its loud default, constraint checking fails it until someone chooses a real value — the same machinery that validates finished data announces unfinished data, the loud-defaults principle of the Default Values chapter restated from the checking side.

9. Default Values

An editor opening a fresh semantic needs a starting value for every field — the store rejects null field values outright. This chapter describes how the starter set supports default values with no defaults-specific machinery in the model at all: a default is ordinary working-path content — a complete semantic of the target pattern — kept out of domain reads by two structural dimensions the model already has, its referenced component and its module.

9.1. The Default Value Semantic

D **DEFAULT VALUE CONCEPT** is the attachment point for default values: the default value semantic of a pattern is a complete semantic of that pattern — every field populated, in declared order — whose referenced component is this concept, and whose fields carry the values an editor offers as starting values for that pattern's fields. Its identity is computed, never queried: the stock `singleSemanticUuid` of the pattern's public id and this concept's public id. A consumer resolves "the default for this pattern" by computing that UUID — `StampCalculator.getDefault(pattern)` does exactly this — so no registry is needed, regeneration merges idempotently onto the same chronology, and a second default for the same pattern cannot be minted without an identity collision: the derivation doubles as the uniqueness key.

Attachment here is a support declaration only, never a domain assertion about this concept or any other component, and this concept never joins a domain member set — consumers scope affirmatively, so a support declaration can never read as a domain assertion. Per-user preferences are per-path versions of this same computed-identity chronology, and migration between sandboxes is an identity-keyed copy-forward.

9.2. The Defaults and Templates Module

 **DEFAULTS AND TEMPLATES MODULE** — preferred regular name simply "Defaults" — is the one module every default value and template semantic stamps its versions in: the category boundary, and the import/export packaging. The boundary is bidirectional and live-and-die: every version of a defaults or template chronology lives in this module, and this module holds only defaults and template content — a violation in either direction is an error to be withdrawn, never adopted as precedent. That invariant is what makes exclusion trivial: `StampCalculator`'s version-iteration operations exclude versions stamped in this module, so defaults and templates never surface as ordinary pattern or component content — while chronicle-level surfaces are unchanged, and chronicle iterators still see defaults and template chronologies; it is the computation of iteration over versions that triggers the exclusion. The dedicated accessors — `getDefault(pattern)` and `getTemplate(pattern, purpose)` — include the module.

As the export dimension, module crossed with path selects a user's preferences for exchange: include this module to carry defaults and templates alongside domain content, exclude it to leave them behind.

9.3. Templates

 **TEMPLATE CONCEPT** is the parent of the per-purpose template attachment concepts: each child is minted for one template purpose, and a template semantic of a pattern for that purpose is a complete semantic of that pattern whose referenced component is that child — starting content offered when authoring new components, where a default value semantic offers starting values for one pattern's fields. Identity is computed the same way — `singleSemanticUuid` of the pattern's public id and the purpose concept's public id — and retrieval takes both keys, `StampCalculator.getTemplate(pattern, purpose)`, because one purpose concept may host templates of more than one pattern: the pair, not the purpose alone, identifies a template. There is one template per (pattern, purpose) pair, resolved as a whole tuple — templates carry no composition semantics.

As with a default value semantic, attachment to a template purpose concept is a support declaration, never a domain assertion, and no child of this concept joins a domain member set.

9.4. Loud Defaults for Every Data Type

What value should a default actually carry? The governing principle is **loud defaults**: a default must work as an initial value while reading as obviously needing revision — never Java's silent zero-values, which disappear into real data instead of announcing themselves. The starter set demonstrates the principle at full breadth with one pattern carrying a loud default for every data type a pattern field can declare.

P  **DATA TYPE DEFAULTS PATTERN** is the defaults apparatus applied at full breadth: one pattern with sixteen fields, one per data type the platform's field-type resolver (`ConceptToDataType` , tinkar-core) actually recognizes, declared in that resolver's own order — String, Component, ComponentIdSet, ComponentIdList, DiTree, DiGraph, Concept, Semantic, Integer, Float, Boolean, ByteArray, Array, Instant, Long, Decimal. All sixteen fields share one purpose,

 **DATA TYPE DEFAULT EXEMPLAR** : every field exists for the identical reason — to carry its data type's loud default — so distinct per-field purposes would be manufactured variety.

The pattern's single semantic is its own default value semantic: referenced component

 **DEFAULT VALUE CONCEPT** , computed identity, every version in

 **DEFAULTS AND TEMPLATES MODULE** . Each field's value is its data type's loud default — a value that works as an initial value but reads as obviously needing revision, never Java's silent zero-values. Replaying that one semantic writes and reads every field data type the store supports — the ledger's serialization smoke test.

Data Type Defaults Pattern shape shows the pattern's full shape; the indented *default* row under each field is read live from the pattern's own default value semantic, through the same computed identity any editor would resolve.

Table 18. Data Type Defaults Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 DEFAULTED DATA TYPES	 DATA TYPE DEFAULT PROVISION	—
<i>Field 1</i>		
 STRING DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 STRING DATA TYPE
<i>default UNINITIALIZED</i>		
<i>Field 2</i>		
 COMPONENT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 COMPONENT DATA TYPE
<i>default  UNINITIALIZED COMPONENT</i>		
<i>Field 3</i>		
 COMPONENTIDSET DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 COMPONENT ID SET DATA TYPE
<i>default  UNINITIALIZED COMPONENT</i>		

Meaning	Purpose	Data type
Field 4		
 COMPONENTIDLIST DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 COMPONENT ID LIST DATA TYPE
default  UNINITIALIZED COMPONENT		
Field 5		
 DiTREE DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 DiTREE DATA TYPE
default 1-vertex tree		
Field 6		
 DiGRAPH DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 DiGRAPH DATA TYPE
default 2-vertex cycle		
Field 7		
 CONCEPT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 CONCEPT DATA TYPE
default  UNINITIALIZED COMPONENT		
Field 8		
 SEMANTIC DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 SEMANTIC DATA TYPE
default Uninitialized Component (SOLOR)		
Field 9		
 INTEGER DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 INTEGER DATA TYPE
default 777777777		
Field 10		
 FLOAT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 FLOAT DATA TYPE
default NaN		
Field 11		
 BOOLEAN DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 BOOLEAN DATA TYPE

Meaning	Purpose	Data type
<i>default false</i>		
Field 12		
 BYTEARRAY DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 BYTE ARRAY DATA TYPE
<i>default UNINITIALIZED</i>		
Field 13		
 ARRAY DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 ARRAY DATA TYPE
<i>default [UNINITIALIZED]</i>		
Field 14		
 INSTANT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 INSTANT LITERAL
<i>default Pre-inception</i>		
Field 15		
 LONG DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 LONG
<i>default 7777777777777777</i>		
Field 16		
 DECIMAL DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 DECIMAL DATA TYPE
<i>default 77777777.777</i>		

The sixteen decided defaults group into a small number of strategies:

- **The UNINITIALIZED string forms.** String is the literal `UNINITIALIZED`; ByteArray is the UTF-8 bytes of `UNINITIALIZED` — the loud String default carried in byte form; Array is an array of exactly one element, the String `UNINITIALIZED` — present and non-empty, since an empty array would be a silent zero-value.
- **The loud placeholder component.** Component and Concept default to `UNINITIALIZED COMPONENT`, the base set's own loud placeholder for a value still owed — deliberately not `BLANK CONCEPT`: Blank means a deliberate not-applicable, a quiet, legitimate

state, while Uninitialized means a value still owed. ComponentIdSet and ComponentIdList are the singleton set and list holding  **UNINITIALIZED COMPONENT** — present and non-empty, their one member itself the loud placeholder.

- **Native non-values.** Types whose value space offers a true non-value use it — a non-value is louder still, because it cannot be mistaken for data at all. Float is NaN, which also propagates through arithmetic, so any computation over an unrevised default stays loudly not-a-number; Instant is the pre-inception instant (historically 'premundane'), the time type's own before-all-recorded-time marker, chosen over a sentinel date because a true non-value cannot be mistaken for a real timestamp.
- **Stretched-sevens sentinels.** Numeric types without a native non-value use a planted repdigit: Integer is 777777777 (nine sevens), Long is 777777777777777777 (eighteen sevens), and Decimal is 777777777.777 — nine sevens, a decimal point, three more sevens, the decimal point placed deliberately to demonstrate the type's distinguishing property: a scaled decimal, not an integer. A repdigit signature reads as deliberately planted and cannot plausibly occur as natural data, where MIN_VALUE would read as an overflow bug.
- **Structural values that prove their type.** DiGraph is a deliberate simple cycle — two vertices, both  **UNINITIALIZED COMPONENT**, with edges in both directions: a cycle can never be a tree, so the value proves the field truly holds a graph and not a tree. DiTree is the smallest well-formed tree — a single vertex, the loud placeholder at its root.
- **A real, resolvable semantic.** The Semantic field's default is the fully qualified name description semantic of  **UNINITIALIZED COMPONENT** — a real, resolvable semantic whose subject is itself the loud placeholder, so the reference is valid yet plainly unrevised.
- **The flagged compromise.** Boolean is false: a two-valued type has no loud member, so no Boolean default can announce itself as unrevised; false is offered as the initial value with that limitation stated rather than hidden.

9.5. A Worked Default: Preferred Reviewer

  **PREFERRED REVIEWER PATTERN** carries the set's other default value semantic — the minimal end-to-end demonstration the sixteen-type exemplar generalizes. Its single field,  **PREFERRED REVIEWER**, is value-set constrained to the   **STARTER SET AUTHOR ROSTER PATTERN** roster (see the Field Constraints chapter), and its authored default is  **GRETTEL**, that roster's first entry — a real, constraint-satisfying default value, not a placeholder. The full round trip is visible in this guide itself: the default is authored as ordinary ledger content in the Defaults and templates module, StampCalculator.getDefault(pattern) resolves it by computed identity, and the same value renders as the *default* row of Preferred Reviewer Pattern shape.

10. Tinkar Base Model

This chapter describes the component/chronicle/pattern-of-patterns meta-model every other chapter's own terminology ultimately descends from.

10.1. Components and Chronicles

 **MODEL CONCEPT** anchors every concept in this guide that describes the knowledge base's own structure rather than a domain it represents — the model-versus-data distinction  **DATA CONCEPT** draws on the other side ( **DEFAULT DATA CONCEPT** and its siblings group actual data values, not structural terminology).  **TINKAR MODEL CONCEPT** is the root of Tinkar's own meta-schema specifically: components, descriptions, dialects, fields, and axioms — every chapter in this guide, from  **DESCRIPTION PATTERN** to  **AXIOMS** to  **FIELD CATEGORIES**, ultimately descends from this one concept.

A component's persisted form is a **chronicle**:  **CHRONICLE PROPERTIES** names what every chronicle carries regardless of component kind —  **PRIMORDIAL UUID FOR CHRONICLE** (its own birth identity),  **SEMANTIC LIST FOR CHRONICLE** (every semantic attached to it),  **VERSION LIST FOR CHRONICLE** (its own version history).  **COMPONENT CHRONOLOGY PATTERN** and  **COMPONENT VERSION PATTERN** name the generic shape every chronicle/version pair takes;  **CONCEPT FIELD PATTERN** /  **CONCEPT VERSION PATTERN**,  **PATTERN CHRONOLOGY PATTERN** /  **PATTERN VERSION PATTERN**,  **SEMANTIC CHRONOLOGY PATTERN** /  **SEMANTIC VERSION FIELD PATTERN**, and  **STAMP CHRONOLOGY PATTERN** /  **STAMP VERSION FIELD PATTERN** specialize that shape for each of the four component kinds this guide has already named.

 **TINKAR BASE MODEL COMPONENT PATTERN** and  **KOMET BASE MODEL COMPONENT PATTERN** are membership patterns, not shape descriptions — both carrying a referenced-component meaning of  **STARTER DATA AUTHORIZING** and purpose of  **SET MEMBERSHIP**, with no fields of their own: a semantic of one of these patterns marks a component as belonging to Tinkar's own base terminology or Komet's own — the same membership-pattern convention

 **SOLOR CONCEPTS PATTERN** uses (discussed in the Logic Coordinates chapter) to mark a component as part of the SOLOR concept space.

Each chronicle-shape pattern's own referenced component and fields make that specialization concrete.  **COMPONENT CHRONOLOGY PATTERN** — the base shape — carries a referenced-component meaning of  **COMPONENT FIELD** and a purpose of  **CHRONICLE IDENTITY AND HISTORY** — one shared concept naming why every chronicle-shape pattern exists: to record a component's own identity alongside its complete version history — and two fields, each with its own meaning and purpose concept:  **PUBLIC ID FIELD** (meaning) for  **UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS** (purpose), and  **COMPONENT VERSIONS FIELD** (meaning) for  **COMPONENT VERSIONS SET** (purpose) — every chronicle's own identity plus its version list.  **CONCEPT FIELD PATTERN** and  **PATTERN CHRONOLOGY PATTERN** specialize that same two-field shape for concepts and patterns respectively (referenced-component meaning  **CONCEPT FIELD** or  **PATTERN FIELD** in turn, the same shared purpose, fields  **PUBLIC ID FIELD** /  **CONCEPT VERSIONS FIELD** or  **PUBLIC ID FIELD** /  **PATTERN VERSIONS FIELD**).  **SEMANTIC CHRONOLOGY PATTERN** carries two fields more than its siblings — a semantic's own chronicle must also record which pattern it is a semantic **of** and what it is **attached to** — a referenced-component meaning of  **SEMANTIC FIELD**

and the same shared purpose, and four fields:

 PUBLIC ID FIELD /  UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS ,
 SEMANTIC PATTERN FIELD (meaning) for  PATTERN MEMBERSHIP (purpose — which pattern this semantic is an instance of),  SEMANTIC REFERENCED COMPONENT FIELD (meaning) for
 ATTACHMENT TARGET (purpose — what this semantic is attached to), and
 SEMANTIC VERSIONS SET (meaning) for  VERSION HISTORY (purpose).
 P  STAMP CHRONOLOGY PATTERN rounds out the family — meaning  STAMP FIELD , the same shared purpose again, fields
 PUBLIC ID FIELD /  UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS and
 STAMP VERSIONS FIELD /  STAMP VERSIONS SET .

The version side mirrors this, with its own shared purpose concept:

 P  COMPONENT VERSION PATTERN and  P  CONCEPT VERSION PATTERN each carry a referenced-component meaning of  COMPONENT VERSIONS FIELD or  CONCEPT VERSIONS FIELD respectively, and a purpose of  VERSION SNAPSHOT — why every version-shape pattern exists: to record one point-in-time state of a chronicle. Each has a single field,  STAMP FIELD (meaning) for  VERSION PROVENANCE (purpose) — a version, at its simplest, is just its own STAMP, recorded for provenance: who committed it, in what state, module, path, and when.

 P  SEMANTIC VERSION FIELD PATTERN adds a second field to that base shape — meaning  SEMANTIC VERSIONS FIELD , the same shared purpose, fields
 STAMP FIELD /  VERSION PROVENANCE and (already correctly distinct) meaning  SEMANTIC FIELD FIELD for purpose  SEMANTIC FIELD FIELDS SET — a semantic's own version must also carry the actual field values that version holds.  P  STAMP VERSION FIELD PATTERN is the richest of the chronicle/version family: meaning  STAMP VERSIONS FIELD , the same shared purpose, and six fields —  STAMP FIELD /  VERSION PROVENANCE , plus
 STATUS FIELD /  STATUS FOR VERSION ,  TIME FIELD /  TIME FOR VERSION (data type  STRING DATA TYPE),  AUTHOR FIELD /  AUTHOR FOR VERSION ,
 MODULE FIELD /  MODULE FOR VERSION , and  PATH FIELD /  PATH FOR VERSION — the same  STATUS FOR VERSION /  AUTHOR FOR VERSION /
 MODULE FOR VERSION /  PATH FOR VERSION /  TIME FOR VERSION purpose concepts this guide's own STAMP pattern (see the STAMP Concepts chapter) also uses; the base-model layer specializes those five fields into a single, more directly authored five-field pattern.

 P  PATTERN VERSION PATTERN is richer still: meaning  PATTERN VERSIONS FIELD , the same shared purpose, and four fields —  STAMP FIELD /  VERSION PROVENANCE ,
 PATTERN MEANING FIELD (meaning) for  MEANING DECLARATION (purpose),
 PATTERN PURPOSE FIELD (meaning) for  PURPOSE DECLARATION (purpose), and
 FIELD DEFINITION FIELD (purpose  FIELD DEFINITIONS SET , data type  COMPONENT ID SET DATA TYPE) — literally the shape behind every pattern declaration in this starter set's own source: the meaning, purpose, and field-definition list every pattern this guide names is itself an instance of.

A semantic's own properties are named separately from its host chronicle's:

 **SEMANTIC PROPERTIES** groups  **COMPONENT FOR SEMANTIC** (which pattern it is of), **K** **Referenced Component Nid For Semantic** (what it is attached to),  **LOGIC GRAPH FOR SEMANTIC** (its axiom graph, when it carries one), and  **SEMANTIC FIELD NAME** (a field's own name within it).  **CONCEPT TYPE** names special **concept** kinds rather than component kinds —  **ANONYMOUS CONCEPT** (defined on the fly, without a dedicated name),  **PATH CONCEPT** (a concept that is itself a version-control path), and  **SEMANTIC FIELD CONCEPTS** (a concept minted to serve as one semantic's field value).

The base-model shape patterns' own tables follow, in the order the narrative names them: the chronicle and version shapes for components, concepts, patterns, semantics, and STAMPs, then the base-model membership patterns.

Table 19. Component Chronology Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 COMPONENT FIELD	 CHRONICLE IDENTITY AND HISTORY	—
<i>Field 1</i>		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
<i>Field 2</i>		
 COMPONENT VERSIONS FIELD	 COMPONENT VERSIONS SET	 COMPONENT ID SET DATA TYPE

Table 20. Component Version Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 COMPONENT VERSIONS FIELD	 VERSION SNAPSHOT	—
<i>Field 1</i>		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE

Table 21. Concept field pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT FIELD	 CHRONICLE IDENTITY AND HISTORY	—
<i>Field 1</i>		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
<i>Field 2</i>		
 CONCEPT VERSIONS FIELD	 CONCEPT VERSIONS SET	 COMPONENT ID SET DATA TYPE

Table 22. Concept Version Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT VERSIONS FIELD	 VERSION SNAPSHOT	—
<i>Field 1</i>		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE

Table 23. Pattern Chronology Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PATTERN FIELD	 CHRONICLE IDENTITY AND HISTORY	—
<i>Field 1</i>		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
<i>Field 2</i>		
 PATTERN VERSIONS FIELD	 PATTERN VERSIONS SET	 COMPONENT ID SET DATA TYPE

Table 24. Pattern Version Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PATTERN VERSIONS FIELD	 VERSION SNAPSHOT	—
Field 1		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE
Field 2		
 PATTERN MEANING FIELD	 MEANING DECLARATION	 COMPONENT DATA TYPE
Field 3		
 PATTERN PURPOSE FIELD	 PURPOSE DECLARATION	 COMPONENT DATA TYPE
Field 4		
 FIELD DEFINITION FIELD	 FIELD DEFINITIONS SET	 COMPONENT ID SET DATA TYPE

Table 25. Semantic Chronology Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 SEMANTIC FIELD	 CHRONICLE IDENTITY AND HISTORY	—
Field 1		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT
Field 2		
 SEMANTIC PATTERN FIELD	 PATTERN MEMBERSHIP	 COMPONENT
Field 3		
 SEMANTIC REFERENCED COMPONENT FIELD	 ATTACHMENT TARGET	 COMPONENT
Field 4		

Meaning	Purpose	Data type
 SEMANTIC VERSIONS SET	 VERSION HISTORY	 COMPONENT

Table 26. Semantic version field pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 SEMANTIC VERSIONS FIELD	 VERSION SNAPSHOT	—
Field 1		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE
Field 2		
 SEMANTIC FIELD FIELD	 SEMANTIC FIELD FIELDS SET	 COMPONENT ID SET DATA TYPE

Table 27. STAMP Chronology Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 STAMP FIELD	 CHRONICLE IDENTITY AND HISTORY	—
Field 1		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
Field 2		
 STAMP VERSIONS FIELD	 STAMP VERSIONS SET	 COMPONENT ID SET DATA TYPE

Table 28. STAMP version field pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 STAMP VERSIONS FIELD	 VERSION SNAPSHOT	—

Meaning	Purpose	Data type
<i>Field 1</i>		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE
<i>Field 2</i>		
 STATUS FIELD	 STATUS FOR VERSION	 COMPONENT DATA TYPE
<i>Field 3</i>		
 TIME FIELD	 TIME FOR VERSION	 STRING DATA TYPE
<i>Field 4</i>		
 AUTHOR FIELD	 AUTHOR FOR VERSION	 COMPONENT DATA TYPE
<i>Field 5</i>		
 MODULE FIELD	 MODULE FOR VERSION	 COMPONENT DATA TYPE
<i>Field 6</i>		
 PATH FIELD	 PATH FOR VERSION	 COMPONENT DATA TYPE

Table 29. Tinkar base model component pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STARTER DATA AUTHORIZING	 SET MEMBERSHIP	—
<i>e.g.</i>  UNINITIALIZED COMPONENT		

Table 30. Komet base model component pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STARTER DATA AUTHORIZING	 SET MEMBERSHIP	—

Meaning	Purpose	Data type
e.g.   KOMET BASE MODEL COMPONENT PATTERN		

10.2. Base Model Primitives

Beneath  OBJECT sit the primitives every other component kind is ultimately built from:  NID names the native-store integer identity every component resolves to at runtime — never replay-stable, so this guide's own tooling never renders one — and  ANY COMPONENT is the generic container a field can hold when its value might be any component kind at all, rather than one narrowed by a field's own data type.

Three further concepts under  OBJECT are sentinels, not real modeled content:

 BLANK CONCEPT is a placeholder meaning no meaningful concept value applies to a field — this starter set's own constraint apparatus deliberately carries no such sentinel: one constraint pattern per parameter shape means every field of every constraint semantic is meaningful — while  SANDBOX COMPONENT and  UNINITIALIZED COMPONENT mark a component as belonging to a sandbox authoring session or as not yet initialized, respectively, rather than describing any real, committed content.

 PROPERTY SEQUENCE,  REFLEXIVE FEATURE, and  TRANSITIVE FEATURE round out this base model with description-logic property characteristics inherited from SOLOR: a reflexive property relates every concept to itself; a transitive property, if it relates A to B and B to C, also relates A to C — mathematical characteristics a role's own semantics can carry, layered on top of the role terminology this guide's EL++ Concepts chapter already named.

11. Value Constraints and Reference Ranges

Where   TAXONOMY FIELD CONSTRAINT PATTERN and   VALUE-SET FIELD CONSTRAINT PATTERN restrict which values are legal for a **pattern field**,  VALUE CONSTRAINT restricts which **literal values** are legal for a component — a numeric-domain cousin, and a much older idea in this starter set:   VALUE CONSTRAINT PATTERN (introduced in the EL++ Concepts chapter's Concrete Value Operators subsection) is the pattern that carries it.  VALUE CONSTRAINT SOURCE names the organization that specifies the constraint — a reference-range source, for instance — and  REFERENCE RANGE groups the two endpoints a numeric constraint actually bounds:  REFERENCE RANGE MINIMUM and  REFERENCE RANGE MAXIMUM. Together with  CONCRETE VALUE OPERATOR's comparison operators, this terminology is what a lab-value or dose constraint is actually built from — a component "has specific value requirements that need to be met," in  VALUE CONSTRAINT's own words, exactly the framing the Semantic Field Model chapter's constraint patterns use for pattern fields, just one domain over.

As a pattern, **P** **VALUE CONSTRAINT PATTERN** itself carries a referenced-component meaning of **VALUE CONSTRAINT** and a purpose of **NUMERIC VALUE RESTRICTION** — the numeric-domain counterpart to **FIELD VALUE RESTRICTION** — and six fields, each with its own meaning and purpose concept: **VALUE CONSTRAINT SOURCE** (meaning) for **REFERENCE RANGE AUTHORITY** (purpose — which organization specifies the constraint); **MINIMUM VALUE OPERATOR** and **MAXIMUM VALUE OPERATOR** (meaning) for **CONCRETE VALUE OPERATOR** (purpose); **REFERENCE RANGE MINIMUM** and **REFERENCE RANGE MAXIMUM** (meaning) for **REFERENCE RANGE** (purpose); and **EXAMPLE UCUM UNITS** (meaning) for **UNIT EXAMPLE** (purpose — a representative unit of measure for the constrained value, for a reader's reference).

DESCRIPTION-LOGIC PROFILE names the description-logic profile a set of axioms classifies against — **EL++ LOGIC PROFILE** is this starter set's own (EL, already introduced in the EL Concepts chapter) — grouped here because it is, like **VALUE CONSTRAINT**, a property of a component's logical shape rather than a taxonomy-organizing concept in its own right.

Value Constraint Pattern shape shows the pattern's shape.

Table 31. Value Constraint Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
VALUE CONSTRAINT	NUMERIC VALUE RESTRICTION	—
<i>Field 1</i>		
VALUE CONSTRAINT SOURCE	REFERENCE RANGE AUTHORITY	CONCEPT DATA TYPE
<i>Field 2</i>		
MINIMUM VALUE OPERATOR	CONCRETE VALUE OPERATOR	CONCEPT DATA TYPE
<i>Field 3</i>		
REFERENCE RANGE MINIMUM	REFERENCE RANGE	FLOAT DATA TYPE
<i>Field 4</i>		
MAXIMUM VALUE OPERATOR	CONCRETE VALUE OPERATOR	COMPONENT DATA TYPE
<i>Field 5</i>		
REFERENCE RANGE MAXIMUM	REFERENCE RANGE	FLOAT DATA TYPE

Meaning	Purpose	Data type
<i>Field 6</i>		
 EXAMPLE UCUM UNITS	 UNIT EXAMPLE	 STRING DATA TYPE

12. Authoring Actions and Legacy Constraints

 **ACTION PROPERTIES** groups the attributes of an authoring **action** — a step Komet or another editor takes on the author's behalf, rather than a property of the content itself.  **CONCEPT TO FIND** names a concept a search action is looking for;  **CONDITIONAL TRIGGERS** names conditions, based on actions or the reasoner, that fire further behavior;  **ROLE TYPE TO ADD** names the role type an "add role" action would insert.

 **CONCEPT CONSTRAINTS** deserves particular attention here, having already been named once in this guide's Semantic Field Model chapter: a real, if long-dormant, SOLOR concept defined simply as "defined filters for a given concept" — a gloss close enough to the constraint patterns' own purpose that the two are worth contrasting directly.  **CONCEPT CONSTRAINTS** carries no field shape, no worked example, and no consuming code anywhere in this ecosystem; it is a name for the idea of constraining a concept, sitting here among other action-related SOLOR imports, never wired to anything.  **TAXONOMY FIELD CONSTRAINT PATTERN** and  **VALUE-SET FIELD CONSTRAINT PATTERN** are the KB-native mechanism this starter set actually built and demonstrated once that same need became concrete — reusing  **CONCEPT CONSTRAINTS**'s identity was considered and deliberately declined, precisely because a name alone, unaccompanied by any real structure, is not enough to safely resume.

13. Feature-Based and Intrinsic Roles

 **DESCRIPTION TYPE** is best known from the Language Concepts chapter as the parent of  **FULLY QUALIFIED NAME DESCRIPTION TYPE**,  **REGULAR NAME DESCRIPTION TYPE**, and  **DEFINITION DESCRIPTION TYPE** — but its remaining children are a different family entirely: feature- and role-modeling refinements imported from SOLOR alongside the description-type terminology, not further description types themselves.  **INTRINSIC ROLE** and  **UNMODELED ROLE CONCEPT** extend this guide's own EL++ Concepts chapter — an intrinsic role is one inherent to a concept's own definition rather than externally imposed, while an unmodeled role concept marks a relationship SNOMED CT or Tinkar has not yet formally captured as a role.  **PROPERTY SET** and  **FEATURE** describe concrete-domain properties a concept can carry — the same family  **CONCRETE VALUE OPERATOR** and  **VALUE CONSTRAINT** operate on, with  **FEATURE TYPE** naming which kind of feature a given one is — and  **PROPERTY PATTERN IMPLICATION** and  **CONCEPT PATTERN FOR LOGIC COORDINATE** round out the property-modeling side.

 **EXTENDED RELATIONSHIP TYPE** and  **INVERSE NAME** handle terminology-import edge cases: a relationship type from an imported terminology that maps onto a SNOMED relationship (such as is-a), and a description explicitly marked as the inverse of an association's own name.

 **DYNAMIC REFERENCED COMPONENT RESTRICTION** narrows which component types a dynamic field's referenced component may legally be — the same restriction idea the Roles and Role Operators subsection's

 **REFERENCED COMPONENT TYPE RESTRICTION** /  **REFERENCED COMPONENT SUBTYPE RESTRICTION** already introduced, here at the field-validation level rather than the axiom level.

 **SOLOR MODULE** and  **SOLOR OVERLAY MODULE** name the module and overlay-module SOLOR's own imported content is stamped with — SOLOR's counterpart to this starter set's own

 **DEVELOPMENT MODULE** /  **PRIMORDIAL MODULE** .  **DESCRIPTION CORE TYPE** and

 **EXTENDED DESCRIPTION TYPE** close the loop back to descriptions proper: a marker for a non-SNOMED description that still matches one of SNOMED's core types, and one that does not.

14. Annotations, Correlations and Editor Focus

 **ANNOTATION TYPE** groups metadata **about** a component rather than the component's own defining content:  **COMMENT** is free text an author attaches when something needs explaining (this starter set's own  **COMMENT PATTERN** carries exactly this), and  **KOMET ISSUE** is Komet's own specialization — an annotation type Komet itself defines, layered on top of the generic annotation terminology.  **ANNOTATION PROPERTY SET** names the SOLOR-imported set these annotation properties draw from.

As a pattern,  **COMMENT PATTERN** originally carried, straight from its SOLOR source, a referenced-component meaning and purpose of  **COMMENT** for both slots, with its single field also using  **COMMENT** as both its own meaning and purpose — four slots, one concept, conflating "what the field holds" (a comment) with "what role the referenced component plays" (the subject that commentary is about). A later revision splits these apart: the pattern's referenced-component meaning is now  **SUBJECT OF COMMENTARY** — the referenced component's role, distinct from the commentary's own content type — and its purpose is  **EDITORIAL CLARIFICATION** — why the commentary exists: to explain an authoring decision or provide context a formal field can't capture. The field itself keeps  **COMMENT** as its meaning (that part was always correct — the field genuinely does hold a comment) but now carries  **EDITORIAL CLARIFICATION** as its purpose too, the same concept reused at both levels; its data type,  **STRING DATA TYPE**, is unchanged. Tinkar's own `PatternVersion` model supports this cleanly: the original SOLOR-sourced version stays in history untouched, and a `StampCoordinate` positioned after the revision simply resolves the newer one. Two more small patterns round out this starter set's own editorial metadata:

 **STARTER SET AUTHOR ROSTER PATTERN** (this guide's own illustrative value-set example, introduced in the Semantic Field Model chapter) carries a referenced-component meaning of  **STARTER SET AUTHOR ROSTER** and a purpose of  **ROSTER MEMBERSHIP** — why the pattern exists, to enumerate this starter set's own author roster as a value-set source — and two fields:

 **ROSTER AUTHOR** (meaning) for  **ROSTER ENTRY** (purpose) and  **ROSTER ORDER** (meaning) for

 **DISPLAY SEQUENCE** (purpose, data type ) — the sort-order field that makes it a genuine two-field worked example for value-set field disambiguation. **P**  **PREFERRED REVIEWER PATTERN** carries a referenced-component meaning of  **PREFERRED REVIEWER ASSIGNMENT** and a purpose of  **REVIEW ROUTING** — why the pattern exists, to direct a component's future edits to a specific reviewer — and one field,  **PREFERRED REVIEWER** (meaning) for  **ASSIGNED REVIEWER** (purpose) — which author is the preferred reviewer for a given component's future edits.

 **CORRELATION PROPERTIES** groups a different kind of metadata: characteristics describing the relationship **between** two or more variables, rather than an annotation on one component alone.

 **CORRELATION EXPRESSION** and  **CORRELATION REFERENCE EXPRESSION** name the value and the reference value a correlation compares.

 **COMPONENT TYPE FOCUS** names which kind of component an editor's attention is currently on —  **AXIOM FOCUS**,  **CONCEPT FOCUS**, and  **DESCRIPTION FOCUS** — a UI-facing concern distinct from any of the component-kind concept families (concept, pattern, semantic) this guide's Tinkar Base Model chapter already named, since "focus" describes where an author is looking, not what a component is.  **CONCEPT DETAILS TREE TABLE** names the tree-table view Komet renders a concept's details in — one editor surface among several a concept's data can project through.

Comment pattern shape, Starter Set Author Roster Pattern shape, and Preferred Reviewer Pattern shape show these three patterns' shapes.

Table 32. Comment pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 SUBJECT OF COMMENTARY	 EDITORIAL CLARIFICATION	—
<i>Field 1</i>		
 COMMENT	 EDITORIAL CLARIFICATION	 STRING DATA TYPE

Table 33. Starter Set Author Roster Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STARTER SET AUTHOR ROSTER	 ROSTER MEMBERSHIP	—
<i>e.g. P  STARTER SET AUTHOR ROSTER PATTERN</i>		

Meaning	Purpose	Data type
<i>Field 1</i>		
 ROSTER AUTHOR	 ROSTER ENTRY	 COMPONENT DATA TYPE
<i>e.g.</i>  GRETEL		
<i>Field 2</i>		
 ROSTER ORDER	 DISPLAY SEQUENCE	 LONG
<i>e.g.</i> 1		

Table 34. Preferred Reviewer Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PREFERRED REVIEWER ASSIGNMENT	 REVIEW ROUTING	—
<i>Field 1</i>		
 PREFERRED REVIEWER	 ASSIGNED REVIEWER	 COMPONENT DATA TYPE
<i>default</i>  GRETEL		

15. Identifiers

 IDENTIFIER SOURCE names which identifier-issuing authority a component's alternate identifier came from — the field-meaning concept  IDENTIFIER PATTERN itself declares a field for.

 UNIVERSALLYUNIQUEIDENTIFIER is the one identifier source every component in this starter set actually carries: the UUID itself, which uniquely represents a concept in Tinkar independent of any external terminology's own identifier scheme.  IDENTIFIER VALUE names the field holding the literal identifier string once a source has been chosen — an SCTID, an external code, or, for the universally-unique-identifier source, the UUID's own text form.

As a pattern,  IDENTIFIER PATTERN itself carries a referenced-component meaning of  IDENTIFIED COMPONENT — the component an external identifier names, distinct from the field-level  IDENTIFIER SOURCE the discussion above named — and a purpose of  EXTERNAL IDENTITY MAPPING : why the pattern exists, to record an alternate identifier a component carries in an external terminology's own identifier scheme. Two fields each carry their

own meaning and purpose: IDENTIFIER SOURCE (meaning) for IDENTIFIER AUTHORITY (purpose — which authority issued the identifier) and IDENTIFIER VALUE (meaning) for IDENTIFIER TEXT (purpose — the identifier’s own literal text).

Identifier Pattern shape shows the identifier pattern’s shape.

Table 35. Identifier Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern’s semantics attach to</i>		
IDENTIFIED COMPONENT	EXTERNAL IDENTITY MAPPING	—
<i>e.g. UNINITIALIZED COMPONENT</i>		
<i>Field 1</i>		
IDENTIFIER SOURCE	IDENTIFIER AUTHORITY	COMPONENT DATA TYPE
<i>e.g. UNIVERSALLY UNIQUE IDENTIFIER</i>		
<i>Field 2</i>		
IDENTIFIER VALUE	IDENTIFIER TEXT	STRING DATA TYPE
<i>e.g. 55f74246-0a25-57ac-9473-a788d08fb656</i>		

16. Coordinate Properties: Language and Path

The Language Concepts chapter already named LANGUAGE COORDINATE PROPERTIES’ own four preference-list fields in the course of explaining how a `LanguageCoordinate` resolves a description; LANGUAGE COORDINATE NAME and DIALECT PATTERN PREFERENCE LIST FOR LANGUAGE COORDINATE are two of that same coordinate’s own field-meaning concepts, named here directly as members of the property group they belong to, rather than only implicitly through the preference-list discussion.

PATH COORDINATE PROPERTIES plays the analogous role for PATH : PATH COORDINATE NAME names the coordinate’s own name field, and PATH ORIGINS names which path a path itself branched from — the same idea the STAMP Concepts chapter’s Path subsection already introduced as PATH ORIGINS FOR STAMP PATH, here at the coordinate-property level rather than the individual-path level.

17. Tree Amalgams

 **TREE AMALGAM PROPERTIES** names the property group behind a composite navigation tree — a view that blends more than one taxonomy tree into a single presentation, rather than walking one  **NAVIGATION VERTEX** structure alone.  **TREE LIST** names the member trees an amalgam combines, and  **INVERSE TREE LIST** names the same idea inverted — the trees an amalgam excludes, or walks in reverse, depending on how a particular amalgam is configured. Komet's own multi-hierarchy navigation views (a concept's SNOMED-style parents alongside its Tinkar-model parents, say, blended into one tree) are exactly the kind of view this property group exists to support.

18. Clinical Phenomena and Domain Content

Every chapter so far has described this starter set's own meta-schema — the terminology Tinkar and this starter set use to describe knowledge, not any particular domain's content.  **PHENOMENON** is different: it groups genuine, if sparse, seed **domain** content — "a unique thought, fact, or circumstance," in its own words — included here to show what real authored content looks like sitting on top of everything the rest of this guide describes.  **HEALTH CONCEPT** and  **UNCATEGORIZED PHENOMENON** are its two organizing children, and  **EXAMPLE UCUM UNITS** names a field carrying example Unified Code for Units of Measure values for a given term — the concrete-value terminology the EL++ Concepts chapter's Concrete Value Operators subsection already introduced, applied here to real measurement content rather than described only in the abstract.

 **HAS ACTIVE INGREDIENT**,  **HAS DOSE FORM**, and  **LATERALITY** are three more seed domain attributes, each a SOLOR-imported relationship type rather than a meta-schema concept: a medicinal product's active ingredient, its dose form, and a finding or procedure's left/right/bilateral sidedness — small, recognizable examples of exactly the kind of role this guide's Roles and Role Operators subsection described in the abstract, now anchored to real clinical content.

19. Object Properties and Relationships

 **OBJECT PROPERTIES** names the general property-holding terminology every more specific property group in this guide —  **CHRONICLE PROPERTIES**,  **SEMANTIC PROPERTIES**,  **ACTION PROPERTIES**,  **CORRELATION PROPERTIES**,  **LANGUAGE COORDINATE PROPERTIES**,  **PATH COORDINATE PROPERTIES**,  **TREE AMALGAM PROPERTIES**,  **ANNOTATION PROPERTY SET**,  **DATA PROPERTY SET**, and  **PROPERTY SEQUENCE IMPLICATION** among them — ultimately descends from: "objects are instances of classes; properties describe the data an object can have," in its own words.

 **RELATIONSHIP DESTINATION** and  **RELATIONSHIP ORIGIN** name the two directions a relationship between concepts can be walked: destination toward more specific, child concepts, and origin toward more general, parent concepts — the same is-a direction this guide's Navigation Coordinates

chapter already described through  **NAVIGATION VERTEX** 's own parent/child fields, named here as the generic relationship-direction terminology those fields draw from.  **DATA PROPERTY SET** and  **PROPERTY SEQUENCE IMPLICATION** close out the property family: a set of data-carrying properties, and an implication one property sequence carries for another.

20. Provenance and Root

 **INTEGRATED KNOWLEDGE MANAGEMENT** is the concept nearly every root-level concept family in this guide ultimately descends from —  **MODEL CONCEPT** ,  **AUTHOR** ,  **MODULE** ,  **PATH** ,  **STATUS VALUE** ,  **OBJECT** ,  **ANNOTATION TYPE** ,  **PHENOMENON** , and  **CREATIVE COMMONS BY LICENSE** among them — "terminologies represented in a harmonized manner," in its own words: the single organizing idea that this starter set, Tinkar's own base model, and SOLOR's imported terminology are all, ultimately, one coherent knowledge representation rather than three unrelated things bolted together.

 **IKEFOUNDATION ROOT** is this starter set's own root, by contrast — the concept this guide's very first chapter named as the anchor its whole append-only ledger is rooted at, a child of  **MODEL CONCEPT** rather than of  **INTEGRATED KNOWLEDGE MANAGEMENT** directly, since it is IKE's own specific starter content, not a generic model concept.  **CREATIVE COMMONS BY LICENSE** names the license this starter set's own content is released under.

A handful of component-chronicle-level fields round out this guide's coverage:  **CONCEPT VERSION** captures which version of a source terminology a concept came from, and  **DESCRIPTION** ,  **DESCRIPTION LIST FOR CONCEPT** ,  **DESCRIPTION SEMANTIC** , and  **TEXT FOR DESCRIPTION** name the description-pattern fields the Language Concepts chapter's own prose already explained in practice — cited here directly, by their own field-meaning identity, for a reader who arrived at this guide's final chapter looking for exactly this list.

21. Konzept Glossary

Acceptable

Specifies that a description is acceptable, but not preferred within a language or dialect.

⊆ Description acceptability

Parents:  **DESCRIPTION ACCEPTABILITY**

Since: Inception

id: Acceptable | UUID: 12b9e103-060e-3256-9982-18c1191af60e

Action properties

The retired ISAAC action-rule vocabulary: the parameters a rule-driven authoring action took. Retained for identity compatibility.

↳ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Children:  CONDITIONAL TRIGGERS  CONCEPT CONSTRAINTS  ROLE TYPE TO ADD
 CONCEPT TO FIND

Since: Inception

id: ActionProperties | UUID: 80ba281c-7d47-57cf-8100-82b69bce998b

Active state

Concept used to represent a status for components that are active.

↳ Status value

Parents:  STATUS VALUE

Since: Inception

id: ActiveState | UUID: 09f12001-0e4f-51e2-9852-44862a4a0db4

Allowed states for stamp coordinate

Predefined list of values for STAMP coordinate

↳ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: AllowedStatesForStampCoordinate | UUID: 23f69f6f-a502-5876-a835-2b1b4d5ce91e

And

An operator that typically is employed to combine two conditions

↳ Connective operator

Parents:  CONNECTIVE OPERATOR

Since: Inception

id: And | UUID: fa113d51-07d2-587c-8930-0bce207d506d



Annotation property set

The OWL annotation-property grouping: properties that annotate content without logical consequence, carried for OWL interchange.

⊆ Logical expression model

Parents: LOGICAL EXPRESSION MODEL

Since: Inception

id: AnnotationPropertySet | UUID: cb9e33de-f82c-495d-89fa-69afecbcd47d



Annotation type

Metadata about program elements, and annotation types define the structure of these annotations

⊆ Legacy model concepts

Parents: LEGACY MODEL CONCEPTS

Since: Inception

id: AnnotationType | UUID: 3fe77951-58c9-51b3-8e7e-65edcf7ace0a



Anonymous concept

Concepts or entities that do not have a specific, named identity, (defined on-the-fly without a dedicated name)

⊆ Concept type

Parents: CONCEPT TYPE

Since: Inception

id: AnonymousConcept | UUID: f8f936d4-3ac7-5629-9f65-9452608056a1



Any component

A general-purpose container to represent any component with generic data structure. Modifiable based on the specific requirements and characteristics of the components.

⊟ Chronicle and version model

Parents:  CHRONICLE AND VERSION MODEL

Since: Inception

id: AnyComponent | UUID: 927da7ac-3403-5ccc-b07b-88f60cc3a5f8

Array

The retired dynamic-column type for ordered element sequences, superseded by Array data type.

⊟ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: Array | UUID: 318622e6-dd7a-5651-851d-2d5c2af85767

Array data type

A lexical set of semantically related elements/items

⊟ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: ArrayDataType | UUID: b168ad04-f814-5036-b886-fd4913de88c8

Array default

The Array field's loud default: an array of exactly one element, the String "UNINITIALIZED" — present and non-empty (an empty array would be a silent zero-value), its single element the loud String default.

⊟ Defaults and templates model ▯ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: ArrayDefault | UUID: daae3fab-6803-5c61-84ef-9d1d1f17c97b

Assigned Reviewer

Why a preferred reviewer value is recorded: to name which author is the preferred reviewer.

⊆ Editorial model ⊐ Purpose

Parents:  EDITORIAL MODEL  PURPOSE

Since: Inception

id: AssignedReviewer | UUID: b25df610-7523-5db8-8954-97878fd4e5e1

Atom

A logical axiom that participates in a set's expression rather than framing one: connectives (and, or), concept references, disjoint-with assertions, property sequence implications, and the typed atoms (role, interval role, feature). Mirrors LogicalAxiom.Atom in the sealed hierarchy.

⊆ Logical axiom

Parents:  LOGICAL AXIOM

Children:  CONNECTIVE OPERATOR  CONCEPT REFERENCE  TYPED ATOM
 PROPERTY SEQUENCE IMPLICATION  DISJOINT WITH

Since: Inception

id: Atom | UUID: e9d6a6d4-b509-5852-9ef7-8b4d2e637461

Attachment Target

Why a Semantic Chronology Pattern's Semantic referenced component field is recorded: to identify what component this semantic is attached to.

⊆ Chronicle and version model ⊐ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: AttachmentTarget | UUID: 1faad323-8fdd-51e3-8209-c1f904d478e0

Author

The concept identifying who committed a version — the author dimension of a STAMP, and the root of the value space an author field's concept is drawn from; distinct from Author for version, which names why the field is recorded.

⌵ STAMP dimensions ▢ Meaning

Parents:  STAMP DIMENSIONS  MEANING

Children:  SNOCKET CLASSIFIER  IKE COMMUNITY  TINKAR STARTER DATA AUTHOR
 GRETEL  KOMET USER

Since: Inception

id: Author | UUID: f7495b58-6630-3499-a44e-2052b5fcf06c

Author field

A concept field whose value is the author dimension of a STAMP — who committed the version.

⌵ Concept field ▢ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: AuthorField | UUID: a9210ad6-cc48-47df-86e5-2192d56704a6

Author for edit coordinate

Individual or entity who made a particular edit or revision in a document (authoring a specific location or point in the codebase where an edit was made)

⌵ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: AuthorForEditCoordinate | UUID: 337e93ba-531b-59a4-8153-57dca00e58d2

Author for stamp coordinate

Individual or an entity responsible for defining or updating the values associated with the STAMP coordinate

⌵ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: AuthorForStampCoordinate | UUID: 4fda23b8-b016-5d2a-97d5-7ff779d60701

Author for version

Individual or entity who made a specific set of changes or modifications to a codebase/terminology resulting in the creation of a new version or revision

⊆ Version Properties ⊆ Purpose

Parents:  VERSION PROPERTIES  PURPOSE

Since: Inception

id: AuthorForVersion | UUID: 4eb9de0d-7486-5f18-a9b4-82e3432f4103

Axiom Expression

Why an OWL Axiom Syntax Pattern semantic exists: to express a component's definitional axioms as OWL functional-syntax text.

⊆ Logical expression model ⊆ Purpose

Parents:  LOGICAL EXPRESSION MODEL  PURPOSE

Since: Inception

id: AxiomExpression | UUID: 8a080408-6f2a-560e-ab75-65477a0075d8

Axiom focus

A statement or proposition that is assumed to be true without requiring proof, it serves as a foundation principles on which a system or theory is built. Focus refers to the central point of attention or concentration on a specific concept/axioms

⊆ Component type focus

Parents:  COMPONENT TYPE FOCUS

Since: Inception

id: AxiomFocus | UUID: 9c6fbddd-58bd-5881-b926-c813bbff849b

Axiom origin

Groups the two origins a definition can have: stated, as an editor authored it, or inferred, as a reasoner derived it.

⊆ Axioms

Parents:  AXIOMS

Children:  INFERRED PREMISE TYPE  STATED PREMISE TYPE

Since: Inception

id: AxiomOrigin | UUID: b868bd83-5cd4-5d84-9cf7-b08674fbc79b

Axiom Syntax

Groups the syntaxes a definition's axioms can be serialized in for exchange, such as OWL functional syntax.

⊆ Axioms ⊇ Meaning

Parents:  AXIOMS  MEANING

Children:  EXPRESS AXIOM SYNTAX

Since: Inception

id: AxiomSyntax | UUID: 8da1c508-c2a2-4899-b26d-87f8b98a7558

Axiomatized Component

What an OWL Axiom Syntax Pattern semantic's referenced component is: the component whose definitional axioms the semantic expresses — distinct from Axiom syntax, which names the syntax text the field holds.

⊆ Logical expression model ⊇ Meaning

Parents:  LOGICAL EXPRESSION MODEL  MEANING

Since: Inception

id: AxiomatizedComponent | UUID: 94325002-4bf4-5236-ada7-0109d9682e08

Axioms

Groups the logical-axiom meta-schema concepts: the terminology for expressing a concept's formal, machine-processable meaning — EL++ terminological axioms, interval-set axioms, and their syntax and origin.

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Children:  EL++  TERMINOLOGICAL AXIOMS  AXIOM ORIGIN  INTERVAL SET AXIOMS
 AXIOM SYNTAX  TEMPORAL AXIOM

Since: Inception

id: Axioms | UUID: 20746b91-521a-45a6-89da-0fe32384a67f

Blank Concept

The editor's empty-value marker: what an editing surface writes into a component-valued field a person has deliberately cleared, and what it tests for when asking whether that field is empty. An authored, persisted value -- distinct from Uninitialized Component, the platform's null-object for a reference that was never set at all. Neither asserts anything about the world; a claim that no value applies would need a concept this set does not yet carry.

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Since: Inception

id: BlankConcept | UUID: cd23d88d-2fcd-4007-8829-97e37bf336aa

Boolean

The retired dynamic-column type for boolean values, superseded by Boolean data type.

⊆ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: Boolean | UUID: 08f2fb74-980d-5157-b92c-4ff1eac6a506

Boolean data type

The data type for fields holding a two-valued truth value: true or false.

⊆ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: BooleanDataType | UUID: d6b9e2cc-31c6-5e80-91b7-7537690aae32

Boolean default

The Boolean field's default: false — a flagged compromise. A two-valued type has no loud member, so no Boolean default can announce itself as unrevised; false is offered as the initial value with that limitation stated rather than hidden.

⊆ Defaults and templates model ⊐ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: BooleanDefault | UUID: 3125e470-edb7-5498-b00b-6f53e45e09b3

Boolean field

The field kind for boolean-valued fields: a pattern field holding true or false.

⊆ Field definition data type field

Parents:  FIELD DEFINITION DATA TYPE FIELD

Since: Inception

id: BooleanField | UUID: 4229683e-8772-4936-abd5-edc5a180f4d1

Boolean literal

A literal whose value is true or false, as compared in a concrete-domain axiom.

⊆ Literal value

Parents:  LITERAL VALUE

Since: Inception

id: BooleanLiteral | UUID: 49f41695-66a7-5471-846d-21c168f54c19

Boolean reference

Reference(a pointer) to a Boolean object

⊆ Query clauses

Parents:  QUERY CLAUSES

Since: Inception

id: BooleanReference | UUID: de49d207-a26e-5f8a-b905-953a4dd13c21

 Boolean substitution

The process of replacing or substituting boolean values or expression in a logical context

⊆ Field substitution

Parents:  FIELD SUBSTITUTION

Since: Inception

id: BooleanSubstitution | UUID: 03559f9d-f1e4-5485-894b-4d457f145d54

 Branch Point

Why the origin instant is recorded: to fix the moment this path branched from its origin — the point up to which the origin path's versions are visible from this path.

⊆ Provenance model ⊐ Purpose

Parents:  PROVENANCE MODEL  PURPOSE

Since: Inception

id: BranchPoint | UUID: 80c6f90e-a85e-5be5-a447-380efdb0e37c

 Branch Source

Why the origin path value is recorded: to say which path this path branched from. The record's subject is the referenced component — the originated path — never this field.

⊆ Provenance model ⊐ Purpose

Parents:  PROVENANCE MODEL  PURPOSE

Since: Inception

id: BranchSource | UUID: 3f29da5f-a31f-5917-899e-e991032ca2e2

 Byte array

The retired dynamic-column type for raw byte sequences, superseded by Byte array data type.

⊆ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: ByteArray | UUID: 9a84fecf-708d-5de4-9c5f-e17973229e0f

Byte array data type

The data type for fields holding a raw sequence of bytes, uninterpreted by the model – an encoded document, an image, or any opaque payload.

⊞ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: ByteArrayDataType | UUID: dbdd8df2-aec3-596b-88fc-7b83b5594a45

ByteArray default

The ByteArray field's loud default: the UTF-8 bytes of "UNINITIALIZED" — the loud String default carried in byte form, never a silent empty array.

⊞ Defaults and templates model ▢ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: ByteArrayDefault | UUID: 4d6448a1-a5a7-5ed8-a10d-0cf1312cd413

Canceled state

Concept used to represent a status for components that are canceled

⊞ Status value

Parents:  STATUS VALUE

Since: Inception

id: CanceledState | UUID: b42c1948-7645-5da8-a888-de6ec020ab98

Case insensitive evaluation

Evaluates values regardless of the case

⊞ Text comparison measure semantic

Parents:  TEXT COMPARISON MEASURE SEMANTIC

Since: Inception

id: CaseInsensitiveEvaluation | UUID: 74bbdaff-f061-5807-b334-3c88ac3e9421

Case sensitive evaluation

The text-comparison measure that distinguishes upper from lower case: two strings match only if their case agrees.

⊆ Text comparison measure semantic

Parents:  TEXT COMPARISON MEASURE SEMANTIC

Since: Inception

id: CaseSensitiveEvaluation | UUID: a95e5dbc-a179-57f9-9cdd-6de8c026396d

Case Sensitivity Rule

Why a description's case significance value is recorded: to say whether the description's text is case-sensitive.

⊆ Description model ⊃ Purpose

Parents:  DESCRIPTION MODEL  PURPOSE

Since: Inception

id: CaseSensitivityRule | UUID: 7c02e59b-7759-5cc2-a8aa-cc29dbf569f9

Case significance concept nid for description

A field label which captures the case significance for a given concept description

⊆ Description version properties

Parents:  DESCRIPTION VERSION PROPERTIES

Since: Inception

id: CaseSignificanceConceptNidForDescription | UUID: 57271621-3f3c-58dd-8148-2674bc11b7e5

Chinese language

The Chinese language, as a value for a description's language field: a language coordinate that names it selects Chinese-language descriptions for rendering.

Language

Parents:  LANGUAGE

Since: Inception

id: ChineseLanguage | UUID: aacbc859-e9a0-5e01-b6a9-9a255a47b0c9

Chronicle and version model

The meta-schema of chronicles and their versions: the field kinds every chronicle shape records (public identity, the versions collection), the field kinds every version shape records (STAMP, the pattern-declaration and semantic-value slots), and the meaning/purpose concepts those shapes declare. This subsystem describes how knowledge is carried; what any of it is about belongs to the domain content itself.

Model concept

Parents:  MODEL CONCEPT

Children:  PURPOSE DECLARATION  ANY COMPONENT  SEMANTIC PROPERTIES
 VERSION PROPERTIES  SET MEMBERSHIP  UNINITIALIZED COMPONENT  VERSION SNAPSHOT
 CHRONICLE PROPERTIES  MEANING DECLARATION  PATTERN MEMBERSHIP
 ATTACHMENT TARGET  CHRONICLE IDENTITY AND HISTORY  VERSION HISTORY

Since: Inception

id: ChronicleAndVersionModel | UUID: 863dfaec-a508-5c9f-a51f-d8691ab38e8b

Chronicle Identity and History

Why a chronicle-shape pattern's referenced component carries this semantic: to record that component's own identity (public id) alongside its complete version history.

Chronicle and version model Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: ChronicleIdentityAndHistory | UUID: d00b0f8b-05ea-58c5-be24-b6ea54f1de86

Chronicle properties

Attributes or characteristic associated with a historical record or an account of events (metadata, timestamps)

≡ Chronicle and version model

Parents:  CHRONICLE AND VERSION MODEL

Children:  UUID LIST FOR COMPONENT  PRIMORDIAL UUID FOR CHRONICLE
 VERSION LIST FOR CHRONICLE  SEMANTIC LIST FOR CHRONICLE

Since: Inception

id: ChronicleProperties | UUID: 2ba2ef47-30af-57ec-9073-38693f020d7e

Classifier for logic coordinate

The logic-coordinate dimension naming which reasoner owns the inferred content a view resolves; inferred semantics are stamped with the classifier as their author.

≡ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ClassifierForLogicCoordinate | UUID: 4b90e89d-2a0e-5ca3-8ae5-7498d148a9d2

Comment

A filed label to capture free text information which may be necessary to add or change (concepts, relationships, semantics, etc)

≡ Editorial model ▯ Meaning

Parents:  EDITORIAL MODEL  MEANING

Since: Inception

id: Comment | UUID: 147832d4-b9b8-5062-8891-19f9c4e4760a

Comment pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 SUBJECT OF COMMENTARY	 EDITORIAL CLARIFICATION	—

Meaning	Purpose	Data type
<i>Field 1</i>		
 COMMENT	 EDITORIAL CLARIFICATION	 STRING DATA TYPE

Since: Inception

id: CommentPattern | UUID: 3734fb0a-4c14-5831-9a61-4743af609e7a

Commit Provenance

Why a STAMP pattern semantic exists: to record who committed a version, in what state, module, path, and when.

☰ Provenance model ▢ Purpose

Parents:  PROVENANCE MODEL  PURPOSE

Since: Inception

id: CommitProvenance | UUID: 3e03a324-4a64-55ec-915c-cf56f97beef5

Component Chronology Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 COMPONENT FIELD	 CHRONICLE IDENTITY AND HISTORY	—
<i>Field 1</i>		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
<i>Field 2</i>		
 COMPONENT VERSIONS FIELD	 COMPONENT VERSIONS SET	 COMPONENT ID SET DATA TYPE

Since: Inception

id: ComponentChronologyPattern | UUID: c48db76d-5eb0-4ff5-84d0-5c3c4ec77767

Component data type

A field that identifies any component – concept, semantic, pattern, or STAMP – by reference, not restricted to concepts.

⊞ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: ComponentDataType | UUID: fb00d132-fcc3-5cbf-881d-4bcc4b4c91b3

Component default

The Component field's loud default: Uninitialized Component (SOLOR), the base set's own loud placeholder for a component not yet chosen. Not Blank Concept — Blank means a deliberate not-applicable, while Uninitialized means a value still owed.

⊞ Defaults and templates model ⊞ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: ComponentDefault | UUID: 153b82c3-6210-5833-a124-19e773b21d09

Component field

The field kind whose value references a component of any kind — concept, pattern, semantic, or STAMP. The parent of every reference-valued field kind, and the Component Chronology Pattern's referenced-component meaning: the component whose identity and history the chronicle records.

⊞ Field categories ⊞ Meaning

Parents:  FIELD CATEGORIES  MEANING

Children:  CONCEPT FIELD  SEMANTIC REFERENCED COMPONENT FIELD  STAMP FIELD
 SEMANTIC FIELD  PATTERN FIELD

Since: Inception

id: ComponentField | UUID: 8bd36a0c-d05d-46b7-a79a-d11477705cc1

**Component for semantic**

The field naming the component a semantic is attached to, in the semantic-properties vocabulary. Duplicates Referenced component for semantic and the live Semantic referenced component field kind, which is what pattern declarations actually use; retained for identity compatibility.

⊆ Legacy model concepts

Parents: LEGACY MODEL CONCEPTS

Since: Inception

id: ComponentForSemantic | UUID: 0bc32c16-698e-5719-8bd5-efa099c7d782

**Component Id list data type**

An ordered list of component ids.

⊆ Display Fields

Parents: DISPLAY FIELDS

Since: Inception

id: ComponentIdListDataType | UUID: e553d3f1-63e1-4292-a3a9-af646fe44292

**Component Id set data type**

An unordered set of component ids.

⊆ Display Fields

Parents: DISPLAY FIELDS

Since: Inception

id: ComponentIdSetDataType | UUID: e283af51-2e8f-44fa-9bf1-89a99a7c7631

**Component semantic**

The semantic type whose field value is a component reference.

⊆ Semantic type

Parents: SEMANTIC TYPE

Since: Inception

id: ComponentSemantic | UUID: 127e7274-0b88-5519-a9db-85d4b9ce6a4a

 Component type focus

Groups the component kinds a rule or action can be scoped to: concepts, descriptions, or axioms.

Tinkar Model concept

Parents:  TINKAR MODEL CONCEPTChildren:  CONCEPT FOCUS  DESCRIPTION FOCUS  AXIOM FOCUS

Since: Inception

id: ComponentTypeFocus | UUID: f1f179d0-26af-5123-9b29-9fc6cd01dd29

 Component Version Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 COMPONENT VERSIONS FIELD	 VERSION SNAPSHOT	—
<i>Field 1</i>		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE

Since: Inception

id: ComponentVersionPattern | UUID: a38b7d2d-8fa5-4206-9185-a1af9f81be2c

 Component versions field

The field kind holding a chronicle's version collection — one entry per committed version of the component. Every chronicle-shape pattern declares it beside the public id, and the Component Version Pattern carries it as its own referenced-component meaning.

Field categories

Meaning

Parents:  FIELD CATEGORIES  MEANINGChildren:  CONCEPT VERSIONS FIELD  PATTERN VERSIONS FIELD  STAMP VERSIONS FIELD
 SEMANTIC VERSIONS FIELD

Since: Inception

id: ComponentVersionsField | UUID: 1a852426-422a-48db-a618-c906ac4c8e6c

Component versions set

Why a chronicle's versions field is recorded: to hold every version the component has ever had — the complete history a chronicle exists to keep alongside its identity.

☰ Field categories ▢ Purpose

Parents:  FIELD CATEGORIES  PURPOSE

Children:  SEMANTIC VERSIONS SET  STAMP VERSIONS SET  PATTERN VERSIONS SET
 CONCEPT VERSIONS SET

Since: Inception

id: ComponentVersionsSet | UUID: 54d670f1-234d-485a-a354-e1fa7eea1bf2

ComponentIdList default

The ComponentIdList field's loud default: the singleton list holding Uninitialized Component — present and non-empty (an empty list would be a silent zero-value), its one entry itself the loud placeholder.

☰ Defaults and templates model ▢ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: ComponentIdListDefault | UUID: 9b2ebb85-a651-5fa7-8636-9239fe354f3e

ComponentIdSet default

The ComponentIdSet field's loud default: the singleton set holding Uninitialized Component — present and non-empty (an empty set would be a silent zero-value), its one member itself the loud placeholder.

☰ Defaults and templates model ▢ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: ComponentIdSetDefault | UUID: 28cb73e5-68d2-5fa3-8b9b-bc2142ecbffe



Concept constraints

The retired action parameter holding the filters a rule-driven action applied when selecting which concepts to act on.

⊞ Action properties

Parents:  ACTION PROPERTIES

Since: Inception

id: ConceptConstraints | UUID: 081273cd-92dd-593c-9d9b-63d33838e70b



Concept data type

Field for the human readable description for the given concept

⊞ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: ConceptDataType | UUID: ac8f1f54-c7c6-5fc7-b1a8-ebb04b918557



Concept default

The Concept field's loud default: Uninitialized Component (SOLOR) — the loud placeholder for a concept not yet chosen, obviously needing revision wherever it appears.

⊞ Defaults and templates model ▯ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: ConceptDefault | UUID: eb902228-e21c-57e2-a66c-b7db1235c9e5



Concept details tree table

The Komet surface presenting a concept's details as a tree table: descriptions, axioms, and attached semantics in one expandable view.

⊞ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Since: Inception

id: ConceptDetailsTreeTable | UUID: 1655edd8-7b73-52c5-98b0-263d1ab3a90b

Concept field

A component field narrowed to concepts: its value references a concept specifically. The parent of the STAMP-dimension field kinds and of a pattern's meaning- and purpose-declaration field kinds, and the Concept Chronology Pattern's referenced-component meaning.

☰ Component field ☐ Meaning

Parents:  COMPONENT FIELD  MEANING

Children:  VALUE-SET FIELD  PATH FIELD  FIELD DEFINITION PURPOSE FIELD
 CONSTRAINT ANCHOR CONCEPT  STATUS FIELD  CONSTRAINED FIELD  ROSTER AUTHOR
 AUTHOR FIELD  CONSTRAINT KIND  PATTERN MEANING FIELD
 FIELD DEFINITION MEANING FIELD  MODULE FIELD  PREFERRED REVIEWER
 PATTERN PURPOSE FIELD  VALUE-SET PATTERN

Since: Inception

id: ConceptField | UUID: ebe2aa74-f100-41b2-8d75-2d8f06ce5e4e

Concept field pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT FIELD	 CHRONICLE IDENTITY AND HISTORY	—
<i>Field 1</i>		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
<i>Field 2</i>		
 CONCEPT VERSIONS FIELD	 CONCEPT VERSIONS SET	 COMPONENT ID SET DATA TYPE

Since: Inception

id: ConceptFieldPattern | UUID: 3e510cb9-1666-4676-9334-d288a56bf155

Concept focus

The component-type focus selecting concepts: an action or rule scoped to concept components.

⊆ Component type focus

Parents:  COMPONENT TYPE FOCUS

Since: Inception

id: ConceptFocus | UUID: dca9854d-9e4c-5e8a-8b30-6c1af6901bb8

Concept pattern for logic coordinate

The pattern whose active semantics enumerate the SOLOR concepts a Logic Coordinate reasons over.

⊆ ImmutableCoordinate Properties ⊐ Meaning

Parents:  IMMUTABLECOORDINATE PROPERTIES  MEANING

Since: Inception

id: ConceptPatternForLogicCoordinate | UUID: 16486419-5d1c-574f-bde6-21910ad66f44

Concept reference

A field to capture a reference to validate concept

⊆ Atom

Parents:  ATOM

Since: Inception

id: ConceptReference | UUID: e89148c7-4fe2-52f8-abb9-6a53605d20cb

Concept semantic

The semantic type whose field value is a concept reference: a semantic that points at one concept.

⊆ Semantic type

Parents:  SEMANTIC TYPE

Since: Inception

id: ConceptSemantic | UUID: fbf054fb-ceaf-5ab8-b946-bbcc4835ce07

Concept substitution

A field substitution whose substituted value is a concept reference; part of the retired ISAAC field-substitution family.

⊆ Field substitution

Parents:  FIELD SUBSTITUTION

Since: Inception

id: ConceptSubstitution | UUID: 23483990-b738-5f43-bc03-befd43928a37

Concept to find

Find concept (if searching on Komet shows us the results 'details and further information?')

⊆ Action properties

Parents:  ACTION PROPERTIES

Since: Inception

id: ConceptToFind | UUID: 91687b29-3bbb-540b-9de6-91246c75afd0

Concept type

A field that captures a defined concept label

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Children:  SEMANTIC FIELD CONCEPTS  ANONYMOUS CONCEPT  PATH CONCEPT

Since: Inception

id: ConceptType | UUID: 106f3ba1-63b8-5596-8dbe-524fa2e89fc0

Concept version

A field that captures the version of the terminology that it came from

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Since: Inception

id: `ConceptVersion` | UUID: `c202f992-3f4b-5f30-9b32-e376f68367d1`



Concept Version Pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
CONCEPT VERSIONS FIELD	VERSION SNAPSHOT	—
Field 1		
STAMP FIELD	VERSION PROVENANCE	COMPONENT DATA TYPE

Since: Inception

id: `ConceptVersionPattern` | UUID: `7943a5f1-538b-4fda-8acb-019e0bec125b`



Concept versions field

The component versions field narrowed to concept chronicles: its value collects a concept's own versions. The Concept Version Pattern's referenced-component meaning.

⊞ Component versions field ⊞ Meaning

Parents: COMPONENT VERSIONS FIELD MEANING

Since: Inception

id: `ConceptVersionsField` | UUID: `3a08b5f1-f17e-4db5-8cf9-c6540f26f241`



Concept versions set

Why a concept chronicle's versions field is recorded: to hold every version the concept has ever had — the concept-chronicle specialization of Component versions set.

⊞ Component versions set ⊞ Purpose

Parents: COMPONENT VERSIONS SET PURPOSE

Since: Inception

id: `ConceptVersionsSet` | UUID: `806c7f9f-52f9-4b53-9758-122899b28a76`



Concrete value operator

Groups the comparison operators a concrete-domain axiom uses to relate a measured quantity to a literal bound: equal, less than, greater than, and their inclusive forms.

⊆ Tinkar Model concept ⊧ Purpose

Parents: TINKAR MODEL CONCEPT PURPOSE

Children: LESS THAN OR EQUAL TO EQUAL TO GREATER THAN OR EQUAL TO GREATER THAN
 LESS THAN MAXIMUM VALUE OPERATOR MINIMUM VALUE OPERATOR

Since: Inception

id: ConcreteValueOperator | UUID: 843b0b55-8785-5544-93f6-581da9cf1ff3



Conditional triggers

Conditional triggers based on actions, reasoner

⊆ Action properties

Parents: ACTION PROPERTIES

Since: Inception

id: ConditionalTriggers | UUID: a3e4ac54-db82-5345-8713-7e0da98bbb0a



Connective operator

A field that captures what the operator is (logical connective)

⊆ Atom

Parents: ATOM

Children: AND OR

Since: Inception

id: ConnectiveOperator | UUID: 3fdcaadc-d972-58e9-84f1-b3a39903b076



Constrained field

The field-meaning concept of the pattern field this constraint governs.

⊆ Concept field ⊧ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: ConstrainedField | UUID: 45756d94-080b-5a6b-a751-12af64cc7000

Constrained Pattern

The referenced-component role a field constraint semantic's attachment target plays: the pattern that has one of its own fields constrained — one attachment role, shared by the Taxonomy Field Constraint Pattern and the Value-set Field Constraint Pattern, and distinct from the constraint mechanisms themselves.

⊆ Constraint model ⊎ Meaning

Parents:  CONSTRAINT MODEL  MEANING

Since: Inception

id: ConstrainedPattern | UUID: 548d102c-d84a-5135-a45a-b7d35d9718cd

Constraint anchor concept

The concept a kind-of, descendant, leaf-descendant, or immediate-child constraint is relative to.

⊆ Concept field ⊎ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: ConstraintAnchorConcept | UUID: e1782109-1a60-5bb5-a7e4-9742e06d62c2

Constraint kind

Which Taxonomy field constraint kind this constraint semantic expresses.

⊆ Concept field ⊎ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: ConstraintKind | UUID: 4b623f76-1a22-5bdd-9e1a-b1e3c1cbd829

Constraint model

The meta-schema of field constraints (IKE-Network/ike-issues#890): the constraint-declaration roles (constrained pattern and field, rule, scope, anchor), the taxonomy constraint kinds, the value-set apparatus with its member match relations, and the restriction concepts constraint patterns declare as meanings and purposes. Constraints are first-class pattern content — declared, versioned, and exchanged like any other knowledge.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  MEMBER MATCH RELATION  TAXONOMY FIELD CONSTRAINT KIND
 NUMERIC VALUE RESTRICTION  CONSTRAINT SCOPE  CONSTRAINED PATTERN
 TAXONOMY REFERENCE POINT  MATCH RULE  VALUE DISAMBIGUATION  UNIT EXAMPLE
 CONSTRAINT RULE  LEGAL VALUE SOURCE  REFERENCE RANGE AUTHORITY
 FIELD VALUE RESTRICTION

Since: Inception

id: ConstraintModel | UUID: 8147fe74-0e97-5568-b121-244a68902afd

Constraint Rule

Why the Constraint kind value is recorded: to say which taxonomy-relative rule — kind-of, descendant, leaf-descendant, or immediate-child — applies.

⊆ Constraint model ⊐ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: ConstraintRule | UUID: 179e8846-380f-54cc-8eef-27c1820e5e9e

Constraint Scope

Why the Constrained field value is recorded: to say which field of the constrained pattern this rule applies to.

⊆ Constraint model ⊐ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: ConstraintScope | UUID: 13c213d9-b5f2-5b02-ab9f-452724ead9f0

Correlation expression

The subject side of a correlation: the expression being checked for correspondence against a reference expression.

⊆ Correlation properties

Parents:  CORRELATION PROPERTIES

Since: Inception

id: CorrelationExpression | UUID: 1711815f-99a1-5d1a-8f1e-75dc7bf41928

Correlation properties

Characteristics or measures that describe the relationship between two or more variables

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Children:  CORRELATION REFERENCE EXPRESSION  CORRELATION EXPRESSION

Since: Inception

id: CorrelationProperties | UUID: 8f682e00-3d9e-5506-bd19-b2169d6c8752

Correlation reference expression

The reference side of a correlation: the expression an authored expression is compared against when checking whether two expressions correspond.

⊆ Correlation properties

Parents:  CORRELATION PROPERTIES

Since: Inception

id: CorrelationReferenceExpression | UUID: acb73d95-7c96-590c-9f24-23da54f95ff2

Creative Commons BY license

Creative Commons (CC) licenses are a set of public copyright licenses that enable the free distribution of an otherwise copyrighted work

⊆ Provenance model

Parents:  PROVENANCE MODEL

Since: Inception

id: CreativeCommonsBYLicense | UUID: 3415a972-7850-57cd-aa86-a572ca1c2ceb

Czech dialect

The Czech dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in Czech usage; a language coordinate's dialect order consults it when choosing what to show.

⊞ Dialect

Parents:  DIALECT

Since: Inception

id: CzechDialect | UUID: 6979e268-0b59-542f-bac0-313c5ddf6a2e

Czech language

The Czech language, as a value for a description's language field: a language coordinate that names it selects Czech-language descriptions for rendering.

⊞ Language

Parents:  LANGUAGE

Since: Inception

id: CzechLanguage | UUID: 33aa2d26-0541-557c-b796-904cbf245101

Danish language

The Danish language, as a value for a description's language field: a language coordinate that names it selects Danish-language descriptions for rendering.

⊞ Language

Parents:  LANGUAGE

Since: Inception

id: DanishLanguage | UUID: 987681fb-f3ef-595d-90e2-067baf2bc71f

Data Concept

Groups concepts that represent data values and defaults — the data-carrying side of the model, as distinct from Tinkar Model concept's structural, self-descriptive side.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  DEFAULT DATA CONCEPT

Since: Inception

id: DataConcept | UUID: ae7069d1-67fa-4470-a56f-0d24a8fcea83

Data property set

The logical set grouping data-property axioms under a definition root. Mirrors LogicalAxiom.LogicalSet.DataPropertySet.

⊆ Logical set

Parents:  LOGICAL SET

Since: Inception

id: DataPropertySet | UUID: 6b8ed642-de72-4aee-953d-42e5db92c0ab

Data Property Set Axioms

A baseline grouping for data-property-set axioms, retained for identity compatibility: the platform's set meaning is Data property set.

⊆ Logical set

Parents:  LOGICAL SET

Since: Inception

id: DataPropertySetAxioms | UUID: 1402d311-0b4b-4014-81d2-e715c6696346

Data type default exemplar

Why every Data Type Defaults Pattern field value is recorded: to carry its data type's loud default — a value that works as an initial value but reads as obviously needing revision, never Java's silent zero-value. One purpose serves all sixteen fields because every field exists for this identical reason; distinct purposes would be manufactured variety.

⊆ Defaults and templates model ▯ Purpose

Parents:  DEFAULTS AND TEMPLATES MODEL  PURPOSE

Since: Inception

id: `DataTypeDefaultExemplar` | UUID: `ffbec350-7b66-5efd-a2ba-b972a1ae3870`

Data Type Default Provision

Why a Data Type Defaults Pattern semantic exists: to provide, in one semantic, a loud default value for every data type a pattern field can declare — and, replayed, to exercise every field serialization path the store supports end to end.

▢ Defaults and templates model ▢ Purpose

Parents:  DEFAULTS AND TEMPLATES MODEL  PURPOSE

Since: Inception

id: `DataTypeDefaultProvision` | UUID: `96b4fd68-4378-545d-9d4e-926283161a31`

Data Type Defaults Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 DEFAULTED DATA TYPES	 DATA TYPE DEFAULT PROVISION	—
<i>Field 1</i>		
 STRING DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 STRING DATA TYPE
<i>default UNINITIALIZED</i>		
<i>Field 2</i>		
 COMPONENT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 COMPONENT DATA TYPE
<i>default  UNINITIALIZED COMPONENT</i>		
<i>Field 3</i>		
 COMPONENTIDSET DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 COMPONENT ID SET DATA TYPE
<i>default  UNINITIALIZED COMPONENT</i>		
<i>Field 4</i>		

Meaning	Purpose	Data type
 COMPONENTIDLIST DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 COMPONENT ID LIST DATA TYPE
default  UNINITIALIZED COMPONENT		
Field 5		
 DiTREE DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 DiTREE DATA TYPE
default 1-vertex tree		
Field 6		
 DiGRAPH DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 DiGRAPH DATA TYPE
default 2-vertex cycle		
Field 7		
 CONCEPT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 CONCEPT DATA TYPE
default  UNINITIALIZED COMPONENT		
Field 8		
 SEMANTIC DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 SEMANTIC DATA TYPE
default Uninitialized Component (SOLOR)		
Field 9		
 INTEGER DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 INTEGER DATA TYPE
default 777777777		
Field 10		
 FLOAT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 FLOAT DATA TYPE
default NaN		
Field 11		
 BOOLEAN DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 BOOLEAN DATA TYPE
default false		

Meaning	Purpose	Data type
<i>Field 12</i>		
 BYTEARRAY DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 BYTE ARRAY DATA TYPE
<i>default UNINITIALIZED</i>		
<i>Field 13</i>		
 ARRAY DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 ARRAY DATA TYPE
<i>default [UNINITIALIZED]</i>		
<i>Field 14</i>		
 INSTANT DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 INSTANT LITERAL
<i>default Pre-inception</i>		
<i>Field 15</i>		
 LONG DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 LONG
<i>default 7777777777777777</i>		
<i>Field 16</i>		
 DECIMAL DEFAULT	 DATA TYPE DEFAULT EXEMPLAR	 DECIMAL DATA TYPE
<i>default 777777777.777</i>		

Since: Inception

id: DataTypeDefaultsPattern | UUID: e604e510-ffd6-5f04-ab6e-50842dedd43a

Decimal

The retired dynamic-column type for high-precision decimal values, superseded by Decimal data type.

↳ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: `Decimal` | UUID: `dccb0476-3b63-3d48-b5a2-85bd0ad2fa61` **Decimal data type**

Represents values as high-precision decimal values.

 `Display Fields`Parents:  `DISPLAY FIELDS`

Since: Inception

id: `DecimalDataType` | UUID: `b413fe94-4ada-4aee-96f9-22be19699d40` **Decimal default**

The Decimal field's loud default: the stretched-sevens sentinel `77777777.777` — nine sevens, a decimal point, three more sevens. The decimal point placement deliberately demonstrates the type's distinguishing property (a scaled decimal, not an integer) — the same move as the DiGraph cycle proving graph-ness. Never Java's silent zero.

 `Defaults and templates model`  `Meaning`Parents:  `DEFAULTS AND TEMPLATES MODEL`  `MEANING`

Since: Inception

id: `DecimalDefault` | UUID: `3eadc612-bc06-51c8-8a55-323d57c95d0b` **Default Data Concept**

The baseline's placeholder anchor for default data, retained for identity compatibility with upstream stores; its placeholder seeds are deliberately not adopted here.

 `Data Concept`Parents:  `DATA CONCEPT`

Since: Inception

id: `DefaultDataConcept` | UUID: `4a32d2ad-baca-42b5-a432-4c4ae6431668` **Default module for edit coordinate**

The edit-coordinate dimension naming the module a new edit is stamped with, unless the editor chooses another from the options.

 `ImmutableCoordinate Properties`

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: DefaultModuleForEditCoordinate | UUID: e83d322c-e275-5392-a5db-1de5fe98acb5

Default value concept

The attachment point for default values: a default value semantic of a pattern is a complete semantic of that pattern whose referenced component is this concept and whose fields carry the default values an editor offers for that pattern's fields. Attachment here is a support declaration only — never a domain assertion about this concept or any other component — and this concept never joins a domain member set.

⊞ Defaults and templates model

Parents:  DEFAULTS AND TEMPLATES MODEL

Since: Inception

id: DefaultValueConcept | UUID: b3af49c5-9e8a-5b94-94d4-b1e2b72e1c2f

Defaulted Data Types

What a Data Type Defaults Pattern semantic means: the sixteen data types ConceptToDataType recognizes for pattern fields, each exemplified by one field carrying that type's loud default.

⊞ Defaults and templates model ⊞ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: DefaultedDataTypes | UUID: a7b1ea9b-1297-566a-a175-e9b3096175c7

Defaults and templates model

The meta-schema of authoring support content (IKE-Network/ike-issues#885): the attachment points default value and template semantics reference, the per-data-type default concepts, and the exemplar/provision concepts the Data Type Defaults apparatus declares. Support content itself lives in the Defaults and templates module; these are the foundation concepts that describe it.

⊞ Model concept

Parents:  MODEL CONCEPT

Children:  DEFAULTED DATA TYPES  LONG DEFAULT  CONCEPT DEFAULT  BOOLEAN DEFAULT
 ARRAY DEFAULT  SEMANTIC DEFAULT  DIGRAPH DEFAULT  BYTEARRAY DEFAULT
 COMPONENTIDLIST DEFAULT  STRING DEFAULT  INTEGER DEFAULT  COMPONENTIDSET DEFAULT
 DECIMAL DEFAULT  FLOAT DEFAULT  DITREE DEFAULT  COMPONENT DEFAULT
 INSTANT DEFAULT  DEFAULT VALUE CONCEPT  TEMPLATE CONCEPT
 DATA TYPE DEFAULT PROVISION  DATA TYPE DEFAULT EXEMPLAR

Since: Inception

id: DefaultsAndTemplatesModel | UUID: 78ea9f07-028e-5e7a-8d9c-993947a663dc

Defaults and templates module

The module scoping every default value and template semantic. The boundary is bidirectional and live-and-die: every version of a defaults or template chronology lives in this module, and this module holds only defaults and template content — a violation in either direction is an error to be withdrawn, never adopted as precedent. As the export dimension, module crossed with path selects a user's preferences for exchange: include this module to carry defaults and templates alongside domain content, exclude it to leave them behind.

⊆ Module

Parents:  MODULE

Since: Inception

id: DefaultsAndTemplatesModule | UUID: 8d013392-1db6-58cd-a1cb-74a1e603399a

Definition description type

Semantic value describing the description type for the description pattern is a definition

⊆ Description type

Parents:  DESCRIPTION TYPE

Since: Inception

id: DefinitionDescriptionType | UUID: 700546a3-09c7-3fc2-9eb9-53d318659a09

Definition root

The root vertex of a definition graph: every stated or inferred definition begins here, and its children are the definition's logical sets. Mirrors LogicalAxiom.DefinitionRoot.

⊆ Logical axiom

Parents:  LOGICAL AXIOM

Since: Inception

id: DefinitionRoot | UUID: e7271c01-6ed4-5240-963f-34d1f24153b0

Descendant field constraint

Legal values are every descendant of the anchor concept, not including the anchor concept itself.

⊆ Taxonomy field constraint kind

Parents:  TAXONOMY FIELD CONSTRAINT KIND

Since: Inception

id: DescendantFieldConstraint | UUID: f503b7bc-84ff-5fab-b5a1-384b831894fa

Description

Human-readable text attached to a component: the words a person reads where the machinery holds an identity. Every rendering surface resolves one, choosing by language, dialect, and description type.

⊆ Tinkar Model concept ⊐ Purpose

Parents:  TINKAR MODEL CONCEPT  PURPOSE

Since: Inception

id: Description | UUID: 87118daf-d28c-55fb-8657-cd6bc8425600

Description acceptability

Whether a given human readable text for a concept is permissible

⊆ Tinkar Model concept ⊐ Meaning ⊐ Purpose

Parents:  TINKAR MODEL CONCEPT  MEANING  PURPOSE

Children:  PREFERRED  ACCEPTABLE

Since: Inception

id: DescriptionAcceptability | UUID: 96b61063-0d29-5aea-9652-3f5f328aad3

Description Attachment

Why a Description Pattern semantic exists: to attach a human-readable name or definition to a concept.

⊆ Description model ⊇ Purpose

Parents:  DESCRIPTION MODEL  PURPOSE

Since: Inception

id: DescriptionAttachment | UUID: 179bfdd3-4e9c-55ae-86b9-47214fcfcc4a

Description case sensitive

Assumes the description is dependent on capitalization

⊆ Description case significance

Parents:  DESCRIPTION CASE SIGNIFICANCE

Since: Inception

id: DescriptionCaseSensitive | UUID: 0def37bc-7e1b-384b-a6a3-3e3ceee9c52e

Description case significance

Specifies how to handle the description text in terms of case sensitivity

⊆ Tinkar Model concept ⊇ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Children:  NOT APPLICABLE  DESCRIPTION INITIAL CHARACTER CASE SENSITIVE

 DESCRIPTION CASE SENSITIVE  DESCRIPTION NOT CASE SENSITIVE

Since: Inception

id: DescriptionCaseSignificance | UUID: c3dde9ea-b144-5f49-845a-20cc7d305250

Description core type

Used to mark non-snomed descriptions as one of the core snomed types

⊆ Description type

Parents:  DESCRIPTION TYPE

Since: Inception

id: DescriptionCoreType | UUID: 351955ff-30f4-5806-a0a5-5dda79756377



Description dialect pair

The pairing of a dialect with a description: the two-field shape a dialect semantic records, saying how acceptable one description is in one dialect. Its two children name the sides of that pair.

⊆ Description version properties

Parents: DESCRIPTION VERSION PROPERTIES

Children: DESCRIPTION FOR DIALECT/DESCRIPTION PAIR DIALECT FOR DIALECT/DESCRIPTION PAIR

Since: Inception

id: DescriptionDialectPair | UUID: a27bbbf8-57b5-590c-8650-1247f6f963eb



Description focus

The component-type focus selecting descriptions: an action or rule scoped to description components.

⊆ Component type focus

Parents: COMPONENT TYPE FOCUS

Since: Inception

id: DescriptionFocus | UUID: 6edf734d-7f57-5430-9164-6ee0824fd94b



Description for dialect/description pair

The description side of a dialect-and-description pair: which description the paired acceptability judgment is about.

⊆ Description dialect pair

Parents: DESCRIPTION DIALECT PAIR

Since: Inception

id: DescriptionForDialectDescriptionPair | UUID: 1137767a-ad8b-5bc5-9842-a1f9b09d1ecc



Description initial character case sensitive

Value which designates initial character as sensitive for a given description

⊞ Description case significance

Parents:  DESCRIPTION CASE SIGNIFICANCE

Since: Inception

id: DescriptionInitialCharacterCaseSensitive | UUID: 17915e0d-ed38-3488-a35c-cda966db306a

Description list for concept

The field holding every description attached to a concept, so a rendering surface can choose among them by language, dialect, and type.

⊞ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Since: Inception

id: DescriptionListForConcept | UUID: ab3e8771-7c7c-5e57-8acf-147b16da36e2

Description logic profile for logic coordinate

The logic-coordinate dimension naming which description-logic profile the reasoner assumes: the coordinate slot whose value is a profile such as EL++ logic profile, as distinct from the profile family itself.

⊞ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: DescriptionLogicProfileForLogicCoordinate | UUID: aef80e34-b2dd-5dca-a989-3e0ee2699be3

Description model

The meta-schema of human-language rendering: how a component's descriptions attach, which roles a description plays (fully qualified name, regular name, definition), and the rules — case sensitivity, dialect acceptability — a rendering surface applies when choosing what to show.

⊞ Model concept

Parents:  MODEL CONCEPT

Children:  DESCRIPTION ATTACHMENT  DESCRIPTION ROLE  CASE SENSITIVITY RULE

Since: Inception

id: DescriptionModel | UUID: 21a76cff-6404-5164-95be-d4be98778c57



Description not case sensitive

Value which designate character as not sensitive for a given description

☐ Description case significance

Parents: DESCRIPTION CASE SIGNIFICANCE

Since: Inception

id: DescriptionNotCaseSensitive | UUID: ecea41a2-f596-3d98-99d1-771b667e55b8



Description Pattern

Contains all metadata and human readable text that describes the concept

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
DESCRIPTION SEMANTIC	DESCRIPTION ATTACHMENT	—
<i>e.g. UNINITIALIZED COMPONENT</i>		
<i>Field 1</i>		
LANGUAGE FOR DESCRIPTION	LANGUAGE	COMPONENT DATA TYPE
<i>e.g. ENGLISH LANGUAGE</i>		
<i>Field 2</i>		
TEXT FOR DESCRIPTION	DESCRIPTION	STRING DATA TYPE
<i>e.g. Uninitialized Component (SOLOR)</i>		
<i>Field 3</i>		
DESCRIPTION CASE SIGNIFICANCE	CASE SENSITIVITY RULE	COMPONENT DATA TYPE
<i>e.g. DESCRIPTION NOT CASE SENSITIVE</i>		

Meaning	Purpose	Data type
<i>Field 4</i>		
 DESCRIPTION TYPE	 DESCRIPTION ROLE	 COMPONENT DATA TYPE
<i>e.g.</i>  FULLY QUALIFIED NAME DESCRIPTION TYPE		

Since: Inception

id: DescriptionPattern | UUID: a4de0039-2625-5842-8a4c-d1ce6aebf021

Description Role

Why a description's type value is recorded: to say which of the three description roles — fully qualified name, regular name, or definition — this description plays.

⊞ Description model ⊞ Purpose

Parents:  DESCRIPTION MODEL  PURPOSE

Since: Inception

id: DescriptionRole | UUID: d0ac5edc-5467-5048-8934-787812063d0d

Description semantic

Purpose and meaning for the description pattern and dialect patterns

⊞ Tinkar Model concept ⊞ Meaning ⊞ Purpose

Parents:  TINKAR MODEL CONCEPT  MEANING  PURPOSE

Since: Inception

id: DescriptionSemantic | UUID: 81487d5f-6115-51e2-a3b3-93d783888eb8

Description type

The role a description plays for its component: fully qualified name, regular name, or definition. A rendering surface picks by type before picking by dialect.

⊞ Tinkar Model concept ⊞ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Children:  DESCRIPTION CORE TYPE  DEFINITION DESCRIPTION TYPE
 FULLY QUALIFIED NAME DESCRIPTION TYPE  INVERSE NAME  REGULAR NAME DESCRIPTION TYPE
 EXTENDED DESCRIPTION TYPE

Since: Inception

id: `DescriptionType` | UUID: `ad0c19e8-2ccc-59c1-8b7e-c56c03aca8eb`

Description type for description

Why a description version records its type: to say which role the description plays for its component -- fully qualified name, regular name, or definition -- so a rendering surface can pick by role.

↳ Description version properties

Parents:  DESCRIPTION VERSION PROPERTIES

Since: Inception

id: `DescriptionTypeForDescription` | UUID: `a00c5ad7-5b8a-5592-a28c-64057dd3caab`

Description type preference list for language coordinate

The language-coordinate dimension holding description types in preference order: which type a surface renders first when a component carries several.

↳ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: `DescriptionTypePreferenceListForLanguageCoordinate` | UUID: `44c7eab6-fdb8-5427-9d7a-52ab63f7a6f9`

Description version properties

Combination of terms that might be used in a specific context or domain

↳ Version Properties

Parents:  VERSION PROPERTIES

Children:  DESCRIPTION TYPE FOR DESCRIPTION  LANGUAGE FOR DESCRIPTION
 CASE SIGNIFICANCE CONCEPT NID FOR DESCRIPTION  DESCRIPTION DIALECT PAIR

Since: Inception

id: `DescriptionVersionProperties` | UUID: `732aad24-4add-59d6-bbc9-840a8b9dc754`

Description-logic profile

The family of description-logic profiles: a profile fixes which logical operators definitions may use; this set's profile is EL++.

⊆ `Tinkar Model concept`

Parents:  `TINKAR MODEL CONCEPT`

Children:  `EL++ LOGIC PROFILE`

Since: Inception

id: `DescriptionLogicProfile` | UUID: `14eadb10-fbd0-5999-af37-05728a8503af`

Destination module for edit coordinate

The edit-coordinate dimension naming the module a moved or promoted version lands in -- the target of a transfer, as distinct from the default module new edits are stamped with and the options an editor may choose among.

⊆ `ImmutableCoordinate Properties`

Parents:  `IMMUTABLECOORDINATE PROPERTIES`

Since: Inception

id: `DestinationModuleForEditCoordinate` | UUID: `349cfd1d-10fd-5f8d-a0a5-d5ef0932b4da`

Development module

Predefines or standard module within a system or application that is specifically designed to support the development phase of a project

⊆ `Module`

Parents:  `MODULE`

Since: Inception

id: `DevelopmentModule` | UUID: `529a7069-bd33-59e6-b2ce-537fa874360a`

Development path

A path that specifies that the components are currently under development

⊆ Path

Parents:  PATH

Since: Inception

id: DevelopmentPath | UUID: 1f200ca6-960e-11e5-8994-feff819cdc9f

Dialect

A regional or professional variety of a language, differing in which terms are preferred rather than in the language itself. Dialect semantics record whether a description is preferred or acceptable in one, and a language coordinate consults them in order.

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Children:  RUSSIAN DIALECT  POLISH DIALECT  IRISH DIALECT  ENGLISH DIALECT
 FRENCH DIALECT  CZECH DIALECT  KOREAN DIALECT

Since: Inception

id: Dialect | UUID: b9c34574-c9ac-503b-aa24-456a0ec949a2

Dialect for dialect/description pair

The dialect side of a dialect-and-description pair: which dialect the paired acceptability judgment applies in.

⊆ Description dialect pair

Parents:  DESCRIPTION DIALECT PAIR

Since: Inception

id: DialectForDialectDescriptionPair | UUID: 850bc47d-5235-5bce-99f4-c41f8a163d69

Dialect pattern preference list for language coordinate

The preference order among dialect patterns a Language Coordinate consults when resolving a description's acceptability.

⊆ Language coordinate properties

Parents:  LANGUAGE COORDINATE PROPERTIES

Since: Inception

id: `DialectPatternPreferenceListForLanguageCoordinate` | UUID: `c060ffbf-e95f-5960-b296-8a3255c820ac`

DiGraph data type

A field that holds a directed graph whose edges are ordered pairs of vertices. Each edge can be followed from one vertex to another vertex.

⌵ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: `DiGraphDataType` | UUID: `60113dfe-2bad-11eb-adc1-0242ac120002`

DiGraph default

The DiGraph field's loud default: a simple cycle — two vertices, both Uninitialized Component, with edges in both directions. The cycle is deliberate: a cycle can never be a tree, so the value proves the field truly holds a graph and not a tree.

⌵ Defaults and templates model ▢ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: `DiGraphDefault` | UUID: `87cb6ced-a7e4-5e96-9a20-c1ef56d68e75`

Digraph for logic coordinate

A value which describes a immutable coordinate property

⌵ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: `DigraphForLogicCoordinate` | UUID: `1cdacc80-0dea-580f-a77b-8a6b273eb673`

Directed graph

A graph whose edges each run from one vertex to another, so processing follows the direction: the representational data structure navigation, lineage, and expression content is carried in. A coordinate can configure a view of a directed graph -- which graphs to include, under what STAMP, with what combining rules -- but a coordinate is configuration, never the structure itself.

⊆ Graph

Parents:  GRAPH

Since: Inception

id: DirectedGraph | UUID: 47a787a7-bdce-528d-bfcc-fde1add8d599

Disjoint with

The atom asserting that two concepts share no instances: nothing can be both. Mirrors LogicalAxiom.Atom.DisjointWithAxiom in the sealed hierarchy.

⊆ Atom

Parents:  ATOM

Since: Inception

id: DisjointWith | UUID: f8433993-9a2d-5377-b564-80a45c7b7824

Display Fields

Groups the data types a pattern's field definition can declare: what kind of value a field holds, and therefore which editor and renderer a surface uses for it.

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Children:  COMPONENT ID SET DATA TYPE  DiTREE DATA TYPE  VERTEX DISPLAY FIELD
 IMAGE DISPLAY FIELD  COMPONENT DATA TYPE  CONCEPT DATA TYPE
 COMPONENT ID LIST DATA TYPE  BYTE ARRAY DATA TYPE  DOUBLE DISPLAY FIELD
 INTEGER DATA TYPE  DiGRAPH DATA TYPE  FLOAT DATA TYPE  BOOLEAN DATA TYPE
 LOGICAL EXPRESSION DISPLAY FIELD  STRING DATA TYPE  UUID DISPLAY FIELD
 DECIMAL DATA TYPE  ARRAY DATA TYPE  SEMANTIC DATA TYPE

Since: Inception

id: DisplayFields | UUID: 4e627b9c-cecb-5563-82fc-cb0ee25113b1



Display Sequence

Why a roster order value is recorded: to say where this entry falls in the roster's own display order.

⊞ Editorial model ▢ Purpose

Parents: EDITORIAL MODEL PURPOSE

Since: Inception

id: DisplaySequence | UUID: f40fa97d-81b3-5eb0-9060-190579961b24



DiTree data type

A field that holds a directed tree graph obtained from an undirected tree by replacing each undirected edge with two directed edges of opposite direction.

⊞ Display Fields

Parents:

Since: Inception

id: DiTreeDataType | UUID: 32f64fc6-5371-11eb-ae93-0242ac130002



DiTree default

The DiTree field's loud default: a single-vertex tree whose vertex meaning is Uninitialized Component — the smallest well-formed tree, carrying the loud placeholder at its root.

⊞ Defaults and templates model ▢ Meaning

Parents: DEFAULTS AND TEMPLATES MODEL MEANING

Since: Inception

id: DiTreeDefault | UUID: 48afa790-115b-5bf4-a134-a58d5615dfca



Double

The retired dynamic-column type for double-precision values, superseded by Double display field.

⊞ Dynamic column data types

Parents:

Since: Inception

id: Double | UUID: 7172e6ac-a05a-5a34-8275-aef430b18207

Double display field

The data type for fields holding a double-precision floating-point number.

⌵ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: DoubleDisplayField | UUID: 85ff6e8f-9151-5428-a5f0-e07844b69260

Dutch language

The Dutch language, as a value for a description's language field: a language coordinate that names it selects Dutch-language descriptions for rendering.

⌵ Language

Parents:  LANGUAGE

Since: Inception

id: DutchLanguage | UUID: 21d11bd1-3dab-5034-9625-81b9ae2bd8e7

Dynamic column data types

The retired ISAAC dynamic-semantic column type vocabulary, superseded by the Display Fields data types that field definitions actually declare. Retained for identity compatibility; its members duplicate the live vocabulary one for one.

⌵ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Children:  DECIMAL  SIGNED INTEGER  UUID DATA TYPE  BYTE ARRAY  DOUBLE
 ARRAY  FLOAT  LONG  BOOLEAN

Since: Inception

id: DynamicColumnDataTypes | UUID: 61da7e50-f606-5ba0-a0df-83fd524951e7

**Dynamic referenced component restriction**

An ISAAC-era restriction on a dynamic semantic's referenced component type, retired to Legacy: patterns declare their referenced-component meaning directly.

↳ Legacy model concepts

Parents: [LEGACY MODEL CONCEPTS](#)

Since: Inception

id: `DynamicReferencedComponentRestriction` | UUID: `0d94ceeb-e24f-5f1a-84b2-1ac35f671db5`

**Edge**

A directed connection from one vertex of a graph to another. In the platform's carriers an edge is adjacency — recorded on the vertexes it connects, carrying no properties of its own; what an edge means is given by the meanings of the vertexes it joins.

↳ Graph model

Parents: [GRAPH MODEL](#)

Since: Inception

id: `Edge` | UUID: `9b3a5142-c1db-54a6-b130-0b7290602535`

**Editorial Clarification**

Why a free-text comment is captured: to explain an authoring decision or provide context a formal field cannot, for a future reviewer or maintainer.

↳ Editorial model ▯ Purpose

Parents: [EDITORIAL MODEL](#) [PURPOSE](#)

Since: Inception

id: `EditorialClarification` | UUID: `2cdee396-1e5a-5053-bc5b-4db5c0300930`

**Editorial model**

The meta-schema of editorial workflow: commentary and its subjects, review routing and reviewer assignment, and the author-roster apparatus. These concepts describe how people work on the knowledge base — as distinct from what the knowledge base asserts.

↳ Model concept

Parents:  MODEL CONCEPT

Children:  ROSTER ORDER  STARTER SET AUTHOR ROSTER  COMMENT  KOMET USER LIST
 PREFERRED REVIEWER ASSIGNMENT  ROSTER MEMBERSHIP  ASSIGNED REVIEWER
 REVIEW ROUTING  DISPLAY SEQUENCE  SUBJECT OF COMMENTARY  ROSTER ENTRY
 EDITORIAL CLARIFICATION

Since: Inception

id: EditorialModel | UUID: 6dec60bc-67e5-5af7-8228-bb5161feffee

El profile set operator

Groups the set operators the EL profile admits in definitions: the sufficient and necessary set operators.

⊆ Meaning

Parents:  MEANING

Since: Inception

id: ElProfileSetOperator | UUID: 2352b7a2-11fd-5a68-8ece-fcb3b36570da

EL++ ditree

The directed graph that results from classifying a set of EL++ axioms

⊆ Tree

Parents:  TREE

Since: Inception

id: ELDitree | UUID: ee04d7db-3407-568f-9b93-7b1f9f5bb0fc

EL++ Inferred Axioms Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 INFERRED DEFINITION	 LOGICAL DEFINITION	—

Meaning	Purpose	Data type
<i>Field 1</i>		
 EL++ INFERRED TERMINOLOGICAL AXIOMS	 LOGICAL DEFINITION	 DiTREE DATA TYPE

Since: Inception

id: `ELInferredAxiomsPattern` | UUID: `9f011812-15c9-5b1b-85f8-bb262bc1b2a2`

EL++ Inferred Concept Definition

The inferred definition of a concept in the EL++ profile: the logical expression a reasoner derives from the stated definitions.

⊆ Logical Definition

Parents:  LOGICAL DEFINITION

Since: Inception

id: `ELInferredConceptDefinition` | UUID: `b2897aa0-a697-5bf2-9156-7a437c6a5057`

EL++ Inferred terminological axioms

The inferred side of a concept's EL++ terminological axioms: the definitional axioms a classifier concluded follow from what was stated — computed, never hand-written. The EL++ Inferred Axioms Pattern's one field carries them as a directed tree.

⊆ EL++ terminological axioms ⊧ Meaning

Parents:  EL++ TERMINOLOGICAL AXIOMS  MEANING

Since: Inception

id: `ELInferredTerminologicalAxioms` | UUID: `b6d3be7d-1d7f-5c44-a425-5357f878c212`

EL++ logic profile

The EL++ description-logic profile: the expressivity this set's definitions stay within, chosen because classification remains tractable at terminology scale.

⊆ Description-logic profile

Parents:  DESCRIPTION-LOGIC PROFILE

Since: Inception

id: ELLogicProfile | UUID: 1f201e12-960e-11e5-8994-feff819cdc9f

 EL++ Stated Axioms Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STATED DEFINITION	 LOGICAL DEFINITION	—
<i>e.g.  UNINITIALIZED COMPONENT</i>		
<i>Field 1</i>		
 EL++ STATED TERMINOLOGICAL AXIOMS	 LOGICAL DEFINITION	 DiTREE DATA TYPE
<i>e.g. 4-vertex tree</i>		

Since: Inception

id: ELStatedAxiomsPattern | UUID: e813eb92-7d07-5035-8d43-e81249f5b36e

 EL++ Stated Concept Definition

The stated definition of a concept in the EL++ profile: the logical expression an editor asserts, before classification.

 Logical Definition
Parents:  LOGICAL DEFINITION

Since: Inception

id: ELStatedConceptDefinition | UUID: 0c464a4a-a0bc-53ef-9c01-ef5a049f2656

 EL++ Stated terminological axioms

The stated side of a concept's EL++ terminological axioms: the definitional axioms an author actually wrote — necessary and sufficient sets, role restrictions — as distinct from what a classifier later infers. The EL++ Stated Axioms Pattern's one field carries them as a directed tree.

 EL++ terminological axioms  Meaning
Parents:  EL++ TERMINOLOGICAL AXIOMS  MEANING

Since: Inception

id: `ELStatedTerminologicalAxioms` | UUID: `1412bd09-eb0c-5107-9756-10c1c417d385`

EL++ terminological axioms

The set of relationships or axioms has defined by the EL++ Logic

⊆ Axioms

Parents:  **AXIOMS**

Children:  **EL++ INFERRED TERMINOLOGICAL AXIOMS**  **EL++ STATED TERMINOLOGICAL AXIOMS**

Since: Inception

id: `ELTerminologicalAxioms` | UUID: `b3ec50c4-e8cf-4720-b192-31374705f3b7`

English Dialect

Specifies the dialect of the English language

⊆ Dialect

Parents:  **DIALECT**

Children:  **GREAT BRITAIN ENGLISH DIALECT**  **UNITED STATES OF AMERICA ENGLISH DIALECT**

Since: Inception

id: `EnglishDialect` | UUID: `c0836284-f631-3c86-8cfc-56a67814efab`

English Language

The English language, as a value for a description's language field: a language coordinate that names it selects English-language descriptions for rendering.

⊆ Language

Parents:  **LANGUAGE**

Since: Inception

id: `EnglishLanguage` | UUID: `02018e5a-46ba-5297-92f1-6931b9f98a12`

Equal match relation

A value matches a member when they are equal by the member datatype's natural equality: entity references by identity, scalars by value, byte arrays by content, and Decimal by numeric equality (`compareTo == 0` — an identical-representation rule would be its own relation). Float compares by IEEE semantics, so NaN matches nothing — keeping the loud-defaults coherence: an unrevised NaN default fails its constraint until revised. Graph types are outside this relation's operand conformance — graph enumerations await the isomorphic relations.

⊆ Member match relation

Parents:  MEMBER MATCH RELATION

Since: Inception

id: `EqualMatchRelation` | UUID: `bf47171e-a556-5b87-9e53-23252d12d3f1`

Equal to

The concrete-domain operator asserting that a value matches the stated one exactly.

⊆ Concrete value operator

Parents:  CONCRETE VALUE OPERATOR

Since: Inception

id: `EqualTo` | UUID: `5c9b5844-1434-5111-83d5-cb7cb0be12d9`

Exact

Source and target are semantic or exact lexical match

⊆ Grouping

Parents:  GROUPING

Since: Inception

id: `Exact` | UUID: `8aa6421d-4966-5230-ae5f-aca96ee9c2c1`

Example UCUM Units

The Unified Code for Units of Measure (UCUM) is a code system intended to include all units of measures being contemporarily used in international science, engineering, and business. (www.unitsofmeasure.org) This field contains example units of measures for this term expressed as UCUM units.

⊆ Phenomenon ⊧ Meaning

Parents:  PHENOMENON  MEANING

Since: Inception

id: ExampleUCUMUnits | UUID: 80cd4978-314d-46e3-bc13-9980280ae955

Existential restriction

Existential restrictions describe objects that participate in at least one relationship along a specified property to objects of a specified class.

⊆ Role operator

Parents:  ROLE OPERATOR

Since: Inception

id: ExistentialRestriction | UUID: 91e9080f-78f6-5d23-891d-f5b6e77995c8

Express axiom syntax

Why an axiom-syntax semantic exists: to carry a component's definitional axioms as serialized syntax text, for exchange with systems that read that syntax.

⊆ Axiom Syntax ⊧ Purpose

Parents:  AXIOM SYNTAX  PURPOSE

Since: Inception

id: ExpressAxiomSyntax | UUID: db55557c-e9ee-4504-aae3-df695b6d6c57

Extended description type

Used to store non-snomed description types when other terminologies are imported

⊆ Description type

Parents:  DESCRIPTION TYPE

Since: Inception

id: ExtendedDescriptionType | UUID: 5a2e7786-3e41-11dc-8314-0800200c9a66

Extended relationship type

Used to store non-snomed relationship types when other terminologies are imported- especially when a relationship is mapped onto a snomed relationship type (such as isa)

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: ExtendedRelationshipType | UUID: d41d928f-8a97-55c1-aa6c-a289b413fbfd

External Identity Mapping

Why an Identifier Pattern semantic exists: to record an alternate identifier a component carries in an external terminology's own identifier scheme.

⊆ Identifier model ⊐ Purpose

Parents:  IDENTIFIER MODEL  PURPOSE

Since: Inception

id: ExternalIdentityMapping | UUID: 7aba1c11-4a23-57a1-9512-f1df89165789

Feature

The typed atom asserting a concrete-domain comparison: a feature type, a concrete value operator, and a literal, such as a strength of exactly 500 milligrams. Mirrors LogicalAxiom.Atom.TypedAtom.Feature in the sealed hierarchy.

⊆ Typed atom

Parents:  TYPED ATOM

Since: Inception

id: Feature | UUID: 5e76a88e-794a-5fdd-8eb2-4a9e4b1386b6

Feature Role Type

The role type marking a role as feature-valued: the bridge between role machinery and concrete-domain features.

⊆ Role type

Parents:  ROLE TYPE

Since: Inception

id: `FeatureRoleType` | UUID: `acb8d47e-adac-491d-bc60-78e94cacd312`

Feature Type

The type vocabulary for feature atoms: names which measurable characteristic a feature axiom compares, the concrete-domain counterpart of a role type.

⊆ Logical expression model

Parents:  LOGICAL EXPRESSION MODEL

Since: Inception

id: `FeatureType` | UUID: `c9120d8b-1acc-5267-9f33-fa716abdb69d`

Field categories

Groups the field-level meta-schema concepts: the terminology describing what kind of value — a component, a field definition, a semantic field, or a set of these — a semantic's field can hold.

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Children:  SEMANTIC FIELD FIELD  SEMANTIC FIELD FIELDS SET  FIELD VALUE FIELD

 COMPONENT VERSIONS FIELD  COMPONENT FIELD  FIELD DEFINITIONS SET

 FIELD DEFINITION FIELD  COMPONENT VERSIONS SET  FIELD DEFINITION DATA TYPE FIELD

Since: Inception

id: `FieldCategories` | UUID: `ed230c7c-20f9-470d-8566-5057f92748a5`

Field definition data type field

The field kind naming a field definition's data type slot: which data type a pattern declares for one of its fields.

⊆ Field categories

Parents:  FIELD CATEGORIES

Children:  STRING FIELD  INSTANT FIELD  INTEGER FIELD  BOOLEAN FIELD

Since: Inception

id: `FieldDefinitionDataTypeField` | UUID: `02273b53-fce7-4cbe-921d-2cff67e81ad5`

Field definition field

The field kind naming a pattern version's field-definition slot: each value is one field definition — the meaning, purpose, and data type the pattern declares for one of its fields.

⊆ Field categories ⊐ Meaning

Parents:  FIELD CATEGORIES  MEANING

Since: Inception

id: FieldDefinitionField | UUID: 14171f07-e74f-409a-b555-06b478818f76

Field definition meaning field

The field kind naming a field definition's meaning slot: the concept saying what the declared field is.

⊆ Concept field

Parents:  CONCEPT FIELD

Since: Inception

id: FieldDefinitionMeaningField | UUID: 74dffbed-0bef-44a4-8ad6-8cff84fe47ae

Field definition purpose field

The field kind naming a field definition's purpose slot: the concept saying why the declared field is recorded.

⊆ Concept field

Parents:  CONCEPT FIELD

Since: Inception

id: FieldDefinitionPurposeField | UUID: 93239959-50e6-4645-b5fc-6d47da92e666

Field definitions set

Why a pattern version's field-definition field is recorded: to hold the pattern's complete, ordered field definitions — the declared shape every semantic of the pattern must fit.

⊆ Field categories ⊐ Purpose

Parents:  FIELD CATEGORIES  PURPOSE

Since: Inception

id: FieldDefinitionsSet | UUID: 975de83e-ab99-4a9e-9051-4cbf310a2123

Field substitution

Replacing a placeholder variable in a field with a specific value

≡ Meaning

Parents:  MEANING

Children:  INSTANT SUBSTITUTION  FLOAT SUBSTITUTION  CONCEPT SUBSTITUTION
 BOOLEAN SUBSTITUTION

Since: Inception

id: FieldSubstitution | UUID: 8fdce1aa-ca82-5abc-8cfa-230c14688abc

Field value field

The field kind for one field value a semantic version actually carries, as distinct from the field definition its pattern declares.

≡ Field categories

Parents:  FIELD CATEGORIES

Since: Inception

id: FieldValueField | UUID: 7e4a96fc-0522-4d74-a7d1-ca74be3bc236

Field Value Restriction

Why a field constraint semantic exists: to restrict which values are legal for one of the constrained pattern's own fields.

≡ Constraint model ▢ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: FieldValueRestriction | UUID: c7e33912-1f1c-5821-9c88-4b0a4504c028

Float

The retired dynamic-column type for floating-point values, superseded by Float data type.

Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: Float | UUID: fb591801-7b37-525d-980d-98a1c63ceee0

Float data type

Represents values as high-precision fractional values.

Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: FloatDataType | UUID: 6efe7087-3e3c-5b45-8109-90d7652b1506

Float default

The Float field's loud default: NaN, the type's native non-value — chosen over a stretched-sevens sentinel because a true non-value is louder still: it cannot be mistaken for data at all, and it propagates through arithmetic, so any computation over an unrevised default stays loudly not-a-number.

Defaults and templates model Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: FloatDefault | UUID: 8ce26a3b-46fc-563b-853d-ccb5ee9183e7

Float literal

Numbers with decimal point or an exponential part

Literal value

Parents:  LITERAL VALUE

Since: Inception

id: FloatLiteral | UUID: da754dd9-9961-5819-91f5-8245d49850b4

Float substitution

A field substitution whose substituted value is a float; part of the retired ISAAC field-substitution family.

⊞ Field substitution

Parents:  FIELD SUBSTITUTION

Since: Inception

id: FloatSubstitution | UUID: cf18fe25-bd21-586e-9da4-da6cb335fd12

French dialect

The French dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in French usage; a language coordinate's dialect order consults it when choosing what to show.

⊞ Dialect

Parents:  DIALECT

Since: Inception

id: FrenchDialect | UUID: 75d00a0d-8e46-5e42-ad34-3e46269b28a3

French Language

The French language, as a value for a description's language field: a language coordinate that names it selects French-language descriptions for rendering.

⊞ Language

Parents:  LANGUAGE

Since: Inception

id: FrenchLanguage | UUID: 8b23e636-a0bd-30fb-b8e2-1f77eaa3a87e

Fully qualified name description type

Fully qualified name is a description that uniquely identifies and differentiates it from other concepts with similar descriptions

⊞ Description type

Parents:  DESCRIPTION TYPE

Since: Inception

id: FullyQualifiedNameDescriptionType | UUID: 00791270-77c9-32b6-b34f-d932569bd2bf



GB Dialect Pattern

Particular form of language specific form of English language, particular to Great Britain.

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
DESCRIPTION ACCEPTABILITY	DESCRIPTION SEMANTIC	—
<i>Field 1</i>		
GREAT BRITAIN ENGLISH DIALECT	DESCRIPTION ACCEPTABILITY	COMPONENT DATA TYPE

Since: Inception

id: GBDialectPattern | UUID: 561f817a-130e-5e56-984d-910e9991558c



German Language

The German language, as a value for a description's language field: a language coordinate that names it selects German-language descriptions for rendering.

≡ Language

Parents: LANGUAGE

Since: Inception

id: GermanLanguage | UUID: 5f144b18-76a8-5c7e-8480-55a5030d707f



Graph

A set of vertexes and the edges that connect them — the representation logical expressions, navigation structures, and lineage records are all carried in. A graph as carried here is directed: each edge runs from one vertex to another, and processing follows that direction.

≡ Graph model

Parents: GRAPH MODEL

Children:  DIRECTED GRAPH  TREE

Since: Inception

id: Graph | UUID: da454dbd-ed6e-55cf-af5a-0d51b40d7640

Graph model

The meta-schema of graph representation and processing: vertexes that carry a meaning and properties, edges that connect them, graphs assembled from both, and trees as the single-parent special case. Logical expressions, navigation structures, and version-control lineage are all carried as graphs; this subsystem names the representation those carriers share.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  VERTEX MEANING  GRAPH  VERTEX  EDGE  VERTEX PROPERTY

Since: Inception

id: GraphModel | UUID: c99b70a6-e563-5c8a-b3d9-5f599d0ee9bd

Great Britain English dialect

Great Britain: English Language reference set

⊆ English Dialect ⊆ Meaning

Parents:  ENGLISH DIALECT  MEANING

Since: Inception

id: GreatBritainEnglishDialect | UUID: eb9a5e42-3cba-356d-b623-3ed472e20b30

Greater than

The concrete-domain operator asserting that a value is strictly above the stated bound.

⊆ Concrete value operator

Parents:  CONCRETE VALUE OPERATOR

Since: Inception

id: GreaterThan | UUID: 65af466b-360c-58b2-8b7d-2854150029a8

Greater than or equal to

The concrete-domain operator asserting that a value is at least the stated bound.

⊆ Concrete value operator

Parents:  CONCRETE VALUE OPERATOR

Since: Inception

id: GreaterThanOrEqualTo | UUID: c1baba19-e918-5d2c-8fa4-b0ad93e03186

Gretel

The default editing identity a fresh Komet installation offers, so a user can author before establishing their own author concept.

⊆ Author

Parents:  AUTHOR

Since: Inception

id: Gretel | UUID: 1c0023ed-559e-3311-9e55-bd4bd9e5628f

Grouping

Groups the correlation match markers: whether an expression correlation is exact or partial.

⊆ Tinkar Model concept

Parents:  TINKAR MODEL CONCEPT

Children:  PARTIAL  EXACT

Since: Inception

id: Grouping | UUID: 8d76ead7-6c75-5d25-84d4-ca76d928f8a6

Has Active Ingredient

A domain role type from the medication model: relates a product to an ingredient active in it. Held as a SNOMED-heritage example; migrates to a domain starter set when one exists.

⊆ Role type

Parents:  ROLE TYPE

Since: Inception

id: HasActiveIngredient | UUID: 65bf3b7f-c854-36b5-81c3-4915461020a8

Has Dose Form

A domain role type from the medication model: relates a product to its dose form. Held as a SNOMED-heritage example; migrates to a domain starter set when one exists.

⊆ Role type

Parents:  **ROLE TYPE**

Since: Inception

id: HasDoseForm | UUID: 072e7737-e22e-36b5-89d2-4815f0529c63

Health concept

The domain anchor for health-related content under Phenomenon. Held as a SNOMED-heritage example; domain content grows in domain starter sets.

⊆ Phenomenon

Parents:  **PHENOMENON**

Since: Inception

id: HealthConcept | UUID: a892950a-0847-300c-b477-4e3cbb945225

Identified Component

What an Identifier Pattern semantic's referenced component is: the component an external identifier names — the carrier of an alternate identity in another identifier scheme, distinct from Identifier source, which names the field value saying where that identifier came from (IKE-Network/ike-issues#891).

⊆ Identifier model ⊆ Meaning

Parents:  **IDENTIFIER MODEL**  **MEANING**

Since: Inception

id: IdentifiedComponent | UUID: ab2c34b1-7194-5245-ae7a-fde89830442b

Identifier Authority

Why an identifier source value is recorded: to say which identifier-issuing authority the identifier came from.

⊞ Identifier model ⊞ Purpose

Parents:  IDENTIFIER MODEL  PURPOSE

Since: Inception

id: IdentifierAuthority | UUID: 120121c5-5b15-5ffb-9e8a-6cf39d2a2acd

Identifier model

The meta-schema of identity interchange: how a component carries identifiers minted by external authorities, which authority stands behind each identifier, and how an external identity maps onto a knowledge-base component. Distinct from public identity itself (a chronicle-model concern): this subsystem is about the identifiers other systems know a component by.

⊞ Model concept

Parents:  MODEL CONCEPT

Children:  IDENTIFIER AUTHORITY  IDENTIFIER TEXT  EXTERNAL IDENTITY MAPPING  NID
 IDENTIFIED COMPONENT

Since: Inception

id: IdentifierModel | UUID: ca825cab-4ea6-5ef2-824c-3ca40a98bf72

Identifier Pattern

An identifier pattern is used to identity a concept which contains the identifier source and the actual value.

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 IDENTIFIED COMPONENT	 EXTERNAL IDENTITY MAPPING	—
<i>e.g.  UNINITIALIZED COMPONENT</i>		
<i>Field 1</i>		

Meaning	Purpose	Data type
 IDENTIFIER SOURCE	 IDENTIFIER AUTHORITY	 COMPONENT DATA TYPE
<i>e.g.</i>  <i>UNIVERSALLY UNIQUE IDENTIFIER</i>		
<i>Field 2</i>		
 IDENTIFIER VALUE	 IDENTIFIER TEXT	 STRING DATA TYPE
<i>e.g.</i> 55f74246-0a25-57ac-9473-a788d08fb656		

Since: Inception

id: IdentifierPattern | UUID: 5d60e14b-c410-5172-9559-3c4253278ae2

Identifier Source

An identifier used to label the identity of a unique component.

↳ Tinkar Model concept ▢ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Children:  UNIVERSALLY UNIQUE IDENTIFIER

Since: Inception

id: IdentifierSource | UUID: 5a87935c-d654-548f-82a2-0c06e3801162

Identifier Text

Why an identifier value is recorded: to hold the identifier's own literal text, once a source has been chosen.

↳ Identifier model ▢ Purpose

Parents:  IDENTIFIER MODEL  PURPOSE

Since: Inception

id: IdentifierText | UUID: a3f37520-ca20-5bf0-bb48-ce4892160233

Identifier Value

The literal text of an identifier, once an issuing source has been named -- the value side of an identifier semantic.

⊞ Tinkar Model concept ⊞ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: IdentifierValue | UUID: b32dd26b-c3fc-487e-987e-16ace71a0d0f

IKE Community

Community authorship: the IKE Network itself, attributed as author for content synthesized by tooling on the Network's behalf (e.g. the identity-exact starter-set ingest, IKE-Network/ike-issues#872) rather than by an individual editor or an ingested upstream source.

⊞ Author

Parents:  AUTHOR

Since: Inception

id: IKECommunity | UUID: a03bedb4-f505-5153-a213-7581b651c929

IkeFoundation module

The module scoping every stamp of the IkeFoundation content set; the export dimension for this knowledge.

⊞ Module

Parents:  MODULE

Since: Inception

id: IkeFoundationModule | UUID: 61615e8f-173a-5875-937e-51f1fa7bbadb

IkeFoundation root

Root concept of the Ike starter set.

⊞ Model concept

Parents:  MODEL CONCEPT

Since: Inception

id: IkeFoundationRoot | UUID: 391b61a5-f53c-513b-acfd-bfa0100db7d8

Image display field

The data type for fields holding an image, encoded as the bytes of a renderable image format.

⊆ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: ImageDisplayField | UUID: cd9ea037-0af9-586b-9369-7bc044cdb8f7

Immediate child field constraint

Legal values are the anchor concept's direct children only, not deeper descendants.

⊆ Taxonomy field constraint kind

Parents:  TAXONOMY FIELD CONSTRAINT KIND

Since: Inception

id: ImmediateChildFieldConstraint | UUID: 97f24f21-0210-5a00-98cd-2efb76d1f4cc

ImmutableCoordinate Properties

Groups the dimension concepts every coordinate kind is assembled from -- stamp, language, logic, path, edit, and navigation -- named individually so a view's configuration is itself describable knowledge. Coordinates are immutable: a changed dimension yields a new coordinate, never a mutated one.

⊆ View coordinate model

Parents:  VIEW COORDINATE MODEL

Children:  MODULE OPTIONS FOR EDIT COORDINATE  ROOT FOR LOGIC COORDINATE
 STATED PATTERN FOR LOGIC COORDINATE  DIGRAPH FOR LOGIC COORDINATE
 DESCRIPTION LOGIC PROFILE FOR LOGIC COORDINATE  AUTHOR FOR EDIT COORDINATE
 VERTEX SORT  PATH FOR PATH COORDINATE  LANGUAGE NID FOR LANGUAGE COORDINATE
 INFERRED PATTERN FOR LOGIC COORDINATE  CONCEPT PATTERN FOR LOGIC COORDINATE
 DESTINATION MODULE FOR EDIT COORDINATE  MODULE PREFERENCE LIST FOR STAMP COORDINATE
 VERTEX STATE SET  MODULES FOR STAMP COORDINATE  PATH OPTIONS FOR EDIT COORDINATE
 MODULE EXCLUSION SET FOR STAMP COORDINATE  AUTHOR FOR STAMP COORDINATE
 DESCRIPTION TYPE PREFERENCE LIST FOR LANGUAGE COORDINATE
 LANGUAGE SPECIFICATION FOR LANGUAGE COORDINATE  ALLOWED STATES FOR STAMP COORDINATE

 MODULE PREFERENCE LIST FOR LANGUAGE COORDINATE
 MODULE PREFERENCE ORDER FOR STAMP COORDINATE  PATH ORIGINS FOR STAMP PATH
 CLASSIFIER FOR LOGIC COORDINATE  DEFAULT MODULE FOR EDIT COORDINATE  POSITION ON PATH

Since: Inception

id: `ImmutableCoordinateProperties` | UUID: `ab41a788-8a83-5452-8dc0-2d8375e0bfe6`

Implication set

A baseline grouping for implication axioms, retained for identity compatibility: property-sequence implications are the atoms it grouped.

⊆ Logical set

Parents:  LOGICAL SET

Since: Inception

id: `ImplicationSet` | UUID: `ee467a5b-9292-4e0a-a165-3b1a359a8c98`

Inactive state

Concept used to represent a status for components that are no longer active

⊆ Status value

Parents:  STATUS VALUE

Since: Inception

id: `InactiveState` | UUID: `03004053-c23e-5206-8514-fb551dd328f4`

Include Lower Bound

The field saying whether an interval includes its lower bound: closed when included, open when not.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: `IncludeLowerBound` | UUID: `2300a210-d722-48af-8c36-118a3f980312`

Include Upper Bound

The field saying whether an interval includes its upper bound: closed when included, open when not.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: IncludeUpperBound | UUID: 990b7e1d-3dcc-4c6e-a068-e30400607d50

Inclusion set

A set of relationships that indicate something is has an inclusion. Not necessarily or sufficient but inclusive.

⊆ Logical set

Parents:  LOGICAL SET

Since: Inception

id: InclusionSet | UUID: def77c09-e1eb-40f2-931a-e7cf2ce0e597

Inferred Definition

The relationships/axioms of a concept that have been inferred

⊆ Tinkar Model concept ⊧ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: InferredDefinition | UUID: b1abf4dc-9838-4b46-ac55-10c4f92ba10b

Inferred navigation

Navigation computed by the reasoner from inferred definitions: the classified parent and child adjacency navigators render.

⊆ Navigation

Parents:  NAVIGATION

Since: Inception

id: InferredNavigation | UUID: 4bc6c333-7fc9-52f1-942d-f8decba19dc2

Inferred Navigation Pattern

A pattern specifying the origins and destinations for concepts based on the inferred terminological axioms.

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 IS-A	 TAXONOMY NAVIGATION CACHE	—
<i>Field 1</i>		
 RELATIONSHIP DESTINATION	 IS-A	 COMPONENT ID SET DATA TYPE
<i>Field 2</i>		
 RELATIONSHIP ORIGIN	 IS-A	 COMPONENT ID SET DATA TYPE

Since: Inception

id: InferredNavigationPattern | UUID: a53cc42d-c07e-5934-96b3-2ede3264474e

Inferred pattern for logic coordinate

The pattern the classifier's derived axioms are written to, for a Logic Coordinate.

⊆ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: InferredPatternForLogicCoordinate | UUID: 9ecf4d76-4346-5e5d-8316-bdff48a5c154

Inferred premise type

The axiom view following the application of the reasoner

⊆ Axiom origin

Parents:  AXIOM ORIGIN

Since: Inception

id: InferredPremiseType | UUID: a4c6bf72-8fb6-11db-b606-0800200c9a66

Instant default

The Instant field's loud default: the pre-inception instant (historically 'premundane'), the time type's native non-value — the platform's own before-all-recorded-time marker, chosen over a sentinel date because a true non-value cannot be mistaken for a real timestamp.

⊆ Defaults and templates model ▢ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: InstantDefault | UUID: 4f18ebf5-f890-571c-a2e1-1432efc95988

Instant field

The field kind for instant-valued fields: a pattern field holding a point in time.

⊆ Field definition data type field

Parents:  FIELD DEFINITION DATA TYPE FIELD

Children:  TIME FIELD

Since: Inception

id: InstantField | UUID: e9bde1bc-aa72-430a-afe1-aa8aec8833b4

Instant literal

May refer to a specific point in time which is often represented by a date or time value

⊆ Literal value

Parents:  LITERAL VALUE

Since: Inception

id: InstantLiteral | UUID: 1fbf42e2-42b7-591f-b7fd-ba5de659529e

Instant substitution

A field substitution whose substituted value is an instant; part of the retired ISAAC field-substitution family.

⊆ Field substitution

Parents:  [FIELD SUBSTITUTION](#)

Since: Inception

id: `InstantSubstitution` | UUID: `56599345-31c5-5817-9d36-57dd3a626b3a`

Integer data type

Data type that represents some range of mathematical integers

⊞ Display Fields

Parents:  [DISPLAY FIELDS](#)

Since: Inception

id: `IntegerDataType` | UUID: `ff59c300-9c4e-5e77-a35d-6a133eb3440f`

Integer default

The Integer field's loud default: the stretched-sevens sentinel 77777777 (nine sevens) — a repdigit signature that reads as deliberately planted and cannot plausibly occur as natural data, where `Integer.MIN_VALUE` would read as an overflow bug. Never Java's silent zero.

⊞ Defaults and templates model ▯ Meaning

Parents:  [DEFAULTS AND TEMPLATES MODEL](#)  [MEANING](#)

Since: Inception

id: `IntegerDefault` | UUID: `11f902b5-7c46-5157-9cfd-289d3019a559`

Integer field

The field kind for integer-valued fields: a pattern field holding a whole number.

⊞ Field definition data type field

Parents:  [FIELD DEFINITION DATA TYPE FIELD](#)

Since: Inception

id: `IntegerField` | UUID: `db249d1f-ea2e-4608-ae13-166ed20ca825`

Integrated Knowledge Management

Terminologies that are represented in a harmonized manner

⊆ Integrated Knowledge Management

Parents:  INTEGRATED KNOWLEDGE MANAGEMENT

Children:  MODEL CONCEPT  PHENOMENON  INTEGRATED KNOWLEDGE MANAGEMENT

Since: Inception

id: IntegratedKnowledgeManagement | UUID: 7c21b6c5-cf11-5af9-893b-743f004c97f5

Interval Lower Bound

The field holding an interval's lower bound value.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: IntervalLowerBound | UUID: 52b3e38a-fccb-4779-aa61-4e87abd56419

Interval property set

The logical set grouping interval-property axioms under a definition root. Mirrors LogicalAxiom.LogicalSet.IntervalPropertySet in the sealed hierarchy.

⊆ Logical set

Parents:  LOGICAL SET

Since: Inception

id: IntervalPropertySet | UUID: 9afc988a-3724-4754-8b74-651426472b19

Interval role

The typed atom asserting a role bounded to an interval: a role whose filler is constrained by interval bounds. Mirrors LogicalAxiom.Atom.TypedAtom.IntervalRole in the sealed hierarchy.

⊆ Typed atom

Parents:  TYPED ATOM

Since: Inception

id: IntervalRole | UUID: ed9d3506-65ad-48ea-bd01-95474fecdbc4

Interval Role Type

The role-type vocabulary for interval roles: names which relation an interval role asserts.

⊆ Role type

Parents:  ROLE TYPE

Since: Inception

id: IntervalRoleType | UUID: 6fa58611-af37-402e-a0c2-6ee1d6068651

Interval Set Axioms

Groups the interval-axiom vocabulary: the bound values, openness markers, and unit of measure an interval-valued assertion carries.

⊆ Axioms

Parents:  AXIOMS

Children:  UNIT OF MEASURE  LOWER BOUND OPEN  INTERVAL TYPE  UPPER BOUND OPEN
 INCLUDE UPPER BOUND  INTERVAL UPPER BOUND  INTERVAL LOWER BOUND
 INCLUDE LOWER BOUND

Since: Inception

id: IntervalSetAxioms | UUID: b253e725-d7cd-46e3-bc3a-5db8b3ffbd52

Interval Type

The concept naming which kind of interval an interval assertion carries.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: IntervalType | UUID: ba3191ee-a260-41a6-99fd-74a22fdc937e

Interval Upper Bound

The field holding an interval's upper bound value.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: IntervalUpperBound | UUID: 6565f774-ff6c-4882-832f-31ddc462adf7

Intrinsic role

An ISAAC-era marker for roles intrinsic to a concept, retired to Legacy.

⊆ Legacy model concepts

Parents: LEGACY MODEL CONCEPTS

Since: Inception

id: IntrinsicRole | UUID: a2d37d2d-ac49-589f-ba36-ac9b8808b00b

Inverse name

This is the extended description type that may be attached to a description within a concept that defines as Association reflex to signify that the referenced description is the inverse of the association name

⊆ Description type

Parents: DESCRIPTION TYPE

Since: Inception

id: InverseName | UUID: c342d18a-ec1c-5583-bfe3-59e6324ae189

Inverse tree list

The legacy tree-amalgam field holding a navigation tree's child-to-parent adjacency, paired with Tree list.

⊆ Tree amalgam properties

Parents: TREE AMALGAM PROPERTIES

Since: Inception

id: InverseTreeList | UUID: 45167fb6-e01d-53a8-8be3-768aae18729d

Irish dialect

The Irish dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in Irish usage; a language coordinate's dialect order consults it when choosing what to show.

⊆ Dialect

Parents:  DIALECT

Since: Inception

id: IrishDialect | UUID: c0f77638-6274-5b40-b832-ac1cba7ec515

Irish language

The Irish language, as a value for a description's language field: a language coordinate that names it selects Irish-language descriptions for rendering.

⊆ Language

Parents:  LANGUAGE

Since: Inception

id: IrishLanguage | UUID: 58e82fc4-1492-5cf8-8997-43800360bbd6

Is-a

The subtype relation: everything the child names is also what the parent names. The relation taxonomy navigation is computed from, and the only axiom shape the ingested foundation uses.

⊆ Tinkar Model concept ⊧ Meaning ⊧ Purpose

Parents:  TINKAR MODEL CONCEPT  MEANING  PURPOSE

Since: Inception

id: IsA | UUID: c93a30b9-ba77-3adb-a9b8-4589c9f8fb25

Is-a inferred navigation

Designates the parent child relationship following the application of the reasoner

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: IsAInferredNavigation | UUID: b620768f-1479-5afa-a027-5a9ae6caf0d5

**Is-a stated navigation**

Designates the parent child relationship as authored

↳ Legacy model concepts

Parents:  [LEGACY MODEL CONCEPTS](#)

Since: Inception

id: `IsAStatedNavigation` | UUID: `d555dde9-c97e-5480-819a-7472eda3dbfa`

**Italian Language**

The Italian language, as a value for a description's language field: a language coordinate that names it selects Italian-language descriptions for rendering.

↳ Language

Parents:  [LANGUAGE](#)

Since: Inception

id: `ItalianLanguage` | UUID: `bdd59458-381a-5818-8577-60525f11ac6c`

**Kind-of field constraint**

Legal values are the anchor concept itself plus every one of its descendants (self included) — Tinkar's own "kind of" relation.

↳ Taxonomy field constraint kind

Parents:  [TAXONOMY FIELD CONSTRAINT KIND](#)

Since: Inception

id: `KindOfFieldConstraint` | UUID: `127975b1-8ca7-50f0-aea5-bd69c47682fd`

**Komet base model component pattern**

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STARTER DATA AUTHORIZING	 SET MEMBERSHIP	—

Meaning	Purpose	Data type
e.g.  <i>KOMET BASE MODEL COMPONENT PATTERN</i>		

Since: Inception

id: KometBaseModelComponentPattern | UUID: bbbbf1fe-00f0-55e0-a19c-6300dbaab9b2

Komet issue

Komet being the 'annotation type' - specified type

⊞ Legacy model concepts

Parents:  *LEGACY MODEL CONCEPTS*

Since: Inception

id: KometIssue | UUID: e1dd7bf2-224d-53a5-a5fb-7b25b05d17a6

KOMET module

The module for content the Komet application itself defines, such as its window and layout terminology.

⊞ Module

Parents:  *MODULE*

Since: Inception

id: KOMETModule | UUID: 34a6dae3-e5e9-50db-a9ee-69c1067911d8

KOMET user

Authorized to author, edit and/or view in Komet

⊞ Author

Parents:  *AUTHOR*

Since: Inception

id: KOMETUser | UUID: 61c1a544-2acf-58cd-8cc0-9ac581d4227e

**KOMET user list**

The roster of author identities a Komet installation offers at sign-in.

⊞ Editorial model

Parents: EDITORIAL MODEL

Since: Inception

id: KOMETUserList | UUID: 5e77558d-97d0-52b6-adf0-d54beb97b3a6

**Korean dialect**

The Korean dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in Korean usage; a language coordinate's dialect order consults it when choosing what to show.

⊞ Dialect

Parents: DIALECT

Children: STANDARD KOREAN DIALECT

Since: Inception

id: KoreanDialect | UUID: 6fb2eb9c-fb9e-5959-802c-fb17bcba3079

**Korean Language**

The Korean language, as a value for a description's language field: a language coordinate that names it selects Korean-language descriptions for rendering.

⊞ Language

Parents: LANGUAGE

Since: Inception

id: KoreanLanguage | UUID: 1464f995-d658-5e9d-86e0-8308a6fa57eb

**Language**

Specifies the language of the description text.

⊞ Tinkar Model concept ⊞ Purpose

Parents: TINKAR MODEL CONCEPT PURPOSE

Children:  ENGLISH LANGUAGE  RUSSIAN LANGUAGE  POLISH LANGUAGE  CZECH LANGUAGE
 CHINESE LANGUAGE  LITHUANIAN LANGUAGE  ITALIAN LANGUAGE  SWEDISH LANGUAGE
 GERMAN LANGUAGE  DANISH LANGUAGE  FRENCH LANGUAGE  SPANISH LANGUAGE
 DUTCH LANGUAGE  KOREAN LANGUAGE  IRISH LANGUAGE

Since: Inception

id: Language | UUID: f56fa231-10f9-5e7f-a86d-a1d61b5b56e3

Language coordinate name

The field naming a stored language coordinate, so a surface can present saved coordinates by name.

≡ Language coordinate properties

Parents:  LANGUAGE COORDINATE PROPERTIES

Since: Inception

id: LanguageCoordinateName | UUID: 42dff20f-5ed2-559a-91ad-91d44a573c63

Language coordinate properties

Groups the language-coordinate dimensions: which language, which dialect order, which description types in which preference order, and which modules a rendering surface consults when choosing text to show.

≡ View coordinate model

Parents:  VIEW COORDINATE MODEL

Children:  DIALECT PATTERN PREFERENCE LIST FOR LANGUAGE COORDINATE

 LANGUAGE COORDINATE NAME

Since: Inception

id: LanguageCoordinateProperties | UUID: ea1a52f7-0305-5487-8766-e846330f167a

Language for description

Captures the language code for a description

≡ Description version properties ▯ Meaning

Parents:  DESCRIPTION VERSION PROPERTIES  MEANING

Since: Inception

id: `LanguageForDescription` | UUID: `cd56cceb-8507-5ae5-a928-16079fe6f832`

Language nid for language coordinate

The language-coordinate dimension naming the language whose descriptions the coordinate selects.

⊆ `ImmutableCoordinate Properties`

Parents:  `IMMUTABLECOORDINATE PROPERTIES`

Since: Inception

id: `LanguageNidForLanguageCoordinate` | UUID: `38e0c7b8-1e33-56a2-9eb2-ee20c4960684`

Language specification for language coordinate

The language-coordinate dimension carrying the coordinate's full language specification as one value: language plus dialect and description-type preferences.

⊆ `ImmutableCoordinate Properties`

Parents:  `IMMUTABLECOORDINATE PROPERTIES`

Since: Inception

id: `LanguageSpecificationForLanguageCoordinate` | UUID: `b0ad4d77-e1bc-5fd1-922e-5fad675e9bfd`

Laterality

A domain role type from the anatomy model: distinguishes sided body structures, left from right. Held as a SNOMED-heritage example; migrates to a domain starter set when one exists.

⊆ `Role type`

Parents:  `ROLE TYPE`

Since: Inception

id: `Laterality` | UUID: `26ca4590-bbe5-327c-a40a-ba56dc86996b`

Leaf descendant field constraint

Legal values are the anchor concept's leaf descendants only — descendants with no children of their own — excluding grouping concepts that exist only to organize other concepts.

⊢ Taxonomy field constraint kind

Parents:  TAXONOMY FIELD CONSTRAINT KIND

Since: Inception

id: LeafDescendantFieldConstraint | UUID: 88c5f3a3-2932-5e45-ab7e-3b0bdd9a5233

Legacy model concepts

A visible deprecation signal for model machinery, not a functional restriction: content reparented here is dormant or superseded and a candidate for removal in a near-future revision. Being here says nothing about whether the content is still resolvable or referenced elsewhere — it may well be — only that it is no longer this set's preferred terminology going forward. The scope is deliberate: machinery deprecates in place, where its history stays browsable; domain content that does not belong to this set migrates to its home starter set instead, and is never marked legacy here.

⊢ Model concept

Parents:  MODEL CONCEPT

Children:  TREE AMALGAM PROPERTIES  SANDBOX COMPONENT
 REFERENCED COMPONENT FOR SEMANTIC  COMPONENT FOR SEMANTIC  ANNOTATION TYPE
 PROPERTY PATTERN IMPLICATION  EXTENDED RELATIONSHIP TYPE  IS-A STATED NAVIGATION
 ACTION PROPERTIES  UNMODELED ROLE CONCEPT  CORRELATION PROPERTIES
 DYNAMIC REFERENCED COMPONENT RESTRICTION  KOMET ISSUE  OBJECT
 DYNAMIC COLUMN DATA TYPES  OBJECT PROPERTIES  IS-A INFERRED NAVIGATION
 INTRINSIC ROLE

Since: Inception

id: LegacyModelConcepts | UUID: 7fd61405-6776-52e6-9e4e-3d21dfdf28ad

Legal Value Source

Why the Value-set pattern value is recorded: to name the pattern whose active semantics enumerate the legal values for a value-set constraint.

⊢ Constraint model ▯ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: LegalValueSource | UUID: 97153d1a-91d8-5a53-8cb4-69c003d41526

**Less than**

The concrete-domain operator asserting that a value is strictly below the stated bound.

⊆ Concrete value operator

Parents: CONCRETE VALUE OPERATOR

Since: Inception

id: LessThan | UUID: 6f96e8cf-5568-5e49-8a90-aa6c65125ee9

**Less than or equal to**

The concrete-domain operator asserting that a value is at most the stated bound.

⊆ Concrete value operator

Parents: CONCRETE VALUE OPERATOR

Since: Inception

id: LessThanOrEqualTo | UUID: 6dfacbd5-8344-5794-9fda-bec95b2aa6c9

**Literal value**

Groups the literal kinds a concrete-domain axiom can compare against: a fixed value written into the expression itself, rather than a reference to a component.

⊆ Meaning

Parents: MEANING

Children: INSTANT LITERAL FLOAT LITERAL BOOLEAN LITERAL

Since: Inception

id: LiteralValue | UUID: 208a40a7-e615-5efa-9de0-2e2a5a8488b7

**Lithuanian language**

The Lithuanian language, as a value for a description's language field: a language coordinate that names it selects Lithuanian-language descriptions for rendering.

⊆ Language

Parents: LANGUAGE

Since: Inception

id: `LithuanianLanguage` | UUID: `8fa63274-70e3-5b11-9669-1b7bdb372b1a`

Logic coordinate name

The field naming a stored logic coordinate, so a surface can present saved coordinates by name.

⊞ Logic coordinate properties

Parents:  [LOGIC COORDINATE PROPERTIES](#)

Since: Inception

id: `LogicCoordinateName` | UUID: `78972f14-e0f6-5f72-bf82-59310b5f7b26`

Logic coordinate properties

Groups the logic-coordinate dimensions: which reasoner classifies, which description-logic profile it assumes, which patterns carry stated and inferred axioms, and which root it classifies beneath.

⊞ View coordinate model

Parents:  [VIEW COORDINATE MODEL](#)

Children:  [LOGIC COORDINATE NAME](#)

Since: Inception

id: `LogicCoordinateProperties` | UUID: `1fa63819-5ac1-5938-95b1-47871a5f2b17`

Logic graph for semantic

The field holding a semantic's logical expression graph: the definition a logical-expression semantic carries.

⊞ Semantic properties

Parents:  [SEMANTIC PROPERTIES](#)

Since: Inception

id: `LogicGraphForSemantic` | UUID: `fc2a0662-2396-575b-95f0-e9b38a418620`

Logical axiom

The root of the logical-axiom taxonomy: one element of a stated or inferred definition's expression graph. Mirrors the sealed interface LogicalAxiom: every axiom is an atom, a definition root, or a logical set — a closed alternative the compiler enforces in code and this taxonomy states as knowledge.

⊆ Logical expression model

Parents:  LOGICAL EXPRESSION MODEL

Children:  LOGICAL SET  ATOM  DEFINITION ROOT

Since: Inception

id: LogicalAxiom | UUID: 9f622a2f-ed85-57a1-a1b0-c448127fb906

Logical Definition

The semantic value describing the purpose of the stated and inferred terminological axioms.

⊆ Tinkar Model concept ∩ Purpose

Parents:  TINKAR MODEL CONCEPT  PURPOSE

Children:  EL++ STATED CONCEPT DEFINITION  EL++ INFERRED CONCEPT DEFINITION

Since: Inception

id: LogicalDefinition | UUID: 7dccd042-b0b8-5cec-a1bc-6de676b92f4b

Logical expression display field

The data type for fields holding a logical expression: a definition graph carried as a DiTree.

⊆ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: LogicalExpressionDisplayField | UUID: c16eb414-8840-54f8-9bd2-e2f1ab37e19d

Logical expression model

The meta-schema of stated and inferred definitions: the closed taxonomy of logical-axiom kinds — mirroring, one for one, the sealed LogicalAxiom hierarchy the platform's reasoner and builders compile against — together with the operator vocabularies those axioms use. What the compiler enforces as the closed set of axiom kinds, this subtree states as the closed taxonomy of axiom-kind concepts.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  REFLEXIVE FEATURE  AXIOM EXPRESSION  ROLE OPERATOR
 ANNOTATION PROPERTY SET  TRANSITIVE FEATURE  PROPERTY SEQUENCE  LOGICAL AXIOM
 AXIOMATIZED COMPONENT  FEATURE TYPE

Since: Inception

id: LogicalExpressionModel | UUID: 048b509b-7446-5adb-a81b-245386981760

Logical expression semantic

The semantic type whose field value is a logical expression: a definition graph carried as a DiTree.

⊆ Semantic type

Parents:  SEMANTIC TYPE

Since: Inception

id: LogicalExpressionSemantic | UUID: d19306b1-4744-5028-a715-17ca4a4d657f

Logical set

A logical axiom that groups atoms under a definition root and fixes their force: necessary (always true of the concept), sufficient (enough to classify an individual as the concept), property, data property, interval property, and inclusion sets. Mirrors LogicalAxiom.LogicalSet in the sealed hierarchy.

⊆ Logical axiom

Parents:  LOGICAL AXIOM

Children:  NECESSARY BUT NOT SUFFICIENT CONCEPT DEFINITION  IMPLICATION SET
 SUFFICIENT SET  DATA PROPERTY SET  INCLUSION SET  DATA PROPERTY SET AXIOMS
 NECESSARY SET  PROPERTY SET  INTERVAL PROPERTY SET  PROPERTY SET AXIOMS

Since: Inception

id: LogicalSet | UUID: 70eb421f-2489-5b4f-8699-686487e4593b

Logically equivalent to

An operator for the reasoner to determine the equivalence

⊆ Taxonomy operator

Parents:  TAXONOMY OPERATOR

Since: Inception

id: LogicallyEquivalentTo | UUID: 3642d9a3-8e23-5289-836b-366c0b1e2900

Long

The retired dynamic-column type for long integer values, superseded by the Long field kind.

⊆ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: Long | UUID: dea8cdf1-de75-5991-9791-79714e4a964d

Long default

The Long field's loud default: the stretched-sevens sentinel 7777777777777777 (eighteen sevens) — a repdigit signature that reads as deliberately planted and cannot plausibly occur as natural data, where Long.MIN_VALUE would read as an overflow bug (and sits one away from the platform's pre-inception time sentinel). Never Java's silent zero.

⊆ Defaults and templates model ▯ Meaning

Parents:  DEFAULTS AND TEMPLATES MODEL  MEANING

Since: Inception

id: LongDefault | UUID: bb92b64a-d0f4-5ead-a2ec-a3d178527e10

Lower Bound Open

The marker that an interval's lower bound is open: the bound value itself lies outside the interval.

Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: LowerBoundOpen | UUID: a0096ba1-0718-4c03-ad8f-8143c44091e7

Master path

The path released consumer-facing content lands on: the mainline other paths promote into.

Path

Parents:  PATH

Since: Inception

id: MasterPath | UUID: 1f20134a-960e-11e5-8994-feff819cdc9f

Match Rule

Why the Member match relation value is recorded: to say how a constrained field's value must match an enumerated member — which member match relation the constraint holds under.

Constraint model $\sqcap$ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: MatchRule | UUID: 0e389613-6adc-5998-b3e3-7fef8a167a67

Maximum Value Operator

The Value Operator assigned to the Maximum Value in a Range

Concrete value operator $\sqcap$ Meaning

Parents:  CONCRETE VALUE OPERATOR  MEANING

Since: Inception

id: MaximumValueOperator | UUID: 7b8916ab-fd50-41df-8fc2-0b2a7a78be6d

Meaning

The interpretation or explanation field for a pattern/semantics

Model concept

Parents:  MODEL CONCEPT

Children:  DEFAULTED DATA TYPES  LONG DEFAULT  VALUE-SET FIELD  IS-A
 CONCEPT VERSIONS FIELD  AUTHOR  CONCEPT DEFAULT  PATH FIELD  PATH
 BOOLEAN DEFAULT  PUBLIC ID FIELD  ARRAY DEFAULT  DESCRIPTION ACCEPTABILITY
 ROSTER ORDER  CONCEPT FIELD  CONSTRAINT ANCHOR CONCEPT  RELATIONSHIP DESTINATION
 SEMANTIC DEFAULT  STATUS FIELD  MEMBER MATCH RELATION
 PRESENTATION UNIT DIFFERENT  SEMANTIC FIELD FIELD
 SUFFICIENT CONCEPT DEFINITION OPERATOR  EL PROFILE SET OPERATOR
 DESCRIPTION CASE SIGNIFICANCE  GREAT BRITAIN ENGLISH DIALECT
 EL++ INFERRED TERMINOLOGICAL AXIOMS  PATTERN VERSIONS FIELD  DIGRAPH DEFAULT
 STARTER SET AUTHOR ROSTER  STATED DEFINITION  BYTEARRAY DEFAULT
 SEMANTIC VERSIONS SET  SEMANTIC PATTERN FIELD  LANGUAGE FOR DESCRIPTION
 CONCEPT PATTERN FOR LOGIC COORDINATE  IDENTIFIER SOURCE  INFERRED DEFINITION
 EL++ STATED TERMINOLOGICAL AXIOMS  RELATIONSHIP ORIGIN  VERSION PROPERTIES
 CONSTRAINED FIELD  QUERY CLAUSES  STATUS VALUE  REFERENCE RANGE MINIMUM
 COMMENT  COMPONENTIDLIST DEFAULT  CONSTRAINED PATTERN  ROSTER AUTHOR
 AXIOM SYNTAX  STRING DEFAULT  SEMANTIC REFERENCED COMPONENT FIELD
 INTEGER DEFAULT  IDENTIFIED COMPONENT  COMPONENTIDSET DEFAULT  TIME FIELD
 ORIGINATED MODULE  STARTER DATA AUTHORIZING  TEXT COMPARISON MEASURE SEMANTIC
 DECIMAL DEFAULT  PREFERRED REVIEWER ASSIGNMENT  FLOAT DEFAULT  DITREE DEFAULT
 FIELD SUBSTITUTION  DESCRIPTION TYPE  COMPONENT VERSIONS FIELD  AUTHOR FIELD
 COMPONENT FIELD  CONSTRAINT KIND  REFERENCE RANGE MAXIMUM  COMPONENT DEFAULT
 DESCRIPTION SEMANTIC  PATTERN MEANING FIELD  INSTANT DEFAULT  IDENTIFIER VALUE
 TIME  TAXONOMY OPERATOR  MODULE FIELD  UNITED STATES OF AMERICA ENGLISH DIALECT
 STAMP VERSIONS FIELD  PREFERRED REVIEWER  AXIOMATIZED COMPONENT  STAMP FIELD
 SEMANTIC FIELD  SEMANTIC VERSIONS FIELD  MODULE  ORIGINATED PATH
 PATTERN PURPOSE FIELD  PATH ORIGINS  SUBJECT OF COMMENTARY  MODULE ORIGINS
 MAXIMUM VALUE OPERATOR  SEMANTIC TYPE  VALUE-SET PATTERN  FIELD DEFINITION FIELD
 LITERAL VALUE  MINIMUM VALUE OPERATOR  VALUE CONSTRAINT SOURCE  PATH CONCEPT
 EXAMPLE UCUM UNITS  VALUE CONSTRAINT  PATTERN FIELD  TEXT FOR DESCRIPTION

Since: Inception

id: Meaning | UUID: a06158ff-e08a-5d7d-bcfa-6cbfdb138910

Meaning Declaration

Why a Pattern Version Pattern's Pattern meaning field is recorded: to hold this pattern-version's own declared meaning concept.

⊆ Chronicle and version model ⊇ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: MeaningDeclaration | UUID: 864fa92e-244b-5eca-a566-ae3262bac9c5

Member match relation

The closed taxonomy of relations a Value-set Field Constraint Pattern semantic can require between a constrained field's value and an enumerated member. Relations are directed and named in full, and a relation is admitted only when its evaluator ships in code: relation concepts correspond one-to-one with the service-loaded evaluators — the bijection gate.

⊆ Constraint model ⊇ Meaning

Parents:  CONSTRAINT MODEL  MEANING

Children:  EQUAL MATCH RELATION

Since: Inception

id: MemberMatchRelation | UUID: 8bd2739b-7209-53b4-9c98-adbb413dd415

Membership semantic

The semantic type carrying no fields: a semantic whose entire content is that its referenced component belongs to the pattern's own set. The SOLOR-era name for what this starter set's own content now calls Set membership.

⊆ Semantic type ⊇ Purpose

Parents:  SEMANTIC TYPE  PURPOSE

Since: Inception

id: MembershipSemantic | UUID: 4fa29287-a80e-5f83-abab-4b587973e7b7

Minimum Value Operator

The Value Operator assigned to the Minimum Value in a Range

⊆ Concrete value operator ⊇ Meaning

Parents:  CONCRETE VALUE OPERATOR  MEANING

Since: Inception

id: `MinimumValueOperator` | UUID: `ded98e42-f74a-4432-9ae7-01b94dc2fdea`

Model concept

A concept representing a model construct within Integrated Knowledge Management — the structural and data-type terminology a knowledge base uses to describe itself, as distinct from the domain content it represents.

⊞ Integrated Knowledge Management

Parents:  INTEGRATED KNOWLEDGE MANAGEMENT

Children:  CONSTRAINT MODEL  EDITORIAL MODEL  IKEFOUNDATION ROOT  DATA CONCEPT
 PROVENANCE MODEL  LOGICAL EXPRESSION MODEL  VIEW COORDINATE MODEL
 LEGACY MODEL CONCEPTS  CHRONICLE AND VERSION MODEL  STAMP DIMENSIONS
 DESCRIPTION MODEL  PURPOSE  MEANING  TINKAR MODEL CONCEPT
 DEFAULTS AND TEMPLATES MODEL  GRAPH MODEL  IDENTIFIER MODEL

Since: Inception

id: `ModelConcept` | UUID: `7bbd4210-381c-11e7-9598-0800200c9a66`

Module

The concept identifying the export or authoring boundary a version belongs to — the module dimension of a STAMP, and the root of the value space a module field's concept is drawn from; distinct from Module for version, which names why the field is recorded.

⊞ STAMP dimensions ⊞ Meaning

Parents:  STAMP DIMENSIONS  MEANING

Children:  DEFAULTS AND TEMPLATES MODULE  SOLOR MODULE  DEVELOPMENT MODULE
 USERS MODULE  KOMET MODULE  SOLOR OVERLAY MODULE  IKEFOUNDATION MODULE
 PRIMORDIAL MODULE  SANDBOX MODULE

Since: Inception

id: `Module` | UUID: `40d1c869-b509-32f8-b735-836eac577a67`

Module exclusion set for stamp coordinate

The stamp-coordinate dimension holding modules to ignore outright: a version in an excluded module is invisible to the view regardless of any preference order.

ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ModuleExclusionSetForStampCoordinate | UUID: 3fe047f0-33b0-5254-91c2-43e65f90d30b

Module field

A concept field whose value is the module dimension of a STAMP — the module the version belongs to.

Concept field $\sqcap$ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: ModuleField | UUID: e6359a86-a1df-4721-8a1a-1f1f075ec3d9

Module for user

The per-user editing default naming which module a user's edits land in.

View coordinate model

Parents:  VIEW COORDINATE MODEL

Since: Inception

id: ModuleForUser | UUID: c8fd4f1b-d842-5245-9a7d-a58dc0ac1c11

Module for version

Why a version's module field is recorded: to say which export and authoring boundary the version belongs to.

Version Properties $\sqcap$ Purpose

Parents:  VERSION PROPERTIES  PURPOSE

Since: Inception

id: ModuleForVersion | UUID: 67cd64f1-96d7-5110-b847-556c055ac063

**Module Lineage**

Why a module's own origin record is captured: to track which modules a given module originated from.

⊆ Provenance model ⊐ Purpose

Parents: PROVENANCE MODEL PURPOSE

Since: Inception

id: ModuleLineage | UUID: f8c2a07e-8969-5123-874e-aa0d0ab3f6a0

**Module options for edit coordinate**

The edit-coordinate dimension listing the modules an editor may choose to commit into.

⊆ ImmutableCoordinate Properties

Parents: IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ModuleOptionsForEditCoordinate | UUID: 19305aff-95d9-55d9-b015-825cc68eadc7

**Module origins**

The origin-set value a Module origins pattern semantic carries: the set of modules a module originated from — what the pattern's one field holds, distinct from Originated Module, the module whose origins are declared.

⊆ Provenance model ⊐ Meaning

Parents: PROVENANCE MODEL MEANING

Since: Inception

id: ModuleOrigins | UUID: 462862d4-5df9-426e-b785-a1264e24769f

**Module origins pattern**

Pattern of module origins

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 ORIGINATED MODULE	 MODULE LINEAGE	—
<i>Field 1</i>		
 MODULE ORIGINS	 ORIGIN MODULE SET	 COMPONENT ID SET DATA TYPE

Since: Inception

id: ModuleOriginsPattern | UUID: 536b0ec4-4974-47ae-93a6-ae6c4d169780

Module preference list for language coordinate

The language-coordinate dimension holding modules in preference order, for choosing among descriptions that differ only by module.

≡ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ModulePreferenceListForLanguageCoordinate | UUID: f36e7ca6-34a2-58b5-8b25-736457515f9c

Module preference list for stamp coordinate

The stamp-coordinate dimension listing the modules a view accepts; together with the exclusion set it makes module scoping affirmative.

≡ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ModulePreferenceListForStampCoordinate | UUID: f56ef2df-6758-5271-a587-317a4fed6c2e

Module preference order for stamp coordinate

The stamp-coordinate dimension ranking acceptable modules: when versions tie across modules, the earlier module in this order wins.

ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ModulePreferenceOrderForStampCoordinate | UUID: ddeda759-e89c-5186-aa40-d63070756ab4

Modules for stamp coordinate

The stamp-coordinate dimension holding the modules a view accepts versions from.

ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: ModulesForStampCoordinate | UUID: bf69c4f1-95c9-5956-a10a-d3ba9276c019

Navigation

Groups navigation: the precomputed parent and child adjacency structures a navigator renders, and the concepts that classify them.

Purpose

Parents:  PURPOSE

Children:  NAVIGATION CONCEPT SET  INFERRED NAVIGATION  TAXONOMY NAVIGATION CACHE
 STATED NAVIGATION

Since: Inception

id: Navigation | UUID: 4d9707d8-adf0-5b15-89fc-039e4ff6fec8

Navigation concept set

The navigation-coordinate dimension holding the concept set a navigator traverses, scoping navigation to a chosen part of the taxonomy.

Navigation

Parents:  NAVIGATION

Since: Inception

id: NavigationConceptSet | UUID: fc965c5d-ad17-555e-bcb5-b78fd45c8c5f



Navigation vertex

A vertex as it appears in navigation: one node of the precomputed navigation graph, presenting the concept it stands for.

⊆ Vertex

Parents: VERTEX

Since: Inception

id: NavigationVertex | UUID: c7f01834-34ca-5f8b-8f80-193fbeb12eae



Necessary but not sufficient concept definition

The definition status marking a concept whose stated conditions are all necessary of it but not enough to classify an instance as it: a reasoner can check membership but never infer it. The contrast to Sufficient concept definition.

⊆ Logical set

Parents: LOGICAL SET

Since: Inception

id: NecessaryButNotSufficientConceptDefinition | UUID: e1a12059-3b01-3296-9532-d10e49d0afc3



Necessary set

A set of relationships that is always true of a concept.

⊆ Logical set

Parents: LOGICAL SET

Since: Inception

id: NecessarySet | UUID: acaa2eba-8364-5493-b24c-b3884d34bb60



NID

The store-local integer identifier a component holds while loaded: a fast internal handle, never an exchangeable identity – identity crossing any boundary is a public identifier.

⊆ Identifier model

Parents:  IDENTIFIER MODEL

Since: Inception

id: NID | UUID: d1a17272-9785-51aa-8bde-cc556ab32ebb

Not Applicable

The case-significance value for text where case carries no meaning at all, so the question of sensitivity does not arise.

⊆ Description case significance

Parents:  DESCRIPTION CASE SIGNIFICANCE

Since: Inception

id: NotApplicable | UUID: d4cc29ae-c0c1-563a-985d-5165a768dd44

Numeric Value Restriction

Why a Value Constraint Pattern semantic exists: to restrict which literal values are legal for a component — the numeric-domain counterpart to Field Value Restriction.

⊆ Constraint model ⊐ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: NumericValueRestriction | UUID: eea1cc9e-1f3a-50f3-8a37-c4c3cd0bf589

Object

An encapsulation of data together with procedures

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: Object | UUID: 72765109-6b53-3814-9b05-34ebddd16592

Object Properties

Objects are instances of classes, the properties describe the data or attributes that an object can have

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: ObjectProperties | UUID: 3ef4311c-70c0-5149-9e06-53d745f85b15



Or

The disjunctive connective: an expression satisfying any one of its children satisfies the whole. Mirrors LogicalAxiom.Atom.Connective.Or in the sealed hierarchy.

⊆ Connective operator

Parents:  CONNECTIVE OPERATOR

Since: Inception

id: Or | UUID: 2c940bcf-22a8-5fc9-b232-580021e758ed



Order for axiom attachments

The per-user display preference fixing the order an axiom's attached semantics are presented in.

⊆ View coordinate model

Parents:  VIEW COORDINATE MODEL

Since: Inception

id: OrderForAxiomAttachments | UUID: abcb0946-20e1-5483-8469-3e8fa0ce20c4



Order for concept attachments

The per-user display preference fixing the order a concept's attached semantics are presented in.

⊆ View coordinate model

Parents:  VIEW COORDINATE MODEL

Since: Inception

id: OrderForConceptAttachments | UUID: 6167efcb-50e8-534d-9827-fdd60b02ae00

**Order for description attachments**

The per-user display preference fixing the order a description's attached semantics, such as dialect judgments, are presented in.

⊞ View coordinate model

Parents: VIEW COORDINATE MODEL

Since: Inception

id: OrderForDescriptionAttachments | UUID: 69ee3f13-e2ba-5a96-9b91-5eecfad8e587

**Origin Module Set**

Why a module's origin set value is recorded: to name the modules this module originated from.

⊞ Provenance model ⊞ Purpose

Parents: PROVENANCE MODEL PURPOSE

Since: Inception

id: OriginModuleSet | UUID: 77723bf6-084d-519b-9275-c6e6449aef0e

**Originated Module**

What a Module origins pattern semantic's referenced component is: the module whose origins are declared — the module that originated from the modules the semantic names, distinct from Module origins, which names the origin-set value itself.

⊞ Provenance model ⊞ Meaning

Parents: PROVENANCE MODEL MEANING

Since: Inception

id: OriginatedModule | UUID: 57fb5067-c863-5745-8f21-4937e68d7611

**Originated Path**

What a Path origins pattern semantic's referenced component is: the path whose branch point is declared — the path that originated from the path the semantic names, distinct from Path origins, which names the origin-record value itself.

⊞ Provenance model ⊞ Meaning

Parents: PROVENANCE MODEL MEANING

Since: Inception

id: `OriginatedPath` | UUID: `8383342f-ef73-5d8c-8d61-5000795b9f9d`

OWL Axiom Syntax Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 AXIOMATIZED COMPONENT	 AXIOM EXPRESSION	—
<i>Field 1</i>		
 AXIOM SYNTAX	 EXPRESS AXIOM SYNTAX	 STRING DATA TYPE

Since: Inception

id: `OWLAxiomSyntaxPattern` | UUID: `c0ca180b-aae2-5fa1-9ab7-4a24f2dfe16b`

Part of

The taxonomy operator asserting partitive relation: one concept names a part of what another names, as distinct from a kind of it.

⊆ Taxonomy operator

Parents:  TAXONOMY OPERATOR

Since: Inception

id: `PartOf` | UUID: `b4c3f6f9-6937-30fd-8412-d0c77f8a7f73`

Partial

The correlation grouping value for a partial match: the expressions correspond in some but not all of their parts, as opposed to an exact correspondence.

⊆ Grouping

Parents:  GROUPING

Since: Inception

id: `Partial` | UUID: `a7f9574c-8e8b-515d-9c21-9896063cc3b8`

Path

A set of assets under version control that can be managed distinctly from other assets. Paths “branch” from other paths when established, and can be “merged” with other paths as well.

⊞ STAMP dimensions ⊞ Meaning

Parents:  STAMP DIMENSIONS  MEANING

Children:  MASTER PATH  SANDBOX PATH  PRIMORDIAL PATH  DEVELOPMENT PATH

Since: Inception

id: Path | UUID: 4459d8cf-5a6f-3952-9458-6d64324b27b7

Path concept

A concept that is itself a version-control path — the concept kind a path is represented by. As the Path origins pattern's first field meaning it names the field holding the origin path a path branched from.

⊞ Concept type ⊞ Meaning

Parents:  CONCEPT TYPE  MEANING

Since: Inception

id: PathConcept | UUID: 1b9d9f95-fc0a-55ac-b2e6-7c8b37660046

Path coordinate name

The field naming a stored path coordinate, so a surface can present saved coordinates by name.

⊞ Path coordinate properties

Parents:  PATH COORDINATE PROPERTIES

Since: Inception

id: PathCoordinateName | UUID: a293a9a0-eb1e-5418-83c7-bec376b62245

Path coordinate properties

Character or attribute of coordinates referring to a series of connected points, that form a shape or trajectory

⊞ View coordinate model

Parents:  [VIEW COORDINATE MODEL](#)

Children:  [PROMOTION PATH FOR EDIT COORDINATE](#)  [PATH COORDINATE NAME](#)

Since: Inception

id: PathCoordinateProperties | UUID: ec41e427-f009-5e45-a643-6dc658d63d83

Path field

A concept field whose value is the path dimension of a STAMP — the path the version was committed on.

⊞ Concept field ⊞ Meaning

Parents:  [CONCEPT FIELD](#)  [MEANING](#)

Since: Inception

id: PathField | UUID: 6622a391-e2e6-45a0-97e1-c58cd0184092

Path for path coordinate

The path-coordinate dimension naming the version-control path a position sits on; with a time, it is the coordinate's complete position.

⊞ ImmutableCoordinate Properties

Parents:  [IMMUTABLECOORDINATE PROPERTIES](#)

Since: Inception

id: PathForPathCoordinate | UUID: 748e073c-fea7-58dd-8aa3-f18fdd82ddfc

Path for user

The per-user editing default naming which version-control path a user's edits land on.

⊞ View coordinate model

Parents:  [VIEW COORDINATE MODEL](#)

Since: Inception

id: PathForUser | UUID: 12131382-1535-5a77-928b-6eacad221ea2

Path for version

Why a version's path field is recorded: to say which version-control path the version was committed on.

⊞ Version Properties ⊞ Purpose

Parents:  VERSION PROPERTIES  PURPOSE

Since: Inception

id: PathForVersion | UUID: ad3dd2dd-ddb0-584c-bea4-c6d9b91d461f

Path Lineage

Why a path's own origin record is captured: to track which path it branched from, and when.

⊞ Provenance model ⊞ Purpose

Parents:  PROVENANCE MODEL  PURPOSE

Since: Inception

id: PathLineage | UUID: 386b515e-dae4-5dcd-8390-cd56d7c58bcc

Path options for edit coordinate

The edit-coordinate dimension listing the paths an editor may choose to commit on.

⊞ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: PathOptionsForEditCoordinate | UUID: 2110c10c-9174-55aa-8ffe-91650c77d0b3

Path origins

A path's own origin record — the path-coordinate property saying where a path branched from. As the Path origins pattern's second field meaning it names the origin instant that record fixes: the branch point up to which the origin path's versions are visible.

⊞ Provenance model ⊞ Meaning

Parents:  PROVENANCE MODEL  MEANING

Since: Inception

id: PathOrigins | UUID: 6e6a112e-7d8c-53c7-aaf1-c46e2d69743c

Path origins for stamp path

The record of which path a path branched from, and when: each origin is a position up to which the origin path's versions are visible from the branching path.

ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: PathOriginsForStampPath | UUID: f33e1668-34dd-53dd-8728-31b29934b482

Path origins pattern

Pattern of path origins

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 ORIGINATED PATH	 PATH LINEAGE	—
<i>e.g.  MASTER PATH</i>		
<i>Field 1</i>		
 PATH CONCEPT	 BRANCH SOURCE	 COMPONENT DATA TYPE
<i>e.g.  DEVELOPMENT PATH</i>		
<i>Field 2</i>		
 PATH ORIGINS	 BRANCH POINT	 INSTANT LITERAL
<i>e.g. Latest</i>		

Since: Inception

id: PathOriginsPattern | UUID: 70f89dd5-2cdb-59bb-bbaa-98527513547c

Pattern Chronology Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PATTERN FIELD	 CHRONICLE IDENTITY AND HISTORY	—
Field 1		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
Field 2		
 PATTERN VERSIONS FIELD	 PATTERN VERSIONS SET	 COMPONENT ID SET DATA TYPE

Since: Inception

id: PatternChronologyPattern | UUID: 5bc93adb-9d39-43fe-a7a4-1492245b7efb

Pattern field

A component field narrowed to patterns: its value references a pattern. The Pattern Chronology Pattern's referenced-component meaning, and the parent of Semantic pattern field.

⊞ Component field ⊞ Meaning

Parents:  COMPONENT FIELD  MEANINGChildren:  SEMANTIC PATTERN FIELD

Since: Inception

id: PatternField | UUID: 751790c7-e1e4-42bc-b531-54c54bd6eebd

Pattern meaning field

A concept field whose value is a pattern version's declared meaning concept — naming what a semantic of the pattern is about.

⊞ Concept field ⊞ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: PatternMeaningField | UUID: 996d0023-a355-422f-a84d-16dda6ece1b0

Pattern Membership

Why a Semantic Chronology Pattern's Semantic pattern field is recorded: to identify which pattern this semantic is an instance of.

⊆ Chronicle and version model ⊇ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: PatternMembership | UUID: 3eeb5f15-b87e-5201-a214-0f1d8bbbf00e

Pattern purpose field

A concept field whose value is a pattern version's declared purpose concept — naming why semantics of the pattern are captured.

⊆ Concept field ⊇ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: PatternPurposeField | UUID: 352c821b-7a11-454c-a127-48ad3206573d

Pattern Version Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PATTERN VERSIONS FIELD	 VERSION SNAPSHOT	—
<i>Field 1</i>		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE
<i>Field 2</i>		
 PATTERN MEANING FIELD	 MEANING DECLARATION	 COMPONENT DATA TYPE
<i>Field 3</i>		

Meaning	Purpose	Data type
 PATTERN PURPOSE FIELD	 PURPOSE DECLARATION	 COMPONENT DATA TYPE
<i>Field 4</i>		
 FIELD DEFINITION FIELD	 FIELD DEFINITIONS SET	 COMPONENT ID SET DATA TYPE

Since: Inception

id: PatternVersionPattern | UUID: a90f8a4d-ae13-476b-98b8-814914f9704e

Pattern versions field

The component versions field narrowed to pattern chronicles: its value collects a pattern's own versions. The Pattern Version Pattern's referenced-component meaning.

⊆ Component versions field ⊐ Meaning

Parents:  COMPONENT VERSIONS FIELD  MEANING

Since: Inception

id: PatternVersionsField | UUID: 7b8ecbbf-55b4-41bc-acbf-51824e74446a

Pattern versions set

Why a pattern chronicle's versions field is recorded: to hold every version the pattern has ever had — the pattern-chronicle specialization of Component versions set.

⊆ Component versions set ⊐ Purpose

Parents:  COMPONENT VERSIONS SET  PURPOSE

Since: Inception

id: PatternVersionsSet | UUID: a254ccee-ef02-4504-9645-0a2ed7af955d

Phenomenon

A unique thought, fact, or circumstance

⊆ Integrated Knowledge Management

Parents:  INTEGRATED KNOWLEDGE MANAGEMENT

Children:  UNCATEGORIZED PHENOMENON  HEALTH CONCEPT  EXAMPLE UCUM UNITS

Since: Inception

id: Phenomenon | UUID: c2e8bc47-3353-5e02-b0d1-2a5916efed4d

Polish dialect

The Polish dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in Polish usage; a language coordinate's dialect order consults it when choosing what to show.

⊞ Dialect

Parents:  DIALECT

Since: Inception

id: PolishDialect | UUID: 315cd879-1557-5a30-b325-a5d3df9e1c2b

Polish Language

The Polish language, as a value for a description's language field: a language coordinate that names it selects Polish-language descriptions for rendering.

⊞ Language

Parents:  LANGUAGE

Since: Inception

id: PolishLanguage | UUID: c924b887-da88-3a72-b8ea-fa86990467c9

Position on path

A position: a path plus a time on it, the point a stamp coordinate resolves latest against.

⊞ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: PositionOnPath | UUID: 31173582-a49d-51c6-813f-f42d0976aaea

Preferred

The acceptability value marking a description as the one to show first in a dialect: at most one preferred description per description type per dialect.

⊞ Description acceptability

Parents:  DESCRIPTION ACCEPTABILITY

Since: Inception

id: Preferred | UUID: 266f1bc3-3361-39f3-bffe-69db9daea56e

 Preferred reviewer

Illustrative constrained field: the reviewer a Preferred Reviewer Pattern semantic names.

⊞ Concept field ⊞ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: PreferredReviewer | UUID: bd91d84a-f50c-5d1c-b7be-af948dfa5d80

 Preferred reviewer assignment

Illustrative pattern whose single field is constrained to membership in the starter set author roster, demonstrating a Value-set Field Constraint Pattern end to end.

⊞ Editorial model ⊞ Meaning

Parents:  EDITORIAL MODEL  MEANING

Since: Inception

id: PreferredReviewerAssignment | UUID: e9557b20-943e-505a-9a8e-dc6b666cdedf

 Preferred Reviewer Pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 PREFERRED REVIEWER ASSIGNMENT	 REVIEW ROUTING	—
Field 1		
 PREFERRED REVIEWER	 ASSIGNED REVIEWER	 COMPONENT DATA TYPE

Meaning	Purpose	Data type
<i>default</i>  <i>GRETTEL</i>		

Since: Inception

id: PreferredReviewerPattern | UUID: 88b444be-682f-5981-8cb8-bd6f1cb4c53a

Presentation unit different

Marks that a value is presented in a unit other than the one it was recorded in, so a reader knows a conversion stands between record and display.

≡ Meaning

Parents:  MEANING

Since: Inception

id: PresentationUnitDifferent | UUID: e86d3887-717b-545f-b6b5-611210913b23

Primordial module

The module for content present from a store's beginning: the baseline's bootstrap module dimension.

≡ Module

Parents:  MODULE

Since: Inception

id: PrimordialModule | UUID: c2012321-3903-532e-8a5f-b13e4ca46e86

Primordial path

The root of the path graph: the path from which every other path ultimately originates, and the one no path precedes. Released foundational knowledge lands here, beginning with a knowledge set's inception release, stamped at the Inception instant. Time on this path before inception is pre-inception (historically, premundane).

≡ Path

Parents:  PATH

Since: Inception

id: PrimordialPath | UUID: e95b6718-f824-5540-817b-8e79544eb97a



Primordial state

Concept used to represent a status for components that have not yet been released and exist in their most basic form.

⊆ Status value

Parents:  STATUS VALUE

Since: Inception

id: PrimordialState | UUID: b17bde5d-98ed-5416-97cf-2d837d75159d



Primordial UUID for chronicle

The first universally unique identifier in a chronicle's public identity: the stable primary key among its identifiers.

⊆ Chronicle properties

Parents:  CHRONICLE PROPERTIES

Since: Inception

id: PrimordialUUIDForChronicle | UUID: e0fcafbf-7191-5cdc-b14a-19d4d97f71bd



Promotion Path for Edit Coordinate

The edit-coordinate dimension naming the path content is promoted to when moving between paths, such as sandbox work promoted to a mainline path.

⊆ Path coordinate properties

Parents:  PATH COORDINATE PROPERTIES

Since: Inception

id: PromotionPathForEditCoordinate | UUID: db124d3b-c1bb-530e-8fd4-577f570355be



Property pattern implication

A baseline duplicate of Property sequence implication, retired to Legacy: the platform's axiom meaning is Property sequence implication; this identity is retained for compatibility.

⊆ Legacy model concepts

Parents:  [LEGACY MODEL CONCEPTS](#)

Since: Inception

id: `PropertyPatternImplication` | UUID: `e0de0d09-6e27-5738-bc8f-0fc94bb115fc`

Property Sequence

An ordered sequence of properties, as used by a property-sequence implication: the chain whose composition is asserted to imply another property.

⊆ Logical expression model

Parents:  [LOGICAL EXPRESSION MODEL](#)

Since: Inception

id: `PropertySequence` | UUID: `d0d759fd-510f-475a-900e-b1439b4536e1`

Property sequence implication

The atom asserting a property-chain implication: the properties in the sequence, composed in order, imply the property this axiom names. Mirrors `LogicalAxiom.Atom.PropertySequenceImplication`.

⊆ Atom

Parents:  [ATOM](#)

Since: Inception

id: `PropertySequenceImplication` | UUID: `9a47a5db-42a6-49ee-9083-54bc305a9456`

Property set

The logical set grouping property axioms, such as transitivity and reflexivity declarations, under a definition root. Mirrors `LogicalAxiom.LogicalSet.PropertySet`.

⊆ Logical set

Parents:  [LOGICAL SET](#)

Since: Inception

id: `PropertySet` | UUID: `e273b5c0-c012-5e53-990c-aec5c2cb33a7`

Property set Axioms

A baseline grouping for property-set axioms, retained for identity compatibility: the platform's set meaning is Property set.

⊆ Logical set

Parents:  LOGICAL SET

Since: Inception

id: PropertySetAxioms | UUID: ca2fdefd-0481-41cb-8074-41a78f94034d

Provenance model

The meta-schema of where content came from: the role concepts of commit provenance, module lineage (which modules a module originated from), and path lineage (which path a path branched from, and when). The STAMP dimensions themselves are operational value spaces at the taxonomy root; this subsystem holds the concepts that describe provenance relationships among them.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  BRANCH SOURCE  MODULE LINEAGE  CREATIVE COMMONS BY LICENSE
 COMMIT PROVENANCE  ORIGINATED MODULE  VERSION PROVENANCE
 STARTER DATA AUTHORIZING  ORIGIN MODULE SET  PATH LINEAGE  ORIGINATED PATH
 PATH ORIGINS  MODULE ORIGINS  BRANCH POINT

Since: Inception

id: ProvenanceModel | UUID: a93e37fd-7333-5577-8cba-8cd4a0823612

Public ID field

The field kind holding a chronicle's public identity — the universally unique identifiers that name a component across every system it is exchanged with. The first field of every chronicle-shape pattern, recorded to uniquely identify knowledge graph components.

⊆ String field ⊧ Meaning

Parents:  STRING FIELD  MEANING

Since: Inception

id: PublicIDField | UUID: 196838c5-55f4-4e40-8618-b9ce60685c2f

Purpose

The reason for which a Tinkar value in a pattern was created or for which it exist.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  BRANCH SOURCE  DESCRIPTION ATTACHMENT  Is-a  PURPOSE DECLARATION
 IDENTIFIER AUTHORITY  MODULE LINEAGE  AXIOM EXPRESSION  IDENTIFIER TEXT
 DESCRIPTION ACCEPTABILITY  STATUS FOR VERSION  CONCRETE VALUE OPERATOR  LANGUAGE
 EXTERNAL IDENTITY MAPPING  NUMERIC VALUE RESTRICTION  DESCRIPTION ROLE
 STAMP VERSIONS SET  CASE SENSITIVITY RULE  EXPRESS AXIOM SYNTAX  SET MEMBERSHIP
 SEMANTIC FIELD FIELDS SET  VERSION SNAPSHOT  CONSTRAINT SCOPE
 MEANING DECLARATION  COMMIT PROVENANCE  PATTERN MEMBERSHIP
 VERSION PROVENANCE  TAXONOMY REFERENCE POINT  MATCH RULE  ORIGIN MODULE SET
 TAXONOMY NAVIGATION CACHE  ROSTER MEMBERSHIP  VALUE DISAMBIGUATION
 REFERENCE RANGE  ASSIGNED REVIEWER  DESCRIPTION SEMANTIC  ATTACHMENT TARGET
 AUTHOR FOR VERSION  UNIT EXAMPLE  UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS
 MEMBERSHIP SEMANTIC  PATTERN VERSIONS SET  PATH LINEAGE  TIME FOR VERSION
 CHRONICLE IDENTITY AND HISTORY  CONSTRAINT RULE  REVIEW ROUTING  DESCRIPTION
 DISPLAY SEQUENCE  PATH FOR VERSION  CONCEPT VERSIONS SET  LEGAL VALUE SOURCE
 DATA TYPE DEFAULT PROVISION  MODULE FOR VERSION  BRANCH POINT
 FIELD DEFINITIONS SET  LOGICAL DEFINITION  VERSION HISTORY  COMPONENT VERSIONS SET
 REFERENCE RANGE AUTHORITY  ROSTER ENTRY  EDITORIAL CLARIFICATION
 FIELD VALUE RESTRICTION  DATA TYPE DEFAULT EXEMPLAR  NAVIGATION

Since: Inception

id: Purpose | UUID: c3dffc48-6493-54df-a2f0-14be8ba03091

Purpose Declaration

Why a Pattern Version Pattern's Pattern purpose field is recorded: to hold this pattern-version's own declared purpose concept.

⊆ Chronicle and version model ⊇ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: PurposeDeclaration | UUID: 2d841ab9-c435-5ce6-8e4b-8308daf22bab

Query clauses

A distinct component/query that serves a specific purpose

⊆ Meaning

Parents:  MEANING

Children:  VIEW COORDINATE KEY  BOOLEAN REFERENCE

Since: Inception

id: QueryClauses | UUID: 4905348c-ba1d-58ae-821f-52877d9acee3

Reference Range

The range of values specific to a component

⊆ Tinkar Model concept ⊐ Purpose

Parents:  TINKAR MODEL CONCEPT  PURPOSE

Children:  REFERENCE RANGE MINIMUM  REFERENCE RANGE MAXIMUM

Since: Inception

id: ReferenceRange | UUID: 87ce975b-309b-47f4-a6c6-4ae6df6649a1

Reference Range Authority

Why a value constraint's source value is recorded: to name the organization that specifies the constraint.

⊆ Constraint model ⊐ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: ReferenceRangeAuthority | UUID: 4e756e4e-be8d-583c-bbb4-97b43438edfe

Reference Range Maximum

The upper bound of a reference range: the greatest value the range admits, paired with an operator saying whether the bound itself is included.

⊆ Reference Range ⊐ Meaning

Parents:  REFERENCE RANGE  MEANING

Since: Inception

id: ReferenceRangeMaximum | UUID: 72d58983-b1e1-4ca9-833f-0e40c1defd39

Reference Range Minimum

The lower bound of a reference range: the least value the range admits, paired with an operator saying whether the bound itself is included.

⊆ Reference Range ⊐ Meaning

Parents:  REFERENCE RANGE  MEANING

Since: Inception

id: ReferenceRangeMinimum | UUID: 37c35a88-9e27-42ca-b626-186773c4b612

Referenced component for semantic

The field naming the component a semantic is attached to, in the semantic-properties vocabulary. Duplicates Component for semantic and the Semantic referenced component field kind; retained for identity compatibility.

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: ReferencedComponentForSemantic | UUID: a9ba4749-c11f-5f35-a991-21796fb89ddc

Referenced component subtype restriction

Restricts a pattern's referenced component to a subtree beneath a named concept, narrowing the kind-level Referenced component type restriction to particular content.

⊆ Role operator

Parents:  ROLE OPERATOR

Since: Inception

id: ReferencedComponentSubtypeRestriction | UUID: 8af1045e-1122-5072-9f29-ce7da9337915

**Referenced component type restriction**

Restricts which component KIND a pattern's referenced component may be -- concept, semantic, pattern, or STAMP -- checked when an editor supplies one. The coarse restriction; its subtype counterpart narrows further, to a particular subtree.

⊆ Role operator

Parents: [ROLE OPERATOR](#)

Since: Inception

id: ReferencedComponentTypeRestriction | UUID: 902f97b6-2ef4-59d7-b6f9-01278a00061c

**Reflexive Feature**

Marks a property as reflexive: everything stands in that relation to itself.

⊆ Logical expression model

Parents: [LOGICAL EXPRESSION MODEL](#)

Since: Inception

id: ReflexiveFeature | UUID: 7e779e4a-61ed-5c4a-aacc-03cf524b7c73

**Regular name description type**

There may be descriptions/synonyms marked as “regular.”

⊆ Description type

Parents: [DESCRIPTION TYPE](#)

Since: Inception

id: RegularNameDescriptionType | UUID: 8bfba944-3965-3946-9bcb-1e80a5da63a2

**Relationship destination**

Signifies path to child concepts which are more specific than the Tinkar term

⊆ Tinkar Model concept ⊃ Meaning

Parents: [TINKAR MODEL CONCEPT](#) [MEANING](#)

Since: Inception

id: RelationshipDestination | UUID: a3dd69af-355c-54ce-ba13-2902a7ae9551

Relationship origin

Signifies path to parent concepts which are more general than the Tinkar term

⊆ Tinkar Model concept ⊇ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: RelationshipOrigin | UUID: ad22d43b-3ee7-550b-9660-a6e68af347c2

Review Routing

Why a Preferred Reviewer Pattern semantic exists: to direct a component's future edits to a specific reviewer.

⊆ Editorial model ⊇ Purpose

Parents:  EDITORIAL MODEL  PURPOSE

Since: Inception

id: ReviewRouting | UUID: e1cc2998-97f6-5b8c-847f-587af5c47df8

Role

Is an abstract representation of a high-level role for a therapeutic medicinal product; the concepts are not intended to describe a detailed indication for therapeutic use nor imply that therapeutic use is appropriate in all clinical situations.

⊆ Typed atom

Parents:  TYPED ATOM

Children:  ROLE TYPE  ROLE RESTRICTION

Since: Inception

id: Role | UUID: 46ae9325-dd24-5008-8fda-80cf1f0977c7

Role group

An association between a set of attribute or axiom value pairs that causes them to be considered together within a concept definition or post coordinated expression.

⊆ Role type

Parents:  ROLE TYPE

Since: Inception

id: RoleGroup | UUID: a63f4bf2-a040-11e5-8994-feff819cdc9f

Role operator

Concept that is used to describe universal vs existential restrictions.

⊆ Logical expression model

Parents:  LOGICAL EXPRESSION MODEL

Children:  EXISTENTIAL RESTRICTION  REFERENCED COMPONENT TYPE RESTRICTION
 UNIVERSAL RESTRICTION  REFERENCED COMPONENT SUBTYPE RESTRICTION

Since: Inception

id: RoleOperator | UUID: f9860cb8-a7c7-5743-9d7c-ffc6e8a24a0f

Role restriction

A restriction on which concepts may fill a role: the legacy carrier of role-value constraints.

⊆ Role

Parents:  ROLE

Since: Inception

id: RoleRestriction | UUID: 988bb02a-9b4a-4ef9-937e-fa8a6afc6c42

Role type

Refers to a concept that represents a particular kind of relationship that can exist between two entities. It defines the specific function or responsibility that one entity plays in relation to another.

⊆ Role

Parents:  ROLE

Children:  INTERVAL ROLE TYPE  LATERALITY  ROLE GROUP  HAS ACTIVE INGREDIENT
 FEATURE ROLE TYPE  HAS DOSE FORM

Since: Inception

id: RoleType | UUID: 76320274-be2a-5ba0-b3e8-e6d2e383ee6a

Role type to add

The retired action parameter naming which role type a rule-driven action would add to a concept's definition.

⊆ Action properties

Parents:  ACTION PROPERTIES

Since: Inception

id: RoleTypeToAdd | UUID: 0c3ca846-0374-5a5c-8da4-67e0e2e28868

Root for logic coordinate

The logic-coordinate dimension naming the taxonomy root a classification is computed beneath.

⊆ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: RootForLogicCoordinate | UUID: 862cc189-bbcb-51a0-89a4-16e1854be247

Roster author

The author concept named by one roster entry.

⊆ Concept field ⊐ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: RosterAuthor | UUID: c6075b23-9497-5059-bec3-41cd43168d59

Roster Entry

Why a roster author value is recorded: to name one member of the roster.

⊆ Editorial model ⊐ Purpose

Parents:  EDITORIAL MODEL  PURPOSE

Since: Inception

id: RosterEntry | UUID: caf2fcea-264f-5501-8aaf-c4de674b4098

Roster Membership

Why a Starter Set Author Roster Pattern semantic exists: to enumerate this starter set's own author roster as a value-set source.

⊞ Editorial model ▢ Purpose

Parents:  EDITORIAL MODEL  PURPOSE

Since: Inception

id: RosterMembership | UUID: 8ac7bd2d-3db5-5644-8575-a1591bc3a1c0

Roster order

The roster entry's display order — the "additional characteristic" field a value-set constraint's Value-set field disambiguates against.

⊞ Editorial model ▢ Meaning

Parents:  EDITORIAL MODEL  MEANING

Since: Inception

id: RosterOrder | UUID: dea12527-af2c-5c05-9f6e-ccab55036d19

Russian dialect

The Russian dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in Russian usage; a language coordinate's dialect order consults it when choosing what to show.

⊞ Dialect

Parents:  DIALECT

Since: Inception

id: RussianDialect | UUID: 300126d1-2604-579f-a59b-e3c1179a173a

Russian language

The Russian language, as a value for a description's language field: a language coordinate that names it selects Russian-language descriptions for rendering.

Language

Parents:  LANGUAGE

Since: Inception

id: RussianLanguage | UUID: 0818dbb7-3fe1-59d7-99c2-c8dc42ff2cce

Sandbox component

An ISAAC-era marker for components in sandbox experimentation, retired to Legacy: sandbox scoping is carried by module and path dimensions.

Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: SandboxComponent | UUID: c93829b2-aa78-5a84-ac9a-c34307844166

Sandbox module

The module for sandbox experimentation content.

Module

Parents:  MODULE

Children:  SANDBOX PATH MODULE

Since: Inception

id: SandboxModule | UUID: c5daf0e9-30dc-5b3e-a521-d6e6e72c8a95

Sandbox path

A path for components under testing.

Path

Parents:  PATH

Since: Inception

id: SandboxPath | UUID: 80710ea6-983c-5fa0-8908-e479f1f03ea9



Sandbox path module

The module for content specific to a sandbox path.

↳ Sandbox module

Parents: SANDBOX MODULE

Since: Inception

id: SandboxPathModule | UUID: 715bd36d-6090-5b37-8ae7-88c9e532010e



Semantic Chronology Pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
SEMANTIC FIELD	CHRONICLE IDENTITY AND HISTORY	—
Field 1		
PUBLIC ID FIELD	UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	COMPONENT
Field 2		
SEMANTIC PATTERN FIELD	PATTERN MEMBERSHIP	COMPONENT
Field 3		
SEMANTIC REFERENCED COMPONENT FIELD	ATTACHMENT TARGET	COMPONENT
Field 4		
SEMANTIC VERSIONS SET	VERSION HISTORY	COMPONENT

Since: Inception

id: SemanticChronologyPattern | UUID: 5f0ad6ca-638e-4052-82b0-3f564ac99b3f



Semantic data type

A field that holds a reference to a semantic.

↳ Display Fields

Parents:  [DISPLAY FIELDS](#)

Since: Inception

id: `SemanticDataType` | UUID: `9c3dfc88-51e4-5e51-a59a-88dd580162b7`

Semantic default

The Semantic field's loud default: the fully qualified name description semantic of Uninitialized Component — a real, resolvable semantic whose subject is itself the loud placeholder, so the reference is valid yet plainly unrevised.

⌵ Defaults and templates model ▢ Meaning

Parents:  [DEFAULTS AND TEMPLATES MODEL](#)  [MEANING](#)

Since: Inception

id: `SemanticDefault` | UUID: `5f7efa76-4723-57aa-b099-20769f7a6a1f`

Semantic field

A component field narrowed to semantics: its value references a semantic. The Semantic Chronology Pattern's referenced-component meaning — the chronicle kind that must also record which pattern its semantic is an instance of and what it is attached to.

⌵ Component field ▢ Meaning

Parents:  [COMPONENT FIELD](#)  [MEANING](#)

Since: Inception

id: `SemanticField` | UUID: `8b6c69d7-a5aa-4db2-bcea-8c7b2817b02f`

Semantic field concepts

The concept type for concepts that serve as semantic field values in the base model.

⌵ Concept type

Parents:  [CONCEPT TYPE](#)

Since: Inception

id: `SemanticFieldConcepts` | UUID: `b4316cb8-14fe-5b32-b03b-f5f966c87819`

**Semantic field field**

The field kind naming a semantic version's field-value slot: each value is one field the version actually carries, as distinct from the field definition its pattern declares.

⊆ Field categories ⊐ Meaning

Parents:  FIELD CATEGORIES  MEANING

Since: Inception

id: SemanticFieldField | UUID: f6572c76-b5c0-41da-99c0-4344694e7e3c

**Semantic field fields set**

Why a semantic version's field-value field is recorded: to hold, together, every field value that version carries — the value-side counterpart to a pattern's Field definitions set.

⊆ Field categories ⊐ Purpose

Parents:  FIELD CATEGORIES  PURPOSE

Since: Inception

id: SemanticFieldFieldsSet | UUID: 8dcfc1a1-31f2-46f7-8247-0a17a6d7c6c0

**Semantic field name**

The field holding a display name for one of a semantic's fields, for surfaces that label fields individually.

⊆ Semantic properties

Parents:  SEMANTIC PROPERTIES

Since: Inception

id: SemanticFieldName | UUID: 15489c68-673d-503e-bff7-e9d59e5dc15c

**Semantic list for chronicle**

The field holding every semantic attached to a chronicle's component -- its descriptions, identifiers, axioms, and memberships.

⊆ Chronicle properties

Parents:  CHRONICLE PROPERTIES

Since: Inception

id: SemanticListForChronicle | UUID: c809b2c0-9235-5f64-bbda-34210d91bdf8

Semantic pattern field

A pattern field whose value identifies which pattern a semantic is an instance of — the field a semantic chronicle records for pattern membership.

⊆ Pattern field ⊐ Meaning

Parents:  PATTERN FIELD  MEANING

Since: Inception

id: SemanticPatternField | UUID: 19dd5dd3-1075-4113-a437-5f1f7c2d55bc

Semantic properties

The attributes or characteristics of a concept, term, or element that convey meaning or semantics in a given context

⊆ Chronicle and version model

Parents:  CHRONICLE AND VERSION MODEL

Children:  SEMANTIC FIELD NAME  LOGIC GRAPH FOR SEMANTIC

 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS

Since: Inception

id: SemanticProperties | UUID: b717ae48-5488-5dda-a980-97855001cc99

Semantic referenced component field

A component field whose value identifies the component a semantic is attached to — the field a semantic chronicle records for its attachment target.

⊆ Component field ⊐ Meaning

Parents:  COMPONENT FIELD  MEANING

Since: Inception

id: SemanticReferencedComponentField | UUID: 4111ba1e-c818-4c5d-9fed-34d07298d009



Semantic type

Groups semantics by the kind of value they carry: concept, component, logical expression, or bare membership.

⊆ Meaning

Parents: MEANING

Children: LOGICAL EXPRESSION SEMANTIC COMPONENT SEMANTIC MEMBERSHIP SEMANTIC CONCEPT SEMANTIC

Since: Inception

id: SemanticType | UUID: 3daac6c4-78c5-5271-9c63-6e28f80e0c52



Semantic version field pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
SEMANTIC VERSIONS FIELD	VERSION SNAPSHOT	—
Field 1		
STAMP FIELD	VERSION PROVENANCE	COMPONENT DATA TYPE
Field 2		
SEMANTIC FIELD FIELD	SEMANTIC FIELD FIELDS SET	COMPONENT ID SET DATA TYPE

Since: Inception

id: SemanticVersionFieldPattern | UUID: 82f93e84-cee1-44bc-bb6d-4cc2a722048b



Semantic versions field

The component versions field narrowed to semantic chronicles: its value collects a semantic's own versions. The Semantic version field pattern's referenced-component meaning.

⊆ Component versions field ⊆ Meaning

Parents: COMPONENT VERSIONS FIELD MEANING

Since: Inception

id: SemanticVersionsField | UUID: aeb73410-a679-4ea8-93fe-7c4785599778

Semantic versions set

Why a semantic chronicle's versions field is recorded: to hold every version the semantic has ever had — the semantic-chronicle specialization of Component versions set.

⊆ Component versions set ⊆ Meaning

Parents:  COMPONENT VERSIONS SET  MEANING

Since: Inception

id: SemanticVersionsSet | UUID: 4fd69aed-556f-4938-94cc-ea7ea707cccf

Set membership

A component's role as an element of a pattern's own set of semantics — the modern replacement for the deprecated SOLOR term Membership semantic.

⊆ Chronicle and version model ⊆ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: SetMembership | UUID: a4716ecb-2838-504c-9501-e69948bd4e62

Signed integer

The retired dynamic-column type for signed whole numbers, superseded by Integer data type.

⊆ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: SignedInteger | UUID: 1d1c2073-d98b-3dd3-8aad-a19c65aa5a0c

SnoRocket classifier

The SnoRocket reasoner's identity: inferred content it writes is stamped with this concept as author; the classifier-as-author provenance model.

⊆ Author

Parents:  AUTHOR

Since: Inception

id: `SnoRocketClassifier` | UUID: `1f201fac-960e-11e5-8994-feff819cdc9f`

SOLOR concept assemblage

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT PATTERN FOR LOGIC COORDINATE	 MEMBERSHIP SEMANTIC	—

Since: Inception

id: `SOLORConceptAssemblage` | UUID: `d39b3ecd-9a80-5009-a8ac-0b947f95ca7c`

Solor Concepts Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONCEPT PATTERN FOR LOGIC COORDINATE	 SET MEMBERSHIP	—

Since: Inception

id: `SolorConceptsPattern` | UUID: `bf3a840d-fe61-5a1f-87ae-b023768fb476`

SOLOR module

The module for SOLOR core content: the inherited upstream content boundary.

Module

Parents:  MODULE

Since: Inception

id: `SOLORModule` | UUID: `f680c868-f7e5-5d0e-91f2-615eca8f8fd2`

SOLOR overlay module

The module for SOLOR overlay content: upstream additions layered over the core module.

Module

Parents:  MODULE

Since: Inception

id: SOLOROverlayModule | UUID: 9ecc154c-e490-5cf8-805d-d2865d62aef3

Spanish language

The Spanish language, as a value for a description's language field: a language coordinate that names it selects Spanish-language descriptions for rendering.

Language

Parents:  LANGUAGE

Since: Inception

id: SpanishLanguage | UUID: 0fcf44fb-d0a7-3a67-bc9f-eb3065ed3c8e

STAMP Chronology Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STAMP FIELD	 CHRONICLE IDENTITY AND HISTORY	—
<i>Field 1</i>		
 PUBLIC ID FIELD	 UNIQUELY IDENTIFY KNOWLEDGE GRAPH COMPONENTS	 COMPONENT DATA TYPE
<i>Field 2</i>		
 STAMP VERSIONS FIELD	 STAMP VERSIONS SET	 COMPONENT ID SET DATA TYPE

Since: Inception

id: STAMPChronologyPattern | UUID: e16abc7a-2a7b-42af-b168-d77aec8116ea

 STAMP dimensions

The five value spaces the versioning machinery selects a version's STAMP constituents from: Status value, Time, Author, Module, and Path. Machinery, so it files under Model concept — yet each dimension's subtree is operational content, not meta-schema: the statuses versions actually carry, the authors who actually commit, the modules and paths content actually lives on. A dimension root's double duty as a STAMP field meaning does not make its subtree a description of anything.

⊞ Model concept

Parents:  MODEL CONCEPT

Children:  AUTHOR  PATH  STATUS VALUE  TIME  MODULE

Since: Inception

id: STAMPDimensions | UUID: aea9ca27-3c50-522f-acda-ae881f954603

 STAMP field

A component field whose value references a STAMP — the five-dimension provenance tuple a version was committed with. The STAMP Chronology Pattern's referenced-component meaning, and the first field of every version-shape pattern, recorded for version provenance.

⊞ Component field ⊞ Meaning

Parents:  COMPONENT FIELD  MEANING

Since: Inception

id: STAMPField | UUID: 3d821e64-a2ee-4414-8949-1bc92ef5d5b6

 STAMP pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 VERSION PROPERTIES	 COMMIT PROVENANCE	—
Field 1		
 STATUS VALUE	 STATUS FOR VERSION	 COMPONENT DATA TYPE

Meaning	Purpose	Data type
<i>Field 2</i>		
 TIME	 TIME FOR VERSION	 LONG
<i>Field 3</i>		
 AUTHOR	 AUTHOR FOR VERSION	 COMPONENT DATA TYPE
<i>Field 4</i>		
 MODULE	 MODULE FOR VERSION	 COMPONENT DATA TYPE
<i>Field 5</i>		
 PATH	 PATH FOR VERSION	 COMPONENT DATA TYPE

Since: Inception

id: STAMPPattern | UUID: 9fd67fee-abf9-551d-9d0e-76a4b1e8b4ee

STAMP version field pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STAMP VERSIONS FIELD	 VERSION SNAPSHOT	—
<i>Field 1</i>		
 STAMP FIELD	 VERSION PROVENANCE	 COMPONENT DATA TYPE
<i>Field 2</i>		
 STATUS FIELD	 STATUS FOR VERSION	 COMPONENT DATA TYPE
<i>Field 3</i>		
 TIME FIELD	 TIME FOR VERSION	 STRING DATA TYPE
<i>Field 4</i>		

Meaning	Purpose	Data type
 AUTHOR FIELD	 AUTHOR FOR VERSION	 COMPONENT DATA TYPE
<i>Field 5</i>		
 MODULE FIELD	 MODULE FOR VERSION	 COMPONENT DATA TYPE
<i>Field 6</i>		
 PATH FIELD	 PATH FOR VERSION	 COMPONENT DATA TYPE

Since: Inception

id: STAMPVersionFieldPattern | UUID: 73c798cf-bc77-49a2-84f7-4c0f4bc4c012

STAMP versions field

The component versions field narrowed to STAMP chronicles: its value collects a STAMP's own versions. The STAMP version field pattern's referenced-component meaning.

⊞ Component versions field ⊞ Meaning

Parents:  COMPONENT VERSIONS FIELD  MEANING

Since: Inception

id: STAMPVersionsField | UUID: b8251bea-4248-4a46-8b4a-349500693a9f

STAMP versions set

Why a STAMP chronicle's versions field is recorded: to hold every version the STAMP has ever had — the STAMP-chronicle specialization of Component versions set.

⊞ Component versions set ⊞ Purpose

Parents:  COMPONENT VERSIONS SET  PURPOSE

Since: Inception

id: STAMPVersionsSet | UUID: edb90567-7822-4129-a406-b359b825f922

Standard Korean dialect

The standard Korean dialect: the general-usage dialect of Korean, as opposed to any specialized register.

⌵ Korean dialect

Parents:  KOREAN DIALECT

Since: Inception

id: StandardKoreanDialect | UUID: f90722cc-5e40-5b9b-a2a6-f4dfa312a6a9

 Starter Data Authoring

Define necessary minimum viable concepts to use Tinkar Data

⌵ Provenance model ▢ Meaning

Parents:  PROVENANCE MODEL  MEANING

Since: Inception

id: StarterDataAuthoring | UUID: 070deb74-acc5-46bf-b9c6-eaee1b58ef52

 Starter set author roster

Illustrative value-set source: an ordered roster of this starter set's own authors, demonstrating a Value-set Field Constraint Pattern's value-set pattern.

⌵ Editorial model ▢ Meaning

Parents:  EDITORIAL MODEL  MEANING

Since: Inception

id: StarterSetAuthorRoster | UUID: 73077978-3e44-5ab3-a056-2dc88151a33e

 Starter Set Author Roster Pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 STARTER SET AUTHOR ROSTER	 ROSTER MEMBERSHIP	—
e.g.  STARTER SET AUTHOR ROSTER PATTERN		
Field 1		

Meaning	Purpose	Data type
 ROSTER AUTHOR	 ROSTER ENTRY	 COMPONENT DATA TYPE
<i>e.g.</i>  <i>GRETEL</i>		
<i>Field 2</i>		
 ROSTER ORDER	 DISPLAY SEQUENCE	 LONG
<i>e.g. 1</i>		

Since: Inception

id: StarterSetAuthorRosterPattern | UUID: 47072664-f25d-5c58-9850-4124240c4e97

Stated Definition

Relationships/Axioms of a concept that have been explicitly stated and defined

⊆ Tinkar Model concept ⊇ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: StatedDefinition | UUID: 28608bd3-ac73-4fe8-a5f0-1efe0d6650a8

Stated navigation

Navigation computed from stated definitions: parent and child adjacency as authored, before classification.

⊆ Navigation

Parents:  NAVIGATION

Since: Inception

id: StatedNavigation | UUID: 614017af-9903-53d9-aab4-15fd02193dce

Stated Navigation Pattern

A pattern specifying the origins and destinations for concepts based on the stated terminological axioms.

Pattern shape		
Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 IS-A	 TAXONOMY NAVIGATION CACHE	—
Field 1		
 RELATIONSHIP DESTINATION	 IS-A	 COMPONENT ID SET DATA TYPE
Field 2		
 RELATIONSHIP ORIGIN	 IS-A	 COMPONENT ID SET DATA TYPE

Since: Inception

id: StatedNavigationPattern | UUID: d02957d6-132d-5b3c-adba-505f5778d998

Stated pattern for logic coordinate

The pattern an author's own stated axioms are recorded under, for a Logic Coordinate.

⊆ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: StatedPatternForLogicCoordinate | UUID: cfd2a47e-8169-5e71-9122-d5b73efd990a

Stated premise type

The axiom origin marking a definition as authored: the expression an editor asserted, before any classification.

⊆ Axiom origin

Parents:  AXIOM ORIGIN

Since: Inception

id: StatedPremiseType | UUID: 3b0dbd3b-2e53-3a30-8576-6c7fa7773060

Status field

A concept field whose value is the status dimension of a STAMP — the version's state (active, inactive, or otherwise) at the time it was committed.

⌵ Concept field ▢ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: StatusField | UUID: f2c79ebb-3095-44ea-831f-992aed48801f

Status for version

Why a version's status field is recorded: to say whether the version is active, inactive, or withdrawn, so a view can accept or ignore it.

⌵ Version Properties ▢ Purpose

Parents:  VERSION PROPERTIES  PURPOSE

Since: Inception

id: StatusForVersion | UUID: 0608e233-d79d-5076-985b-9b1ea4e14b4c

Status value

The status of the STAMP Coordinate(Active, Cancelled, Inactive, Primordial)

⌵ STAMP dimensions ▢ Meaning

Parents:  STAMP DIMENSIONS  MEANING

Children:  PRIMORDIAL STATE  INACTIVE STATE  ACTIVE STATE  CANCELED STATE
 WITHDRAWN STATE

Since: Inception

id: StatusValue | UUID: 10b873e2-8247-5ab5-9dec-4edef37fc219

String data type

A sequence of characters, either as a literal constant or as a variable. Strings could be used to represent terms from code systems or URLs, textual definitions, etc.

⌵ Display Fields

Parents:  [DISPLAY FIELDS](#)

Since: Inception

id: `StringDataType` | UUID: `a46aaf11-b37a-32d6-abdc-707f084ec8f5`

String default

The String field's loud default: the literal "UNINITIALIZED" — a value that works as an initial value but reads as obviously needing revision, never Java's silent empty string.

⌵ Defaults and templates model ▢ Meaning

Parents:  [DEFAULTS AND TEMPLATES MODEL](#)  [MEANING](#)

Since: Inception

id: `StringDefault` | UUID: `60fb29ec-e2ad-55a2-bf03-9d1eb35786ba`

String field

The field kind for string-valued fields: a pattern field holding text.

⌵ Field definition data type field

Parents:  [FIELD DEFINITION DATA TYPE FIELD](#)

Children:  [PUBLIC ID FIELD](#)

Since: Inception

id: `StringField` | UUID: `27d3905b-b19a-41ff-bed1-fc55f49f8ce4`

Subject of Commentary

The referenced-component role a Comment Pattern semantic's attachment target plays: the subject free-text commentary is about — distinct from Comment, which names the commentary's own content type, not the component being commented on.

⌵ Editorial model ▢ Meaning

Parents:  [EDITORIAL MODEL](#)  [MEANING](#)

Since: Inception

id: `SubjectOfCommentary` | UUID: `53e639c4-723c-5bef-a86e-edeaecaa1cf2`

**Sufficient concept definition**

The definition status marking a concept as sufficiently defined: its stated conditions are enough to classify an instance as the concept, so a reasoner can place it by definition alone.

⊆ Sufficient concept definition operator

Parents: SUFFICIENT CONCEPT DEFINITION OPERATOR

Since: Inception

id: SufficientConceptDefinition | UUID: 6d9cd46e-8a8f-310a-a298-3e55dcf7a986

**Sufficient concept definition operator**

Groups the operators marking a definition as sufficient: conditions enough to classify anything meeting them as the concept, not merely necessary of it.

⊆ Meaning

Parents: MEANING

Children: SUFFICIENT CONCEPT DEFINITION

Since: Inception

id: SufficientConceptDefinitionOperator | UUID: dfa80f36-dbe6-5006-8509-c497a26ceab5

**Sufficient set**

A set of relationships that differentiate a concept and its subtypes from all other concepts. A concept that contains at least one set of necessary and sufficient conditions is considered defined.

⊆ Logical set

Parents: LOGICAL SET

Since: Inception

id: SufficientSet | UUID: 8aa48cfd-485b-5140-beb9-0d122f7812d9

**Swedish language**

The Swedish language, as a value for a description's language field: a language coordinate that names it selects Swedish-language descriptions for rendering.

⊆ Language

Parents:  LANGUAGE

Since: Inception

id: SwedishLanguage | UUID: 9784a791-8fdb-32f7-88da-74ab135fe4e3

Taxonomy field constraint kind

The kind of taxonomy-relative rule a Taxonomy Field Constraint Pattern semantic expresses: which concepts are legal values for the constrained pattern field, derived from the taxonomy under the checking view.

↳ Constraint model

Parents:  CONSTRAINT MODEL

Children:  KIND-OF FIELD CONSTRAINT  DESCENDANT FIELD CONSTRAINT

 IMMEDIATE CHILD FIELD CONSTRAINT  LEAF DESCENDANT FIELD CONSTRAINT

Since: Inception

id: TaxonomyFieldConstraintKind | UUID: 3bb160c7-cf0f-5937-824a-88fb819bd3f3

Taxonomy Field Constraint Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONSTRAINED PATTERN	 FIELD VALUE RESTRICTION	—
<i>e.g.  DESCRIPTION PATTERN</i>		
<i>Field 1</i>		
 CONSTRAINED FIELD	 CONSTRAINT SCOPE	 COMPONENT DATA TYPE
<i>e.g. <-2147479135></i>		
<i>Field 2</i>		
 CONSTRAINT KIND	 CONSTRAINT RULE	 COMPONENT DATA TYPE
<i>e.g.  KIND-OF FIELD CONSTRAINT</i>		

Meaning	Purpose	Data type
<i>Field 3</i>		
 CONSTRAINT ANCHOR CONCEPT	 TAXONOMY REFERENCE POINT	 COMPONENT DATA TYPE
<i>e.g.</i>  LANGUAGE		

Since: Inception

id: TaxonomyFieldConstraintPattern | UUID: 3a6f5ed0-730a-57a9-8575-8d0efb816149

Taxonomy Navigation Cache

Why a navigation pattern's referenced component carries an IsA semantic: a pre-computed parent/child structure so traversal doesn't require re-deriving from axioms every time.

⊆ Navigation ⊐ Purpose

Parents:  NAVIGATION  PURPOSE

Since: Inception

id: TaxonomyNavigationCache | UUID: 6d84de25-0aee-52bc-83bb-9e2fa647b442

Taxonomy operator

An operator or set of operations applied within the context of a taxonomy

⊆ Meaning

Parents:  MEANING

Children:  LOGICALLY EQUIVALENT TO  PART OF

Since: Inception

id: TaxonomyOperator | UUID: e9252365-7a43-57ea-bf94-3f23bab4ef99

Taxonomy Reference Point

Why the Constraint anchor concept value is recorded: to give a kind-of, descendant, leaf-descendant, or immediate-child rule something concrete to measure against.

⊆ Constraint model ⊐ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: `TaxonomyReferencePoint` | UUID: `80a0d6b3-1a18-5afe-9155-75084beb06e7`

Template concept

Parent of the per-purpose template attachment concepts: each child is minted for one template purpose, and a template semantic of a pattern for that purpose is a complete semantic of that pattern whose referenced component is that child — starting content offered when authoring new components. As with a default value semantic, attachment there is a support declaration, never a domain assertion, and no child joins a domain member set.

⊆ Defaults and templates model

Parents:  [DEFAULTS AND TEMPLATES MODEL](#)

Since: Inception

id: `TemplateConcept` | UUID: `663fb8eb-8035-5e22-b423-9fca85a3f11c`

Temporal Axiom

Groups axioms about time: the temporal counterpart of the interval-axiom vocabulary.

⊆ Axioms

Parents:  [AXIOMS](#)

Since: Inception

id: `TemporalAxiom` | UUID: `5144d836-18d8-4881-a377-2d4640b710a9`

Text comparison measure semantic

Text comparison with a focus on semantic meaning involves evaluating the similarity or relatedness between pieces of text based on their underlying meaning rather than just their surface structure.

⊆ Meaning

Parents:  [MEANING](#)

Children:  [CASE INSENSITIVE EVALUATION](#)  [CASE SENSITIVE EVALUATION](#)

Since: Inception

id: `TextComparisonMeasureSemantic` | UUID: `b1531e68-4e7a-5194-b1f9-9aaace269372`

Text for description

Captures the human readable text for a description in Komet

⊞ Tinkar Model concept ⊞ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: TextForDescription | UUID: 8bdcbe5d-e92e-5c10-845e-b585e6061672

Time

A point in time, as a general concept — the meaning counterpart to Status value/Author/Module/Path: what kind of value a STAMP's time field holds, distinct from TimeForVersion, which names why it is recorded.

⊞ STAMP dimensions ⊞ Meaning

Parents:  STAMP DIMENSIONS  MEANING

Since: Inception

id: Time | UUID: 13557c25-add3-50fe-aeb1-edb52befcd40

Time field

An instant field whose value is the time dimension of a STAMP — when the version was committed.

⊞ Instant field ⊞ Meaning

Parents:  INSTANT FIELD  MEANING

Since: Inception

id: TimeField | UUID: 15293325-c16b-4f2e-8109-5b22b3355bcd

Time for version

Why a version's time field is recorded: to say when the version was committed, so a view positioned at a moment can resolve what was current then.

⊞ Version Properties ⊞ Purpose

Parents:  VERSION PROPERTIES  PURPOSE

Since: Inception

id: TimeForVersion | UUID: a9b0dfb2-f463-5dae-8ba8-7f2e8385571b

Tinkar base model component pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 STARTER DATA AUTHORIZING	 SET MEMBERSHIP	—
<i>e.g.</i>  UNINITIALIZED COMPONENT		

Since: Inception

id: TinkarBaseModelComponentPattern | UUID: 6070f6f5-893d-5144-adce-7d305c391cf9

Tinkar Model concept

Root of Tinkar's own meta-schema terminology: the concepts Tinkar uses to describe its data model itself — components, descriptions, dialects, fields, and axioms — rather than any modeled domain.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  CONCEPT TYPE  DESCRIPTION LIST FOR CONCEPT  IS-A  DESCRIPTION ACCEPTABILITY
 RELATIONSHIP DESTINATION  CONCRETE VALUE OPERATOR  LANGUAGE
 CONCEPT DETAILS TREE TABLE  DESCRIPTION CASE SIGNIFICANCE  DISPLAY FIELDS
 STATED DEFINITION  IDENTIFIER SOURCE  DESCRIPTION-LOGIC PROFILE
 COMPONENT TYPE FOCUS  INFERRED DEFINITION  RELATIONSHIP ORIGIN  BLANK CONCEPT
 DESCRIPTION TYPE  REFERENCE RANGE  FIELD CATEGORIES  CONCEPT VERSION
 DESCRIPTION SEMANTIC  IDENTIFIER VALUE  GROUPING  DESCRIPTION  AXIOMS
 LOGICAL DEFINITION  VALUE CONSTRAINT SOURCE  DIALECT  VALUE CONSTRAINT
 TEXT FOR DESCRIPTION

Since: Inception

id: TinkarModelConcept | UUID: bc59d656-83d3-47d8-9507-0e656ea95463

**Tinkar Starter Data Author**

The author identity the upstream starter data was committed under; retained for the historic versions consumers already hold; this set's own content is authored by IKE Community.

⊆ Author

Parents: AUTHOR

Since: Inception

id: TinkarStarterDataAuthor | UUID: dd96b2ea-6d7b-3791-ad74-bbdc67c493c1

**Transitive Feature**

Marks a property as transitive: if it relates the first to the second and the second to the third, it relates the first to the third.

⊆ Logical expression model

Parents: LOGICAL EXPRESSION MODEL

Since: Inception

id: TransitiveFeature | UUID: 53f866d0-fd61-5c85-a16c-150bd619a0ac

**Tree**

A graph in which every vertex except the root has exactly one predecessor; so every vertex is reached by exactly one path from the root. The platform's DiTree carrier holds definition expressions this way; the general DiGraph carrier relaxes the single-predecessor restriction.

⊆ Graph

Parents: GRAPH

Children: EL++ DITREE

Since: Inception

id: Tree | UUID: 5a233bd9-2650-5217-bc3e-5d01a5ad50ce

**Tree amalgam properties**

Data structure that consists of nodes connected by edges (a mixture or blend of different elements)

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Children:  INVERSE TREE LIST  TREE LIST

Since: Inception

id: TreeAmalgamProperties | UUID: d6151a47-4610-5a5c-abd0-42c82be9b633

Tree list

The legacy tree-amalgam field holding a navigation tree's parent-to-child adjacency, paired with Inverse tree list.

⊆ Tree amalgam properties

Parents:  TREE AMALGAM PROPERTIES

Since: Inception

id: TreeList | UUID: c11dd7a1-0ba1-5378-81d6-3efdba1e074b

Typed atom

An atom that carries a type concept naming which relation or feature it asserts: a role (existential restriction over a role type), an interval role (a role bounded to an interval), or a feature (a concrete-domain comparison against a feature type). Mirrors LogicalAxiom.Atom.TypedAtom in the sealed hierarchy.

⊆ Atom

Parents:  ATOM

Children:  ROLE  FEATURE  INTERVAL ROLE

Since: Inception

id: TypedAtom | UUID: 62538a4c-cc26-514a-a3ef-a59c66c2bc42

Uncategorized phenomenon

The domain placeholder for phenomena not yet classified into the model: content admitted before its category is settled.

⊆ Phenomenon

Parents:  PHENOMENON

Since: Inception

id: UncategorizedPhenomenon | UUID: 722f5ac8-1f5c-5d8f-96bb-370d79596f66

Uninitialized Component

The platform's null-object component: the value a component reference carries when it has never been set. The machinery produces it -- the nonexistent-stamp sentinel names it as both module and path, and path resolution guards on it -- and rendering surfaces show nothing for it, treating it like an unknown identifier. Distinct from Blank Concept, which an editor writes into a field a person deliberately cleared.

⊆ Chronicle and version model

Parents:  CHRONICLE AND VERSION MODEL

Since: Inception

id: UninitializedComponent | UUID: 55f74246-0a25-57ac-9473-a788d08fb656

Uniquely identify knowledge graph components

The purpose of this externally valid identifier—based on a list of UUIDs—is to provide a globally unique reference to each entity that is reliable across different systems and databases. This ensures consistent identification, serialization, and lookup of entities regardless of their storage location or implementation details.

⊆ Semantic properties ⊆ Purpose

Parents:  SEMANTIC PROPERTIES  PURPOSE

Since: Inception

id: UniquelyIdentifyKnowledgeGraphComponents | UUID: dde9a93d-250c-449b-bea0-ba1133d1387b

Unit Example

Why example UCUM units are recorded on a value constraint: to show a representative unit of measure for the constrained value, for a reader's reference.

⊆ Constraint model ⊆ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: UnitExample | UUID: bfc8c53c-4eee-58d4-a73f-f7d56bd6e04c

Unit of Measure

The field naming the unit an interval's bounds are expressed in.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: UnitOfMeasure | UUID: 40afdda5-89d6-4b80-8181-1ddd6eb92dc8

United States of America English dialect

The United States English dialect dimension: the anchor for dialect semantics recording whether a description is preferred or acceptable in US usage.

⊆ English Dialect ⊐ Meaning

Parents:  ENGLISH DIALECT  MEANING

Children:  US NURSING DIALECT

Since: Inception

id: UnitedStatesOfAmericaEnglishDialect | UUID: bca0a686-3516-3daf-8fcf-fe396d13cfad

Universal Restriction

Universal restrictions constrain the relationships along a given property to concepts that are members of a specific class.

⊆ Role operator

Parents:  ROLE OPERATOR

Since: Inception

id: UniversalRestriction | UUID: fc18c082-c6ad-52d2-b568-cc9568ace6c9

Universally unique identifier

A universally unique identifier that uniquely represents a concept in Tinkar

⊆ Identifier Source

Parents:  IDENTIFIER SOURCE

Since: Inception

id: UniversallyUniqueIdentifier | UUID: 845274b5-9644-3799-94c6-e0ea37e7d1a4

 Unmodeled role concept

An ISAAC-era marker for role concepts awaiting modeling, retired to Legacy.

⊆ Legacy model concepts

Parents:  LEGACY MODEL CONCEPTS

Since: Inception

id: UnmodeledRoleConcept | UUID: 4be7118f-e6ab-5dc7-bcba-b2cc8b028492

 Upper Bound Open

The marker that an interval's upper bound is open: the bound value itself lies outside the interval.

⊆ Interval Set Axioms

Parents:  INTERVAL SET AXIOMS

Since: Inception

id: UpperBoundOpen | UUID: c20b3b1e-112f-4cb2-b901-4046db844629

 US Dialect Pattern

Records whether a description is preferred or acceptable in the United States English dialect — one field, that description's acceptability for this dialect.

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 DESCRIPTION ACCEPTABILITY	 DESCRIPTION SEMANTIC	—
e.g. Uninitialized Component (SOLOR)		
Field 1		
 UNITED STATES OF AMERICA ENGLISH DIALECT	 DESCRIPTION ACCEPTABILITY	 COMPONENT DATA TYPE

Meaning	Purpose	Data type
<i>e.g.</i>  <i>PREFERRED</i>		

Since: Inception

id: `USDialectPattern` | UUID: `08f9112c-c041-56d3-b89b-63258f070074`

US Nursing dialect

The US nursing dialect: a specialization of United States English for nursing practice, where preferred terms differ from general US usage.

⊞ United States of America English dialect

Parents:  UNITED STATES OF AMERICA ENGLISH DIALECT

Since: Inception

id: `USNursingDialect` | UUID: `6e447636-1085-32ff-bc36-6748a45255de`

Users module

The module user-authored content lands in, keeping personal editing separate from released content.

⊞ Module

Parents:  MODULE

Since: Inception

id: `UsersModule` | UUID: `349161ba-9a6a-5c9c-a78f-281f19cfc057`

UUID data type

Distinction of data type of UUID

⊞ Dynamic column data types

Parents:  DYNAMIC COLUMN DATA TYPES

Since: Inception

id: `UUIDDataType` | UUID: `2faa9262-8fb2-11db-b606-0800200c9a66`

 UUID display field

The data type for fields holding a universally unique identifier value.

⊆ Display Fields

Parents:  DISPLAY FIELDS

Since: Inception

id: UUIDDisplayField | UUID: dea8cb0f-9bb5-56bb-af27-a14943cb24ba

 UUID list for component

The complete list of universally unique identifiers in a component's public identity; merging two components unions these lists.

⊆ Chronicle properties

Parents:  CHRONICLE PROPERTIES

Since: Inception

id: UUIDListForComponent | UUID: f8e3238e-7424-5a40-8649-a8d164382fec

 Value Constraint

A component has specific value requirements that needs to be met

⊆ Tinkar Model concept ⊎ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: ValueConstraint | UUID: 8c55fb86-92d8-42a9-ad70-1e23abbf7eec

 Value Constraint Pattern

A pattern specifying value constraint pattern

Pattern shape

Meaning	Purpose	Data type
Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to		
 VALUE CONSTRAINT	 NUMERIC VALUE RESTRICTION	—

Meaning	Purpose	Data type
<i>Field 1</i>		
 VALUE CONSTRAINT SOURCE	 REFERENCE RANGE AUTHORITY	 CONCEPT DATA TYPE
<i>Field 2</i>		
 MINIMUM VALUE OPERATOR	 CONCRETE VALUE OPERATOR	 CONCEPT DATA TYPE
<i>Field 3</i>		
 REFERENCE RANGE MINIMUM	 REFERENCE RANGE	 FLOAT DATA TYPE
<i>Field 4</i>		
 MAXIMUM VALUE OPERATOR	 CONCRETE VALUE OPERATOR	 COMPONENT DATA TYPE
<i>Field 5</i>		
 REFERENCE RANGE MAXIMUM	 REFERENCE RANGE	 FLOAT DATA TYPE
<i>Field 6</i>		
 EXAMPLE UCUM UNITS	 UNIT EXAMPLE	 STRING DATA TYPE

Since: Inception

id: ValueConstraintPattern | UUID: 922697f7-36ba-4afc-9dd5-f29d54b0fdec

Value Constraint Source

The source organization of that specifies the constraint

⊞ Tinkar Model concept ⊞ Meaning

Parents:  TINKAR MODEL CONCEPT  MEANING

Since: Inception

id: ValueConstraintSource | UUID: 09aa031a-6290-4ec9-a44c-23928a767da3

Value Disambiguation

Why the Value-set field value is recorded: to say which Model Feature of the value-set pattern actually holds the member value, when that pattern carries other fields too, such as a sort order.

☰ Constraint model ☐ Purpose

Parents:  CONSTRAINT MODEL  PURPOSE

Since: Inception

id: ValueDisambiguation | UUID: c2788745-e921-5499-a133-8709624e6c6b

Value-set field

For a value-set constraint, the Model Feature of the value-set pattern that holds the member value, named by its meaning concept: a declared field's meaning, or the value-set pattern's referenced-component meaning when the members are the referenced components themselves (a membership pattern) — disambiguating when that pattern carries other fields, such as a sort order.

☰ Concept field ☐ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: ValueSetField | UUID: 313f88e6-fe6f-5086-9a0f-9822846143b6

Value-set Field Constraint Pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 CONSTRAINED PATTERN	 FIELD VALUE RESTRICTION	—
<i>e.g.</i>  PREFERRED REVIEWER PATTERN		
<i>Field 1</i>		
 CONSTRAINED FIELD	 CONSTRAINT SCOPE	 COMPONENT DATA TYPE
<i>e.g.</i>  PREFERRED REVIEWER		
<i>Field 2</i>		
 VALUE-SET PATTERN	 LEGAL VALUE SOURCE	 COMPONENT DATA TYPE

Meaning	Purpose	Data type
e.g.  <i>STARTER SET AUTHOR ROSTER PATTERN</i>		
Field 3		
 VALUE-SET FIELD	 VALUE DISAMBIGUATION	 COMPONENT DATA TYPE
e.g.  <i>ROSTER AUTHOR</i>		
Field 4		
 MEMBER MATCH RELATION	 MATCH RULE	 COMPONENT DATA TYPE
e.g.  <i>EQUAL MATCH RELATION</i>		

Since: Inception

id: ValueSetFieldConstraintPattern | UUID: 86ca7486-15ec-52d0-9823-298a5a29314a

Value-set pattern

For a value-set constraint, the pattern whose active semantics enumerate the legal values.

⊆ Concept field ⊎ Meaning

Parents:  CONCEPT FIELD  MEANING

Since: Inception

id: ValueSetPattern | UUID: af79a5f0-6505-5f84-a4ec-1ffd119b401f

Version control path pattern

Pattern shape

Meaning	Purpose	Data type
<i>Referenced component — the component (concept, semantic, pattern, or STAMP) this pattern's semantics attach to</i>		
 PATH	 SET MEMBERSHIP	—
e.g.  <i>MASTER PATH</i>		

Since: Inception

id: VersionControlPathPattern | UUID: add1db57-72fe-53c8-a528-1614bda20ec6

Version History

Why a Semantic Chronology Pattern's Semantic versions set is recorded: to hold every version of this semantic, in order — the semantic-level counterpart to the versions-field/set pair every other chronicle-shape pattern already names distinctly.

⊞ Chronicle and version model ▢ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: VersionHistory | UUID: 658df6b1-3647-5569-a252-12e5e3d9ab09

Version list for chronicle

The field holding a chronicle's complete version list, in commit order.

⊞ Chronicle properties

Parents:  CHRONICLE PROPERTIES

Since: Inception

id: VersionListForChronicle | UUID: d6f27f80-8e20-58fe-8d69-66ad4644f969

Version Properties

What a STAMP pattern semantic means: the five-dimension provenance tuple of a version — status, time, author, module, and path together, as one value; the properties every version carries, rather than any single dimension alone.

⊞ Chronicle and version model ▢ Meaning

Parents:  CHRONICLE AND VERSION MODEL  MEANING

Children:  STATUS FOR VERSION  DESCRIPTION VERSION PROPERTIES  AUTHOR FOR VERSION
 TIME FOR VERSION  PATH FOR VERSION  MODULE FOR VERSION

Since: Inception

id: VersionProperties | UUID: 93f844df-38e5-5167-ba94-2c948b8bd07c

Version Provenance

Why a version-shape pattern's own STAMP field is recorded: to say who committed this version, in what state, module, path, and when.

⊞ Provenance model ▢ Purpose

Parents:  PROVENANCE MODEL  PURPOSE

Since: Inception

id: VersionProvenance | UUID: 2c40b894-236c-589c-9528-3cebd6eca49c

Version Snapshot

Why a version-shape pattern's referenced component carries this semantic: to record one point-in-time state of a chronicle — the STAMP it was committed with, plus whatever field values were true then.

⊞ Chronicle and version model ▢ Purpose

Parents:  CHRONICLE AND VERSION MODEL  PURPOSE

Since: Inception

id: VersionSnapshot | UUID: 53741119-2eb8-5c07-86a8-927efa563f5b

Vertex

A node of a graph: it carries a meaning — the concept naming what this vertex asserts or denotes — and zero or more properties, each a concept-keyed value. Edges are recorded as vertex adjacency, so the vertex is the unit of both content and connection.

⊞ Graph model

Parents:  GRAPH MODEL

Children:  NAVIGATION VERTEX

Since: Inception

id: Vertex | UUID: 2185c628-0fb7-588a-8c39-76be9da2fb95

Vertex display field

The data type for fields holding a single graph vertex.

⊞ Display Fields

Parents:  [DISPLAY FIELDS](#)

Since: Inception

id: `VertexDisplayField` | UUID: `3e56c6b6-5371-11eb-ae93-0242ac130002`

Vertex meaning

The concept a vertex denotes: what this node of the graph asserts, names, or stands for. Every vertex carries exactly one meaning; processing dispatches on it — a definition root, an and-connective, a role — the way code dispatches on a sealed type.

↳ Graph model

Parents:  [GRAPH MODEL](#)

Since: Inception

id: `VertexMeaning` | UUID: `b1b1afe0-7789-54da-87d1-d029ef997fd4`

Vertex property

A concept-keyed value carried on a vertex: the key is a concept naming what the value is (a role type, an operator, a literal), and the value is any field data type. Properties are how a vertex carries the particulars its meaning calls for.

↳ Graph model

Parents:  [GRAPH MODEL](#)

Since: Inception

id: `VertexProperty` | UUID: `8252c165-c7ed-50f4-9799-3340e7194a3a`

Vertex sort

The navigation-coordinate dimension naming the sort applied to a vertex's children when a navigator renders them.

↳ ImmutableCoordinate Properties

Parents:  [IMMUTABLECOORDINATE PROPERTIES](#)

Since: Inception

id: `VertexSort` | UUID: `e973f077-a99d-59e6-b7bd-804e87e0e639`

Vertex state set

The navigation-coordinate dimension holding the states a component must be in to appear in navigation; active-only in the default view.

⊆ ImmutableCoordinate Properties

Parents:  IMMUTABLECOORDINATE PROPERTIES

Since: Inception

id: VertexStateSet | UUID: 347cd3f2-8130-5032-8960-091e194e9afe

View coordinate key

A query-clause key identifying which view coordinate a stored query executes under, so queries reference a view without embedding one.

⊆ Query clauses

Parents:  QUERY CLAUSES

Since: Inception

id: ViewCoordinateKey | UUID: 4211cf36-bd75-586a-805c-51f059e2eaaa

View coordinate model

The meta-schema of views: the coordinate property families a rendering or editing surface assembles into a reproducible way of looking — stamp, language, logic, and path coordinate properties, and the immutable-coordinate property vocabulary. A coordinate's constituents are dimensions; these concepts name them.

⊆ Model concept

Parents:  MODEL CONCEPT

Children:  IMMUTABLECOORDINATE PROPERTIES  ORDER FOR CONCEPT ATTACHMENTS

 PATH FOR USER  LANGUAGE COORDINATE PROPERTIES  MODULE FOR USER

 LOGIC COORDINATE PROPERTIES  PATH COORDINATE PROPERTIES  ORDER FOR AXIOM ATTACHMENTS

 ORDER FOR DESCRIPTION ATTACHMENTS

Since: Inception

id: ViewCoordinateModel | UUID: f83c63a3-6d1d-55bf-a0ab-759079023592



Withdrawn state

Concept used to represent a status for components that are withdrawn.

↳ Status value

Parents: STATUS VALUE

Since: Inception

id: WithdrawnState | UUID: 35fd4750-6e43-5fa3-ba7f-f2ad376052bc

Last updated 2026-07-21 21:26:38 -0700

Colophon

This document was produced using the IKE Documentation Pipeline.

Rendering pipeline	AsciiDoc → HTML5 → PDF (Prince XML)
AsciiDoc backend	html5
Document date	2026-07-21
Build timestamp	2026-07-21T21:27:56.081009-07:00